

Tcl/Tk 기본 + α

5 Edition

(for Internet Distribution)

1999. 10. 11.

원나라

narawon@naver.com

<https://blog.naver.com/narawon/222056196990>

차 례

1.	들어가는 말.....	4
1.1.	책의 구성 및 이용법	4
1.2.	감사의 글	4
2.	소개.....	5
2.1.	TCL/Tk 와 스크립트 프로그램.....	5
2.2.	TCL/Tk 사용 대상	6
2.3.	TCL/Tk 의 설치	7
2.4.	TCL/Tk 맛보기	8
2.5.	ONLINE MANUAL 사용	10
2.5.1.	UNIX	10
2.5.2.	Windows	10
2.6.	TCL/Tk 자료	11
3.	TCL.....	12
3.1.	기본 문법	12
3.1.1.	한 줄에 한 명령어 씩	12
3.1.2.	주석 (comment).....	12
3.1.3.	명령어	13
3.1.4.	시스템 명령어	13
3.1.5.	변수	14
3.1.6.	수학식(Math Expression).....	16
3.1.7.	백슬래쉬 문자열(Backslash Sequence).....	18
3.1.8.	명령어의 nesting.....	19
3.1.9.	그룹핑(Grouping)과 변수 치환(Substitution).....	19
3.1.10.	찍어 쓰기	20
3.2.	제어구조.....	21
3.2.1.	if then else 구문	21
3.2.2.	switch 구문	22
3.2.3.	while 구문.....	23
3.2.4.	for 구문.....	23
3.2.5.	foreach 구문	23
3.2.6.	break, continue 명령어.....	25
3.2.7.	return 명령어.....	25
3.2.8.	catch 구문.....	25
3.2.9.	error 명령어.....	27
3.3.	문자열 조작	29
3.3.1.	Tcl 문자열에 대한 정의.....	29
3.3.2.	string 명령어.....	29
3.3.3.	format 명령어.....	32
3.3.4.	scan 명령어.....	34
3.3.5.	문자열 덧붙이기와 분할, 병합	35
3.4.	리스트(LIST)	36
3.4.1.	리스트의 생성	36
3.4.2.	리스트 추가 및 연결	37
3.4.3.	리스트 원소 얻기	38
3.4.4.	리스트 원소 삽입 및 치환	39
3.4.5.	리스트 검색과 정렬	39
3.5.	배열	42
3.5.1.	배열의 생성과 참조	42
3.5.2.	array 명령어.....	43

3.5.3.	배열을 이용한 복잡한 데이터 구조 구현	44
3.6.	정규 표현식	47
3.6.1.	정규 표현식 Primitives.....	47
3.6.2.	Backslash Escape.....	51
3.6.3.	regexp 명령어.....	52
3.6.4.	regsub 명령어.....	54
3.7.	함수	57
3.7.1.	함수의 정의	57
3.7.2.	함수의 사용	57
3.7.3.	Default Parameter Value	57
3.7.4.	Parameter 갯수의 유동성	58
3.7.5.	변수 참조 범위	59
3.8.	파일 및 디렉토리 조작	64
3.8.1.	OS 마다 다른 파일 경로 표기법.....	64
3.8.2.	파일 입출력	64
3.8.3.	file 명령어	68
3.8.4.	cd 와 pwd 명령어.....	69
3.9.	외부 프로그램과의 연동	71
3.9.1.	exec 명령어	71
3.9.2.	exit 명령어.....	72
3.10.	기타	73
3.10.1.	명령행 인수 (Command Line Argument).....	73
3.10.2.	환경 변수 (environment Variable).....	73
3.10.3.	incr 명령어.....	74
3.10.4.	console 명령어	74
3.10.5.	after 명령어.....	75
3.10.6.	시스템 변수들	76
3.11.	PACKAGE.....	77
3.11.1.	package 만들기.....	77
3.11.2.	package 사용 하기.....	78
4.	TK.....	80
4.1.	Tk 기본	80
4.1.1.	위젯(Widget).....	80
4.1.2.	위젯 클래스 (Widget Class).....	80
4.1.3.	위젯의 Instance Name.....	80
4.1.4.	위젯의 계층 구조	80
4.1.5.	Event Driven Programming	81
4.1.6.	위젯의 생성과 파괴	81
4.1.7.	위젯의 배치	81
4.1.8.	Widget Attributes.....	82
4.2.	GEOMETRY MANAGER	83
4.2.1.	pack 명령어.....	83
4.2.2.	Grid 명령어.....	87
4.2.3.	Place 명령어.....	89
4.2.4.	위젯 감추기	90
4.3.	BINDING COMMANDS TO EVENTS.....	91
4.3.1.	bind 명령어	91
4.3.2.	Binding Tags	92
4.4.	EVENT	94
4.4.1.	실제 이벤트	94
4.4.2.	가상이벤트	96
4.4.3.	이벤트 정보	97
4.5.	WIDGETS	99
4.5.1.	Frame 위젯.....	99
4.5.2.	Button 위젯.....	99

4.5.3.	Label 위젯.....	100
4.5.4.	Entry 위젯.....	100
4.5.5.	Message 위젯.....	102
4.5.6.	Scale 위젯.....	103
4.5.7.	listbox 위젯.....	103
4.5.8.	scrollbar 위젯.....	104
4.5.9.	toplevel.....	105
4.5.10.	Text 위젯.....	106
4.5.11.	canvas 위젯.....	108
4.5.12.	update 명령어.....	116
4.6.	DIALOGS.....	117
4.6.1.	Message Box 다이얼로그.....	117
4.6.2.	File 선택 다이얼로그.....	118
4.6.3.	Color 선택 다이얼로그.....	119
4.7.	MENU.....	120
4.8.	RESOURCE.....	122
4.8.1.	리소스 파일 로딩.....	122
4.8.2.	리소스 파일 형식.....	123
5.	관련 프로그램.....	124
5.1.	VTCL.....	124
5.2.	TCLPRO.....	124
6.	예제 프로그램.....	125
6.1.	프로그램의 목적 및 기능.....	125
6.2.	프로그램 실행 모습.....	125
6.3.	README 파일.....	126
6.4.	SOURCE FILES.....	129
6.4.1.	sramview.tcl.....	129
6.4.2.	gui.tcl.....	141
6.4.3.	sramview.resource.....	145
6.5.	SAMPLE DATA.....	146
6.5.1.	sample.srv.....	146
7.	연습문제 풀이.....	147
7.1.	구구단 출력 프로그램.....	147
7.2.	숫자 맞추기 게임.....	147
7.3.	데이터 분석 프로그램.....	148
7.4.	STACK.....	149
7.5.	QUEUE.....	149
7.6.	다차원 배열.....	151
7.7.	데이터 변환.....	151
7.8.	VERILOG 파일 분석.....	152
7.9.	총 라인 수 구하기.....	153
7.10.	PACKAGE 만들기.....	154
7.11.	숫자 맞추기 게임 GUI.....	157
7.12.	X SCROLLBAR.....	158
7.13.	구구단 GUI.....	158
7.14.	게임 프로그램.....	159
7.15.	FULL DOWN 메뉴.....	161

1. 들어가는 말

1.1. 책의 구성 및 이용법

본 교재는 현대전자 연수원 강의를 위하여 처음으로 개발되었습니다. 강의 교재로 개발되었던 만큼 부족하거나 오류가 있는 부분에 대해서는 강의 시에 수정 보완이 가능하였습니다. 그러나 이 문서를 막상 인터넷에 게재하려고 하니 수정 보완해야 할 부분이 너무 많습니다. 하지만 저자의 여건이 허락치 않아 이러한 내용에 대한 미처 다 보완 하지 못한 채 인터넷에 게재하게 되었습니다. 이 점 교재를 다룬 받아 활용하시는 모든 분들께 죄송스럽게 생각합니다. 여건이 허락하는 대로 강의 때 보충하려고 남겨둔 부분을 추가하여 배포할 것을 약속드립니다.

현재 Edition 까지 교재의 성격은 Tcl/Tk의 모든 부분을 속속들이 다루기 보다는, Tcl/Tk의 진수들만을 뽑아 보다 빠른 시간 안에 Tcl/Tk를 업무에 적용하여 업무 효율을 높일 수 있도록 하는데 주안점을 두고 있습니다. 따라서 본 교재 만으로는 Tcl/Tk를 완벽히 이해하는데는 한계가 있음을 밝힙니다.

특히, C 언어와의 인터페이스 부분을 다루지 못했다는 점은 매우 아쉬운 점입니다. 그리고 새버전에 추가된 내용인 Tcl/Tk의 지역화 (localization)¹에 대한 부분과 Multi Threading을 이용한 GUI 작성 부분을 다루지 못한 점 또한 매우 아쉬운 부분입니다.

오자, 탈자 및 내용 상의 오류를 없애기 위해 많이 노력하였으나 매번 강의하면서 한 두 개 씩 꼭 틀린 부분이 발견되었습니다. 발견될 때 마다 꾸준히 잘못된 것을 고쳐왔으나 아직도 이러한 것들이 남아 있는 것을 보면 현재 Edition 도 100% 완벽하다는 보장을 할 자신이 없습니다. 이 점 널리 양해해 주시면 감사하겠습니다.

그리고 이 책의 모든 예제의 모든 소스는 UNIX 환경에서 작성되고 테스트되었습니다. PC에서 본 교재를 이용하시는 분들은 이 점을 유의하시고 UNIX에서만 되고 PC에서는 실행되지 않는 프로그램이 있더라도 이해해 주시기 바랍니다. 이러한 경우는 UNIX 시스템 명령어를 사용하는 예제의 경우 발생합니다. 단, GUI 출력은 교재 작성을 제 PC에서 한 결과 PC에서의 출력 형태를 사용하였습니다.

1.2. 감사의 글

처음으로 Tcl/Tk 강의를 제안하시고 Tcl/Tk 교재 제작에 많은 격려와 용기를 주신 **배종홍** 과장님께 진심으로 감사 드립니다. 배종홍 과장님의 끈질긴(?) 독려가 아니었다면 이 교재는 존재할 수 없었을 것입니다.

이 책을 쓰기까지 옆에서 지켜봐 주시고 조언을 아끼지 않으셨던 **최영두** 차장님께 감사의 말씀을 올립니다. 그리고 항상 옆에서 책의 내용과 구성에 대하여 비판과 감수 역할을 해준 **홍영수** 대리님에게도 깊은 감사의 뜻을 전합니다. 이 분들이 없었다면 제 게으름으로 인하여 책을 모두 완성할 수 없었을 것입니다.

염철웅 대리님과 **김영훈**씨께서는 교재가 완성되고 강의를 개설하고 홍보를 하시고 강의 지원 등 너무나 많은 도움을 주시고 계십니다. 이 분들의 노고와 성의에 고개 숙여 감사 드립니다.

그리고 제가 처음으로 Tcl/Tk 공부를 시작할 때 “Practical Programming in Tcl and Tk” 책을 빌려주고, 제가 책을 거의 누더기를 만들었어도 싫은 소리 한번 하지 않았던 **한상현**씨와 **홍영수** 대리님에게도 다시 한번 감사를 드립니다. 이 분들 덕택에 저는 정작 Tcl/Tk 책을 한 권도 사지 않은 채로 이 교재를 완성할 수 있게 되었습니다.

아직도 미비하지만 부실했던 이전 교재를 읽어 보시고 부족한 제 강의를 들어 주셨던 많은 분들께 감사 드립니다. 이 분들께서 교재의 잘못된 부분을 찾아주시고, 여러 가지 예제 작성과 예제 아이디어를 제공해 주셨습니다.

¹ 기존 버전의 경우 영어만을 지원하였으나 최신 버전의 경우 다양한 언어를 지원하기 위한 여러 가지 기능들이 추가 되었다. 따라서 예전에는 불가능하던 한글 출력이 자연스럽게 해결되었다.

2. 소개

이 장에서는 Tcl/Tk 에 대한 기본 개념, 사용 용도에 대해서 이해한다. 그리고 Tcl/Tk 를 실행하기 위하여 필요한 환경을 구축하고 간단한 예제를 통하여 Tcl/Tk 를 체험한다. 마지막으로 Tcl/Tk 에 대한 보다 자세한 정보를 얻을 수 있는 참고 문헌과 웹 사이트를 소개한다.

2.1. Tcl/Tk²와 스크립트 프로그램

초기 스크립트 (Script) 프로그램은 여러 프로그램들을 명시된 순서대로 실행시켜주는 정도가 고작이었다. 따라서 특별히 스크립트 프로그래밍을 위한 별도의 언어가 존재하지 않았고, 단순히 파일 안에 실행되어야 할 프로그램들을 단순히 나열해 주는 정도에 지나지 않았다.

그러나 컴퓨터의 기능이 향상되어 감에 따라 사용자의 요구도 다양해지고 복잡해지기 시작했다. 따라서 프로그램들을 무조건 순차적으로 실행시키는 것만으로는 소기의 성과를 달성할 수 없어지자 스크립트 언어라는 개념이 도입되었다.

스크립트 언어의 가장 대표적인 예로 들 수 있는 것은 UNIX 의 C Shell 이다. C Shell Script 는 프로그래밍 언어에서 볼 수 있는 흐름 제어 구조(Control Flow Structure)를 도입하고 변수를 제공하여 경우에 따라 적절한 프로그램을 실행시키거나 특정 프로그램을 반복 실행하는 것을 가능하게 하였다.

오늘날 스크립트 언어는 발전에 발전을 거듭하여 프로그래밍 언어가 제공하는 거의 모든 기능을 제공하는 수준에 이르게 되었다. 오히려 프로그래밍 언어로 프로그램 한다면 수백 줄에 걸쳐서 기술 해야 할 것을 단 한 줄에 끝낼 수 있도록 해주는 등 프로그래밍 언어보다 강력한 기능을 제공하기에 이르렀다.

이처럼 강력한 기능을 제공하며 현재 각광 받고 있는 대표적인 차세대 스크립트 언어로 Tcl/Tk 와 Perl 을 들 수 있겠다. Tcl/Tk 는 셸 스크립트로 할 수 있는 모든 기능은 물론 보다 복잡한 파일 및 디렉토리 조작을 비롯하여 소켓을 이용한 네트워크 프로그래밍 등 예전에는 C 언어와 같은 프로그래밍 언어로만 가능했던 많은 부분을 스크립트로 손쉽게 작성할 수 있게 해준다. 그리고 C 언어로 작성된 함수들을 Tcl/Tk 내에서 사용할 수 있도록 C 언어 인터페이스를 제공하고 있다. 이에 더하여 강력한 문자열 처리 기능을 겸비하여 문서의 검색, 치환, 관리 프로그램을 손쉽게 작성할 수 있도록 해준다. Perl 도 이러한 많은 기능을 제공하고 있고 어떤 면에서는 Tcl/Tk 보다 좋은 장점이 있지만, Tcl/Tk 는 그래픽 유저 인터페이스를 간단하고 손쉽게 작성할 수 있도록 해줌으로써 Perl 보다 진보된 스크립트 언어의 지위를 굳히게 되었다.

대체로 스크립트 언어들은 인터프리터 방식으로 프로그램을 수행하기 때문에 실행 속도가 프로그래밍 언어로 개발된 프로그램 보다 느리고 소스 코드를 사용자에게 노출시켜야 하기 때문에 상용 툴을 개발하는 도구로는 적합하지 않다고 지적되어 왔다. 하지만 Tcl/Tk 의 경우 인터프리터는 물론 컴파일러까지 개발되어 이러한 선입관을 불식시키기에 이르렀다.

게다가 UNIX 계열 운영체제에서만 사용 가능하던 Tcl/Tk 가 Windows 9x/NT 가 발표되면서 Windows 9x/NT 로도 이식되고 매킨토시로도 이식되어 이제는 UNIX 사용자 뿐만 아니라 PC 사용자들도 Tcl/Tk 의 강력한 기능을 마음껏 누릴 수 있게 되었다.

이러한 Tcl/Tk 의 장점이 널리 인식되어 현재에도 많은 EDA 툴들이 Tcl/Tk 를 기본 인터페이스로 채택하고 있으며, 앞으로는 거의 모든 EDA 툴을 비롯하여 많은 CAD 툴들이 Tcl/Tk 를 기본 인터페이스로 채택할 것으로 전망된다.

Tcl/Tk 는 현재도 계속 버전업되고 있으며, 각종 신개념들을 접목하여 보다 유용하고 편리한 스크립트 언어로 발전하기 위한 노력들이 계속 되고 있다.

² Tcl/Tk 는 “티클 티케이”라고 읽는다.

2.2. Tcl/Tk 사용 대상

인터넷상에서 CGI 를 개발하는 사람들에게는 Tcl/Tk 보다는 Perl 을 권하고 싶다. 인터넷 CGI 경우 굳이 소스를 컴파일할 필요도 없을 뿐 아니라 GUI 는 HTML 을 이용하여 작성하게 되므로 보다 문법이 간단한 Perl 을 사용하여 개발하는 편이 여러 가지 면에서 장점이 있다. 실제로 인터넷 CGI 개발자들 사이에서는 Tcl/Tk 는 구시대적이고 사라져야 할 스크립트 언어로 알려져있는 것이 사실이다. 하지만 Perl 로 CGI 가 아닌 독립적인 프로그램을 작성하는 경우 GUI 를 작성할 수 없다는 점이 큰 단점으로 부각된다. 근래에 Perl/Tk 와 같이 GUI 라이브러리를 도입하여 Perl 에서도 GUI 를 작성할 수 있도록 하려는 노력이 시도되고 있으나 Perl 언어 자체가 GUI 를 염두에 두고 개발된 것이 아니기 때문에 GUI 작성에 각종 문제가 발생한다.

GUI 를 구현하기 위해서는 Tcl/Tk 가 아니더라도 Visual C++, Visual Basic, Delphi, X Window Library, Motif 등과 같은 툴들을 사용할 수 있다. 하지만 이러한 툴들을 사용하여 프로그램을 개발하면 개발된 프로그램이 한가지 OS 상에서 밖에 수행되지 않는다는 단 점이 있다. 따라서 다양한 고객의 요구를 신속히 수용해야 하는 경우, 특히 고객들이 사용하는 OS 의 종류가 다양한 경우 치명적인 단 점으로 작용할 수 있다.

OS 를 가리지 않고 거의 모든 OS 상에서 동작하면서 GUI 를 쉽게 구현하기 위해 Java 를 사용하여 프로그램을 개발할 수 있다. 하지만 Java 프로그램은 Applet 이 아닌 Application 형태로 개발되었다하더라도 반드시 Java Virtual Machine 을 설치한 후에만 실행될 수 있다. 따라서 독립적으로 실행되는 프로그램을 작성하고자 하는 개발자에게는 큰 제약으로 작용하게 된다.

새롭게 부상하고 있는 Phyton 과 같은 제 3 세대 스크립트 언어를 사용하여 프로그램을 작성할 수도 있다. 이러한 언어들은 기본적으로 GUI 를 제공하고 있고, Object Oriented Programming 과 같은 각종 신개념을 도입하여 프로그램 개발을 보다 체계적으로 할 수 있다는 장점이 있다. 하지만 최근에 개발된 만큼 아직 이식되지 않는 OS 들이 많고 컴파일러가 구비되지 않았다는 단점이 있다.

이러한 모든 단점을 극복한 해결책으로는 1999 년 10 월 현재 Tcl/Tk 가 유일하다. Tcl/Tk 는 처음 개발될 당시 부터 GUI 제공을 염두에 두고 개발 되었으며, 현재 안정적인 컴파일러를 제공³하고 있다. 또한 UNIX, Linux, Windows, Mac OS, OS/2, BeOS 등 현재 널리 사용되고 있는 거의 모든 OS 에 이미 포팅 완료되어 있기때문에 프로그램 소스를 한번만 작성하고 나면 Tcl/Tk 가 포팅된 모든 OS 에서 작성된 프로그램을 수행할 수 있다는 장점이 있다. 따라서 다양한 OS 에서 수행되고, 다른 툴의 도움없이 독립적으로 실행가능하며, GUI 를 제공하며, 컴파일하여 배포해야하는 프로그램을 개발해야 하는 개발자의 입장에서 는 Tcl/Tk 가 최선의 선택이 될 수밖에 없다.

³ Perl 의 컴파일러는 공개 소프트웨어 형태로 개발되고 있다. 1999 년 9 월 현재까지는 프로그램이 좀 복잡해지면 컴파일이 안 되는 문제가 있다. Tcl/Tk 의 컴파일러는 상용 프로그램으로 판매되고 있으나 그 가격이 비교적 저렴하고, 복잡한 프로그램에 대해서도 컴파일에 문제가 없다. 그리고 Tcl/Tk 컴파일러 라이선스 하나를 구입하면 각종 OS 용 컴파일러를 모두 준다.

2.3. Tcl/Tk 의 설치

Tcl (Tool Command Language)은 스크립트 언어와 이를 해석하여 실행시켜주는 인터프리터로 구성된다. 그리고 Tk (Tool Kit)는 Tcl 을 이용하여 GUI 를 손쉽게 구성할 수 있는 Tool Kit 을 제공한다.

Tcl/Tk 의 버전은 1999 년 10 월 현재 8.2 버전이 가장 최신 버전이다. 이전 버전과 호환성을 유지하고 있으므로 보다 향상 된 Tcl/Tk 의 기능을 활용하기 위해서 테스트가 완벽하게 끝난 가장 최신 버전을 사용하는 것이 좋다. Tcl/Tk Version 8.2 는 인터넷 상에서 무료로 손쉽게 구할 수 있다. 단, 컴파일러는 한달간 무료로 사용해볼 수 있으며, 영구적인 사용을 위해서는 구매를 해야 한다.

Tcl/Tk 를 구할 수 있는 곳은 다음과 같다. 여기에는 설치에 필요한 자세한 내용도 함께 수록되어 있다.

<http://www.scriptics.com>

위의 홈 페이지에서 사용하고자 하는 운영체제에 맞는 파일을 다운 받아 설치 한다. 각 운영체제에 맞게 미리 컴파일 되어 있는 설치 파일을 다운 받으면 설치가 간편하다. 소스를 다운 받아 컴파일 후 설치를 원한다면 컴파일에 필요한 여러 가지 툴들이 더 필요할 수 있다. UNIX 의 경우 X Window 와 같은 그래픽 처리를 위한 하드웨어 및 소프트웨어가 설치되어 있지 않으면 Tk 를 이용한 GUI 구현이 불가능하다.

Sun OS, Solaris 와 같이 Sun 계열의 OS 를 사용하는 경우에는 Sun 사에서 자사의 OS 상에서 잘 실행될 수 있도록 미리 컴파일하여 배포하고 있다. root 권한을 가진 사람이라면 아래의 사이트에서 파일을 다운 받아 pkgadd 명령어를 사용하여 한 번에 Tcl/Tk 를 설치할 수 있다. Sun 이외의 다른 UNIX 계열의 OS 에서는 직접 소스를 컴파일하여야 한다.

<http://www.sunfreeware.com>

Windows 나 매킨토시 사용자의 경우는 셋업파일을 다운받아 이를 실행시키기만 하면 셋업이 완료된다.

설치가 끝나면 Tcl/Tk 인터프리터가 설치되는데 여기에는 tclsh 와 wish 의 2 가지가 있다. wish 는 GUI 를 구현하기 위하여 Tk 를 사용하는 경우에 이용되는 인터프리터이고, tclsh 는 Tk 를 사용하지 않고 Text Mode 의 프로그램을 작성하는 경우에 사용된다. Tk 를 이용하지 않고 단순히 스크립트만 프로그램할 경우에는, 즉 다시 말해서 그래픽 윈도우가 필요 없는 경우에는 사용하지 않는 빈 윈도우가 화면에 나타나는 것을 막기 위해 tclsh 를 이용한다.

UNIX 일 경우 두 실행 파일의 이름은 wish, tclsh 로 정해져 있으나, Windows 의 경우에는 버전을 명시하여 wish82.exe, tclsh82.exe 로 설치된다.

2.4. Tcl/Tk 맛보기

먼저 Tcl 을 이용하여 “Hello, World!”라는 문장을 화면에 찍어 보자. 적당한 에디터를 열어 다음과 같이 입력하고 저장한다. Windows 의 경우 첫번째 줄은 입력할 필요가 없다. 하지만 향후에 똑 같은 소스를 이용해 UNIX 에서도 실행할 수 있도록 하기 위해서는 항상 첫번째 라인을 써 주도록 하는 편이 좋다. 그리고 Tcl/Tk 가 설치된 경로가 달라서 tclsh 가 존재하는 위치가 /usr/local/bin 이 아닌 경우 적절히 tclsh 실행 파일이 있는 위치를 지정해 주어야 한다. 지금 예제는 Tk 를 전혀 사용하지 않으므로 wish 를 사용하지 않고 tclsh 를 사용한다.

```
#!/usr/local/bin/tclsh
puts "Hello, World!"
```

이 파일을 “hello.tcl”이라고 저장했다고 가정한다.

UNIX 의 경우 실행하는 방법은 다음과 같다.

1. 저장한 파일을 실행 가능 모드로 바꾼다.
`$ chmod +x hello.tcl`
2. 저장한 파일을 실행한다.
`$ hello.tcl`
Hello, World!
`$`

Windows 의 경우 실행하는 방법은 다음과 같다.

1. DOS Box 를 연다.
2. 저장한 파일을 실행한다.
`C:> tclsh82 hello.tcl`
Hello, World!
`C:>`
3. 실행이 안될 경우 tclsh82.exe 의 path 가 제대로 설정되어 있는 지 확인한다.
4. 윈도우 탐색기에서 파일을 2 번 클릭해도 된다.

다음에는 Tk 를 이용하여 “Hello”라는 라벨을 가진 버튼을 만들어 보자. Tk 를 이용하기 위해 tclsh 를 사용하지 않고 wish 를 사용했다는 점에 유의해야 한다.

```
#!/usr/local/bin/wish
button .hello -text Hello -command { puts "Hello, World!" }
pack .hello
```

UNIX 의 경우는 앞서와 마찬가지로 실행시키면 된다. Windows 의 경우 tclsh82 대신 wish82 을 실행시켜야 한다. 이 프로그램의 실행 결과는 그림 1과 같다.



그림 1 Tk 맛보기

만들어진 버튼에 마우스를 가져가 눌러 보면 버튼을 누를 때 마다 “Hello, World!”가 출력될 것이다. UNIX 의 경우에는 명령을 입력하던 창에 출력될 것이고, Windows 의 경우에는 Text 출력을 위해 따로 만들어진 윈도우에 출력될 것이다.

단 세 줄만으로 이처럼 훌륭하게 작동하는 버튼을 만들었다는 것이 놀랍지 않은가? 만약 C 언어로 프

로그를 했다면 단순히 버튼 하나를 만들기 위해 수백 줄에 이르는 복잡한 프로그램이 필요하다. 바로 이것이 Tcl/Tk의 매력이다.

위의 예제와 같이 몇 줄 안되는 간단한 예제의 경우는 따로 파일에 저장할 필요 없이 인터프리터 상에서 바로 입력해서 테스트해 볼 수도 있다. `tclsh` 나 `wish` 를 실행하면 인터프리터가 실행되고 사용자가 명령을 입력할 수 있도록 `prompt` 가 나타난다. 여기에 위의 내용을 입력하여 동작을 확인해 볼 수 있다.

먼저 `tclsh` 를 실행하고 다음과 같이 입력해 보자. 인터프리터를 빠져 나오기 위해서는 `exit` 이라고 입력한다.

```
% puts "Hello, World!"  
Hello, World!  
% exit
```

이번에는 `wish` 를 실행하고 다음과 같이 입력해 보자. `wish` 를 실행하면 아무 것도 없는 빈 윈도우가 화면에 나타날 것이다. 첫번째 줄을 입력할 때까지는 아무 변화가 없다가 두 번째 줄을 입력하고 나면 빈 윈도우의 크기가 적절히 바뀌고 버튼이 나타날 것이다. 마우스로 버튼을 눌러 보면서 동작을 확인한 후에 `exit` 을 입력하고 인터프리터를 종료한다.

```
% button .hello -text Hello -command { puts "Hello, World!" }  
% pack .hello  
% exit
```

2.5. Online Manual 사용

본 교재에서는 Tcl/Tk 에 대한 기본 개념을 잡기 위해 필요한 내용만을 수록하였다. 각 명령어마다 더 자세한 옵션과 내용을 알고 싶다면 Online Manual 을 이용하는 것이 가장 빠르고 확실한 방법이다.

2.5.1. UNIX

UNIX 에서는 Tcl/Tk 의 매뉴얼까지 인스톨 하였다면 다른 일반 명령어들의 매뉴얼과 마찬가지로 `man` 을 이용하여 Tcl Online 매뉴얼을 이용할 수 있다. 단 표준 C 언어 라이브러리에서 제공하는 함수와 이름이 같을 경우에는 우선적으로 C 언어 라이브러리 함수에 대한 매뉴얼을 찾아 주기 때문에 `man` 실행 시 `-sn` 옵션을 명시하여야 Tcl 명령어에 대한 매뉴얼을 찾아 준다.

맞보기 예제에서 사용한 명령어 `puts` 의 매뉴얼을 보기 위해서는 UNIX 프롬프트에서 다음과 같이 입력 한다.

```
$ man -sn puts
```

만일 Online 매뉴얼이 제대로 실행되지 않는 경우 `MAN_PATH` 라는 환경 변수가 제대로 설정되어 있는지 확인한다. `MAN_PATH` 라는 환경 변수가 설정되어 있지 않은 경우 `.cshrc` 파일에 다음과 같은 라인을 추가하도록 한다.

```
setenv MAN_PATH /usr/local/man
```

2.5.2. Windows

인스톨이 완료된 후 시작버튼을 눌러 보면 Program Group 에 Tcl 그룹이 생겨 있다. 여기에 Tcl Help 아이콘이 있다. 이 아이콘을 선택하여 실행하면 Tcl 매뉴얼을 참조할 수 있다. Help 프로그램은 Windows 의 다른 프로그램들에서 제공하는 Help 와 같은 프로그램을 사용하고 있으므로 사용법은 금방 알 수 있을 것이다. Help 프로그램의 검색 기능을 사용하거나 매뉴얼의 색인을 이용하면 쉽게 알고자 하는 명령어에 대한 Reference 를 볼 수 있다.

2.6. Tcl/Tk 자료

Tcl/Tk 를 보다 심도 깊게 공부하기 위한 자료들을 소개하자면 다음과 같다.

책	Practical Programming in Tcl and Tk 2nd Edition ⁴ Brent B. Welch 저 Prentice Hall
	CGI Developers Resource, Web Programming with Tcl and Perl John Ivler Prentice Hall
	Tcl/Tk Tools Mark Harrison 저 O'Reilly & Associates, Inc
WWW	http://www.scriptics.com http://www.yahoo.com/Computers_and_Internet/Programming_Languages/Tcl_Tk
News Group	comp.lang.tcl

⁴ 번역서는 번역에 문제가 많으므로 될 수 있으면 원서를 구입할 것을 권한다.

3. Tcl

스크립트 언어인 Tcl 의 문법을 이해한다. 그리고 프로그램 개발의 편의를 도모하기 위해 Tcl 이 제공하는 여러 가지 기능들을 간단히 살펴 본다.

3.1. 기본 문법

Tcl/Tk 를 사용하기 위해 기본적으로 알아야 할 문법들에 관해서 설명 한다. Tcl/Tk 의 프로그래밍 스타일, 주석, 변수, 명령어, 수학적, 변수 치환, 명령어 그룹핑 등에 대하여 알아 본다.

3.1.1. 한 줄에 한 명령어 씩

Tcl 인터프리터는 한 줄에 한 명령어씩만 나오는 것으로 생각한다. 그리고 모든 명령어는 반드시 한 줄 안에 기술되어야 한다.

그러나 프로그램을 하다 보면 한 줄에 여러 개의 명령어를 쓰는 것이 이해에 도움을 주는 경우가 있다. 이 때에는 “ ; ”를 사용하여 각 명령어들을 구분시켜 줄으로써 한 줄에 여러 개의 명령어를 사용할 수 있다.

또 명령어가 길어져 여러 줄에 걸쳐서 기술해야 할 경우에는 “ \ ” 기호를 줄의 맨 마지막에 붙여서 다음 줄이 계속 한 명령어에 해당한다는 것을 인터프리터에 알려 줄 수 있다. 이 때 주의해야 할 점은 “ \ ” 기호 다음에 스페이스나 탭과 같은 공백 문자가 와서는 절대 안된다는 점이다. 이 방법 외에도 “ { } ”를 사용해서 여러 줄에 걸쳐 명령어를 기술할 수 있다. 중괄호를 이용한 그룹핑에 대한 자세한 설명은 3.1.9절에 나와 있다.

```
% set a 5; puts "a = $a"
a = 5
% set \
a 5
5
```

3.1.2. 주석 (comment)

Tcl 인터프리터는 줄의 시작에 “ # ” 기호가 나타나면 그 줄 전체를 comment 로 간주하고 무시한다. 줄의 시작점이 아닌 곳에 “ # ” 기호가 나오면 주석으로 간주하지 않는다.

줄의 중간에 주석을 넣기 위해서는 “ ; ”를 이용하면 된다. 그리고 여러 줄에 걸친 주석을 쓰기 위해서는 “ \ ”를 이용하면 된다. 백슬래쉬의 이런 용도에 대한 자세한 설명은 3.1.7절을 참조하기 바란다.

```
# This is one line comment

set a 5; # To use comment in the middle of a line, Use " ; "

# You can use multiline comment \
like this with "\"

# comment 는 한글로 해도 된다.
# '#' 의 위치는 반드시 첫번째 컬럼에 와야 한다.
# 지금과 같이 '#' 이 첫번째 컬럼이 아닌 곳에 오면 문법 오류가 발생한다.

if { $x < 0 } {
    # 지금과 같이 중괄호 속에 존재하는 주석은 '#' 이 첫번째 컬럼에 오지 않아도 된다.
}
```

주석을 사용할 때 특히 주의해야 할 것은 주석은 반드시 명령어가 올 자리에 와야 한다는 점이다. “{ }”를 사용한 복잡한 제어 명령의 경우 이 원칙을 지키는 것이 매우 중요하다. switch 문을 설명할 때 좀 더 자세히 설명하겠다.

3.1.3. 명령어

Tcl 의 명령어를 사용하는 방법은 다음과 같다.

```
command argument1 argument2 argument3 ...
```

첫번째로 명령어의 이름이 나오고 그 다음에 명령어로 전해 줄 Argument 들이 나열 된다. 명령어와 각 Argument 들의 구분은 Space 나 Tab 과 같은 공백문자로 이루어 진다. Tcl 명령어에는 Tcl 인터프리터 자체에서 제공하는 Built-In 명령어와 사용자가 정의한 함수들이 포함된다. Tcl 함수를 정의하고 사용하는 방법에 대해서는 “3.7함수”에서 자세히 설명할 것이다.

다음과 같은 Tcl 명령이 있을 때

```
set a 5
```

이를 분석해 보면 명령어는 “set”이고 첫번째 Argument 는 “a”이며 두 번째 Argument 는 “5”이다.

3.1.4. 시스템 명령어

Tcl/Tk 인터프리터는 C Shell 스크립트에서 할 수 있는 모든 기능을 기본적으로 제공한다. 따라서 Tcl/Tk 인터프리터에서는 UNIX 와 C Shell 이 기본적으로 제공하는 모든 명령어를 사용할 수 있다. 그리고 UNIX prompt 상에서 수행할 수 있는 모든 명령어는 Tcl/Tk 인터프리터 상에서도 수행할 수 있으며 스크립트 내에서도 마찬가지이다.

그러나, Tcl/Tk 인터프리터 프롬프트 상에서는 바로 시스템 명령어를 실행할 수 있지만 스크립트에서는 바로 실행할 수 없다. 시스템 명령어를 스크립트 안에서 실행하려면 exec 명령어를 사용해야 한다는 점에 유의하기 바란다.⁵

이러한 내용은 Windows 95/NT 나 매킨토시에서도 마찬가지이다. 하지만 OS 에 따라서 제공하는 시스템 명령어의 이름과 옵션이 다르다. 따라서 여러 가지 OS 에서 사용할 Tcl/Tk 스크립트 내에서는 시스템 명령어를 사용해서는 안된다. 굳이 사용해야 한다면 tcl_platform(platform)이라는 변수의 값을 확인하여 OS 의 종류를 알아낸 다음 OS 에 맞게 적절한 명령어를 사용하도록 프로그램해야 한다.

인터프리터 상에서 시스템 명령어를 사용하는 예는 다음과 같다.

```
% ls -al
total 25
drwxrwxrwx  2 wonnr      512 Aug 21 11:39 .
drwxrwxrwx  4 wonnr      512 Aug 21 11:40 ..
-rwxrwxrwx  1 wonnr      386 Aug 21 10:38 SRAMView.resource
-rw-rw-rw-  1 wonnr     5740 Aug 21 11:37 readme.txt
-rw-rw-rw-  1 wonnr      373 Aug 21 11:37 sample.srv
-rw-rw-rw-  1 wonnr      386 Aug 21 11:37 sramView.resource
-rwxrwxrwx  1 wonnr    13820 Aug 21 11:37 sramView.tcl
%
```

스크립트 안에서 시스템 명령어를 사용하는 예는 다음과 같다.

```
#!/usr/local/bin/tclsh

exec ls -al
```

⁵ exec 명령어에 대한 설명은 “3.9.1exec”절에 있다.

3.1.5. 변수

Tcl 에서 변수 이름에 사용될 수 있는 글자는 `alphabet` 과 숫자, 밑줄 문자(`_`)만이 가능하다고 알고 있는 편이 좋다. 다른 기호도 변수 이름에 사용될 수 있으나 이렇게 할 경우 소스 코드를 이해하기 어렵게 만들고 예기치 않은 논리적 에러를 발생시킬 확률이 높아 별로 권장하고 싶지 않다. 그리고 변수 이름은 숫자로 시작되면 안된다.

변수에 값을 할당하기 위해서는 `set` 명령어를 사용 한다. C 언어와 같이 “ = ”를 사용하여 변수 값을 할당할 수 없다는 점에 유의하기 바란다.

`set variableName ?variableValue?`⁶

`set` 명령어는 주어진 변수에 값을 할당하고 할당된 값을 리턴한다. `Set` 명령어에 `Argument` 로 변수 이름만 주면 그 변수의 값을 리턴한다.

변수의 값을 참조하기 위해서는 변수 이름 앞에 “\$”를 붙이면 된다.

```
% set a 5
5
% set a
5
% set b $a
5
```

경우에 따라서 변수의 내용에 `evaluation` 가능한 형태가 저장되어 있어 이를 막을 방법이 필요할 경우가 있다. 이러한 대표적인 경우가 다음과 같이 변수의 내용에 정규 표현식이 포함된 경우이다. 다음은 정규표현식에 사용될 패턴을 변수에 저장하고 사용하는 방법을 보여 주고 있다. 정규 표현식에 관해서는 “3.6정규 표현식”에서 보다 자세히 설명할 것이다.

```
% set a {[a-z0-9]bc}
[a-z0-9]bc
% set b $a
error
% set b ${a}
[a-z0-9]bc
```

Tcl 에서 변수는 C 언어에서처럼 미리 선언되어야 할 필요는 없다. `set` 명령어에 의해 변수 값이 할당되는 순간 변수가 생성된다. 사용하지 않는 변수를 없애기 위해서는 `unset` 명령어를 사용하면 된다. `unset` 명령어는 이미 존재하는 명령어에 한해서만 사용할 수 있다. 존재하지 않는 변수에 `unset` 명령어를 사용하면 다음과 같은 에러가 발생한다. 규모가 그리 크지 않은 프로그램에서는 `unset` 을 사용하여 변수를 없애지 않아도 큰 문제는 없다.

```
% set a
5
% unset a
% set a
can't read "a": no such variable
% unset c
can't read "c": no such variable
```

⁶ 문법 표시에서 “?”로 싸여진 부분은 생략해도 좋다는 표시이다.

변수가 존재하는지 없는지를 알아보기 위해서는 `info` 명령어에 첫번째 `Argument` 로 `exists` 를 주고 두번째 `Argument` 로 존재 여부를 알아보기 위한 변수 이름을 주면 된다. 존재하는 경우 1 이 리턴되고 존재하지 않는 경우 0 이 리턴된다.

```
% set aa 2
2
% info exists aa
1
% info exists bb
0
```

Tcl 변수는 “”안에서 자유롭게 치환되므로 다음과 같이 사용할 수 있다.

```
% set a 2
2
% puts "a is $a"
a is 2
```

다음과 같이 변수에 글자를 바로 붙여 써야할 경우에는 인터프리터가 변수 이름을 잘못 해석하는 수가 있다. 다음의 예제에서 “1st”와 같이 출력하려고 하는 것이지만 실제로 실행해 보면 에러가 난다. 이는 인터프리터가 변수의 이름을 “a”라고 인식하지 못하고 “ast”라고 인식했기 때문이다.

```
% set a 1
1
% puts "$ast"
error
```

이런 경우에는 다음과 같이 2 가지 해결책이 있다. 하나는 { }를 사용하는 방법이고, 다른 하나는 “\”를 사용하는 방법이다.

```
% set a 1
1
% puts "${a}st"
1st
% puts "$a\st"
1st
```

Tcl 변수에는 미리 정해진 형(Type)이 없다. 할당되는 값에 따라 변수의 형은 자동으로 바뀐다. 다시 말해서 정수형, 실수형 등과 같은 변수형이 없고 변수에 할당되는 값에 따라 적절한 형으로 바뀐다는 의미이다. 다음 예제에서 변수 “a”의 다양한 변신을 음미해 보기 바란다.

```
% set a 2
2
% set a hahaha
hahaha
% set a 1.2
1.2
% set a "This is string"
This is string
% set a { one two three }
one two three
```


3.1.6. 수학적식(Math Expression)

Tcl 에서 수학적식은 `expr` 명령어를 사용하여 계산할 수 있다. 수학적식의 표현 방법은 C 언어와 유사하다.

```
% expr 2 + 3
5
% expr ( 3 + 3 ) / 2
3
% expr cos(0)7
1.0
% set x 3
3
% expr ( $x < 5 )? 1 : 2
1
```

연산자의 종류와 우선 순위를 정리하면 표 1과 같다. 우선 순위는 위쪽으로 갈 수록 높고 밑쪽으로 갈 수록 낮다.

연산자	의미
- ~ !	음수 부호, bitwise NOT, logical NOT
* / %	곱하기, 나누기, 나머지
+ -	더하기, 빼기
<< >>	왼쪽 Shift, 오른쪽 Shift
< > <= >=	작다, 크다, 작거나 같다, 크거나 같다
== !=	같다, 다르다
&	bitwise AND
^	bitwise XOR
	bitwise OR
&&	logical AND
	logical OR
x ? y : z ⁸	If x then y else z

표 1 연산자의 종류와 우선순위

⁷ 삼각함수에서 각도는 라디안 각으로 주어져야 한다.

⁸ 반드시 `expr` 과 함께 사용되어야 한다.

Tcl 에서 제공하는 수학 함수들을 표 2에 정리하였다.

수학함수	의미
acos(x)	arc-cosine
asin(x)	arc-sine
atan(x)	arc-tangent
atan2(y, x)	(r, th)극에 대한 사각형 (x, y), atan2 는 th 의 값을 계산 함
ceil(x)	x 와 같거나 큰 정수 중 가장 작은 수
cos(x)	cosine
cosh(x)	hyper cosine
exp(x)	자연 대수의 x 승
floor(x)	x 와 같거나 작은 정수 중 가장 큰 수
fmod(x,y)	실수의 나머지 연산
hypot(x,y)	$\sqrt{x^2 + y^2}$
log(x)	로그, 밑이 자연 대수
log10(x)	로그, 밑이 10
pow(x,y)	x 의 y 승
sin(x)	sine
sinh(x)	hyper sine
sqrt(x)	제곱근
tan(x)	tangent
tanh(x)	hyper tangent
abs(x)	절대값
double(x)	실수로 변환 함
int(x)	정수로 변환, 소수점 이하 버림
round(x)	정수로 변환, 소수점 이하 반올림

표 2 Tcl Built-in 수학 함수

3.1.7. 백슬래쉬 문자열(Backslash Sequence)

백슬래쉬 다음에 오는 글자는 원래 의미를 잃어버리고 다른 의미로 사용된다. 가장 유용한 경우는 Tcl 에서 의미있는 연산자를 기호로 사용하는 경우이다. 다음에 그 예제가 있다.

```
% puts "[test]"
child process exited abnormally
% puts "\\[test\\] "
[test]
```

그 외에 Tcl 에서 정의한 백슬래쉬 문자열들이 있다. 정리하면 다음과 같다.

백슬래쉬 문자열	의미
\a	Bell (0x7)
\b	Backspace (0x8)
\f	Form Feed (0xC)
\n	New Line (0xA)
\r	Carriage return (0xD)
\t	Tab (0x9)
\v	Vertical Tab (0xB)
\<new line>	개행 문자가 백슬래쉬 바로 다음에 오면 다음 라인을 한 줄로 합침.한 명령어나 주석을 여러 줄에 걸쳐 사용할 때 유용한 기능임.
\\	Backslash 자체를 기호로 사용하기 위해
\0	8 진수 (puts "\041" 을 수행하면 '!'가 찍힘)
\x	16 진수 (puts "\x61"을 수행하면 'a'가 찍힘)
\\$	\$
\"	"
\{	{
\}	}
\[	[
\]	]

표 3 백슬래쉬 문자열

3.1.8. 명령어의 nesting

명령어의 Argument 로 다른 명령어의 리턴 값이 들어갈 경우와 수학적 내에서 다른 명령어의 결과를 이용해야 할 경우 “[]”를 써서 명령어를 nesting 시킬 수 있다. Tcl 인터프리터는 “[]”를 만나면 “[]”내부에 있는 내용을 Tcl 명령어로 보고 이를 실행시킨 후 그 결과를 “[]”가 있던 자리에 대치시킨다.

```
% set a [ expr 3 + 2 ]
5
% set a [ expr 4 + [ expr 2 + 2 ] ]
8
% puts " 7 + 3 = [ expr 7 + 3 ] "
7 + 3 = 10
```

“[]”를 명령어 자리에 사용하여 함수의 결과로 리턴되는 값을 명령어처럼 이용할 수도 있다. 다음에 그 예제가 있다. 프로그램 작성에 실제로 사용되는 경우는 거의 없겠지만 다음의 예제와 같은 프로그램도 작성할 수 있다. 함수의 작성 방법은 “3.7함수”에서 보다 자세히 설명할 것이다.

```
% proc findCommand { what } {
    if { $what == 1 } {
        return set
    }
    return puts
}
% [findCommand 1] a 3
3
% [findCommand 0] "Hello! World"
Hello! World
```

3.1.9. 그룹핑(Grouping)과 변수 치환(Substitution)

Tcl 에서 Argument 의 구분은 공백문자에 의해서 구분되는데 명령어에 넘겨줘야 할 Argument 에 공백이 있을 경우 어떻게 해야할까? 이를 위해서 제공되는 것이 중괄호와 따옴표이다. 중괄호와 따옴표의 차이점은 중괄호는 변수 치환을 허용하지 않는 반면 따옴표는 변수 치환을 허용한다는 점이다. 다음의 예제를 통하여 이것을 쉽게 확인할 수 있을 것이다. 아래 두 경우 모두 명령어 puts 에 전달되는 Argument 는 그룹핑에 의해 하나 밖에 없음에 유의해야 한다.

```
% set a 5
% puts { a = $a }
a = $a
% puts " a = $a "
a = 5
```

중괄호와 따옴표는 그 사이에 오는 모든 문자를 그룹핑 해 준다. 따라서 중괄호와 따옴표를 이용해 여러 줄에 걸친 명령어를 기술할 수도 있다. Tcl 인터프리터는 여는 중괄호를 만나면 닫는 중괄호가 나올 때까지 라인이 끝나지 않은 것으로 간주하고 계속 읽어들인다. 따옴표의 경우도 마찬가지 이다.

```
% set a {
2
}
2
% puts "Multiple line
demo
haha "
Multiple line
demo
haha
```

3.1.10. 띄어 쓰기

Tcl/Tk 는 띄어 쓰기에 좀 민감한 편이다. 특히 명령어 그룹핑에 사용되는 “ { } ”와 명령어 nesting 에 사용되는 “ [] ”를 사용할 때는 기호들 사이에 반드시 띄어 쓰기를 한다고 기억해 두면 좋다. 특히 닫는 괄호와 여는 괄호를 붙여서 사용한 경우에는 에러가 발생한다.

문법적으로 오류가 없는데도 불구하고 오류가 발생할 경우에는 닫는 괄호와 여는 괄호가 붙어 있는 경우가 없는지 확인해 보기 바란다. 다음의 예들은 자주 발생하는 실수이므로 오류의 원인을 확인해 보기 바란다.

```
# This code has error
if { $value >= 0 }{
```

```
# This is valid code
if { $value >= 0 } {
```

그리고 여는 중괄호가 키워드와 붙여 써지지 않는지 확인해 보기 바란다.

```
# This code has error
if{ $value >= 0 } {
```

```
# This is valid code
if { $value >= 0 } {
```

다음과 같이 닫는 사각 괄호가 맨 끝의 Argument 와 붙어 있는 경우 서브루틴으로 닫는 사각 괄호도 함께 넘어가는 수가 있다.

```
# This code has potential logical error
puts [ MySubFunc arg1 arg2]
```

```
# This is valid code
puts [ MySubFunc arg1 arg2 ]
```

3.2. 제어구조

Tcl 에서 제어구조 문법은 프로그램의 흐름을 제어하는 기능을 한다. 다른 프로그래밍 언어나 셸 스크립트 언어에 익숙하다면 그 의미를 쉽게 파악할 수 있을 것이므로 문법과 간단한 예제만으로 설명을 대신하기로 한다.

3.2.1. if then else 구문

```
if booleanExp ?then? if_body ?else else_body?
```

```
if {$x == 0} {  
    puts "Zero Value"  
} else {  
    puts "Nonzero Value"  
}
```

여기서 중요한 점은 if_body 를 시작하는 중괄호가 반드시 if 와 같은 라인에 있어야 한다. 줄이 바뀌기 전에 닫히지 않은 중괄호를 두어 인터프리터로 하여금 if_body 가 여러 줄에 걸쳐 있음을 알려 주기 위함이다. 중괄호를 if 다음 라인에 쓰기 위해서는 다음과 같이 백슬래시를 이용하여 기술하여야 한다.

```
if {$x == 0} \  
{  
    puts "Zero Value"  
} \  
else \  
{  
    puts "Nonzero Value"  
}
```

이러한 규칙은 다른 모든 제어 구문에도 동일하게 적용 된다.

else if 문은 다음과 같이 사용한다.

```
if { $x == 0 } {  
    puts "Zero Value"  
} elseif { $x > 0 } {  
    puts "Plus Value"  
} else {  
    puts "Nonzero Value"  
}
```

&&는 양쪽 모두가 참일 때만 참이다. 따라서 왼쪽 수식을 연산한 결과 거짓이면 오른쪽 수식은 연산할 필요도 없이 그 결과는 거짓이 된다. 따라서 Tcl 인터프리터는 &&의 경우 먼저 왼쪽 수식을 연산하여 그 결과가 거짓이면 오른쪽의 수식을 실행하지 않는다. 따라서 다음과 같이 0 으로 나누어지지 않도록 프로그램할 수 있다.

```
if { $x != 0 && [ expr $y / $x ] < 4 } {  
    puts "hahaha"  
}
```

||의 경우에는 어느 하나라도 참인 경우 참이다. 따라서 왼쪽 수식을 연산한 결과가 참이면 오른쪽 수식은 연산할 필요도 없이 그 결과는 참이 된다. 따라서 Tcl 인터프리터는 ||의 경우 먼저 왼쪽 수식을 연산하여 그 결과가 참이면 오른쪽의 수식을 실행하지 않는다. 따라서 ||의 경우에는 연산이 보다 간단한 것을 왼쪽에 두고 연산이 보다 복잡한 쪽을 오른쪽에 두는 것이 좋다.

3.2.2. switch 구문

```
switch options value pat1 pat1Body ?pat2 pat2Body ... default defBody?
```

switch 문은 value 의 여러 가지 값에 따라 그 처리를 달리하는 경우에 사용된다. switch 문에 가능한 옵션들은 다음과 같다.

- * -exact : value 의 값은 정확히 패턴과 일치해야 한다. Tcl 인터프리터는 다른 옵션을 따로 지정하지 않으면 이 옵션이 지정되어 있는 것으로 간주한다.
- * -glob : glob 스타일 패턴 매칭⁹을 사용한다.
- * -regexp : 정규 표현식을 이용한 패턴 매칭을 사용한다.¹⁰
- * -- : 옵션의 끝을 나타낸다. value 값이 “-”로 시작하는 경우 옵션으로 오인하지 않기 위해 필요하다.

```
switch -exact -- $value {
    Apple { incr apple_count }
    Orange { incr orange_count }
    default { incr nothing_count }
}
```

열거된 어떠한 패턴에도 매칭이 일어나지 않는 모든 경우 수행하고 싶은 일이 있으면 default 라는 키워드를 써서 위와 같이 표현할 수 있다. 그러나 default 를 맨 마지막에 두지 않으면 default 가 키워드로 해석되지 않고 value 의 값이 default 일 때 수행해야 할 일로 해석된다는 점에 유의해야 한다. 아래의 예에서 위의 default 문장은 value 의 값이 default 일 때 default_count 를 증가시키라는 뜻이다. 반면에 밑의 default 문장은 value 의 값이 default 도 아니고 Apple 도 아닐때 nothing_count 를 증가시키라는 뜻이다.

```
switch -exact -- $value {
    default { incr default_count }
    Apple { incr apple_count }
    default { incr nothing_count }
}
```

여러 패턴에 걸쳐 같은 일을 수행하고 싶은 일이 있을 때는 다음과 같이 “-”를 써서 연결할 수 있다.

```
switch -exact -- $value {
    Apple -
    Banana -
    Orange { incr fruit_count }
}
```

switch 문을 사용할 때 특히 주의해야 할 점은 주석을 패턴이 위치할 곳에 사용하면 인터프리터가 주석을 패턴으로 인식한다는 점이다. 주석은 반드시 명령어가 올 자리에 와야 한다.

```
switch -exact -- $value {
    # This is bad comment
    Orange {
        # This is good comment
        incr orange_count
    }
}
```

⁹ “3.3.2.1 문자열 매칭”에 자세한 설명이 나와 있다.

¹⁰ “3.6 정규 표현식”의 내용을 참조한다.

3.2.3. while 구문

while booleanExp body

while 문은 booleanExp 가 참인 동안 body 를 반복해서 수행 한다.

```
set j 0
while { $j < 10 } {
    incr j
}
```

tcl 에서 1 을 참, 0 을 거짓으로 해석하므로 다음과 같이 무한 루프를 만들 수 있다.

```
while { 1 } {
    # this is infinite loop
}
```

3.2.4. for 구문

for initial loopcondition increment body

for 문은 loopcondition 이 참인 동안 body 를 수행한다. 그리고 body 의 수행이 끝나면 increment 루틴을 수행하고 loopcondition 이 만족되는지 조사한다.

```
for { set k 0 } { $k < 10 } { incr k 2 } {
    puts $k
}
```

3.2.5. foreach 구문

foreach loopVariable valueList body

foreach 는 valueList 가 가지고 있는 모든 리스트¹¹ 원소들을 하나씩 차례로 loopVariable 에 할당하면서 반복적으로 body 를 수행한다.

```
foreach element { one two three four } {
    puts $element
}

set var [ list 1 2 3 4 ]
foreach element $var {
    puts $element
}

set haha { apple kiwi orange }
foreach fruit $haha {
    puts $fruit
}
```

¹¹ 리스트는 “3.4리스트(List)”에서 자세히 설명한다.

loopVariable 에 여러 개의 변수가 올 수도 있다. 변수와 리스트의 쌍이 안맞을 경우 Null 값이 할당된다. 루프 횟수는 원소의 갯수가 많은 쪽이 정한다.

```
foreach { e1 e2 } { one 1 two 2 three 3 four 4 } {  
  puts "$e1 $e2"  
}  
  
set data { one 1 two 2 three }  
foreach { e1 e2 } $data {  
  puts "$e1 $e2"  
}
```

loopVariable 과 valueList 쌍이 여러 개 올 수도 있다. 각 리스트의 원소 갯수가 안 맞을 경우 모자라는 쪽이 Null 값이 할당된다. 루프 횟수는 원소 갯수가 많은 쪽 리스트에 의해서 정해진다.

```
foreach e1 { one two three four } e2 { 1 2 3 4 } {  
  puts "$e1 $e2"  
}  
  
set list1 [ list one two ]  
set list2 { 1 2 3 }  
foreach e1 $list1 e2 $list2 {  
  puts "$e1 $e2"  
}
```

이를 조합하면 다음과 같은 조금 복잡한 모양도 가능하다.

```
foreach name { apple orange } { e1 e2 } { one 1 two 2 } {  
  puts "$name : $e1 = $e2"  
}
```

3.2.6. break, continue 명령어

break 는 loop 를 빠져 나가기 위해 사용된다. 무한 루프를 빠져나가기 위한 프로그램을 다음과 같이 작성할 수 있다.

```
while { 1 } {  
    ...  
    if { $x > 10 } {  
        break  
    }  
    ...  
}
```

다음의 예제에서 볼 수 있듯이 break 는 가장 안 쪽 루프만 빠져 나갈 수 있다는 점에 유의하여야 한다.

```
for { set ii 0 } { $ii < 3 } { incr ii } {  
    puts "ii : $ii"  
    for { set jj 0 } { $jj < 5 } { incr jj } {  
        if { $jj > 2 } {  
            break  
        }  
        puts "jj : $jj"  
    }  
}
```

continue 는 continue 이하 명령어들을 실행하지 않고 새로운 loop 를 시작한다.

```
for { set k -2 } { $k < 2 } { incr k } {  
    if { $k == 0 } {  
        continue  
    }  
    set ans [ expr 4 / $k ]  
    puts $ans  
}
```

3.2.7. return 명령어

함수를 종료하고 함수를 호출한 바로 다음 줄로 복귀한다.

```
proc mysub { } {  
    for { set k 0 } { $k < 10 } { incr k } {  
        if { $k > 5 } {  
            return  
        }  
        puts $k  
    }  
}
```

Argument 가 주어질 경우 그 값을 함수의 리턴 값으로 반환한다.

```
proc mysub { } {  
    return 3  
}
```

3.2.8. catch 구문

```
catch body ?errorMessageVariable?
```

body 를 수행하다 error 가 발생하면 0 이 아닌 값을 리턴한다. 에러 발생 시 리턴되는 값을 보고 에러의 종류를 판별할 수 있다. 에러 발생을 설명하는 메시지를 맨 마지막의 변수에 저장할 수 있다.

직접 함수를 제작할 경우에도 적절한 에러 코드를 리턴하도록 하여 어떤 상황에서 에러가 발생했는지 명확히 해주는 것이 좋다. built-in 명령어나 Tcl/Tk 인터프리터와 함께 제공된 라이브러리 함수를 이용하는 경우에는 적절히 error 를 발생시키도록 프로그램되어 있다. 직접 제작한 함수 내에서 error 를 발생시키기 위해서는 “3.2.9error 명령어” 절을 참조하기 바란다.

```
if [ catch { ... } errorMessage ] {
    puts stderr "Error : $errorMessage"
} else {
    puts "OK! I done"
}
```

catch 구문을 사용하는 가장 대표적인 경우는 file IO 함수를 사용하거나 exec 명령어를 사용하는 경우이다. 보다 자세한 내용은 “3.8.3file 명령어”절과 “3.9.1exec 명령어”를 참조하기 바란다.

에러가 발생하면 인터프리터가 자동으로 전역 변수 (global variable)인 errorInfo 를 셋팅해 준다. 이 변수의 사용법은 다음과 같다. 변수 errorInfo 에는 에러가 발생했을 때의 여러 가지 상황에 대한 정보가 담겨 있다.

```
if [ catch { ... } errorMessage ] {
    global errorInfo
    puts stderr "Error : $errorMessage"
    puts stderr "Below is TCL TRACED Error Informations"
    puts stderr $errorInfo
}
```

catch 구문의 body 를 { }가 아닌 []로 묶으면 에러가 발생한다. 그런데 발생하는 에러 메시지가 이러한 사실을 정확히 알려 주지 않고 []로 묶인 명령어에 따라 에러 메시지가 틀리게 나온다. 따라서 이렇게 발생한 버그는 매우 찾기 힘든 버그 중에 하나이다.

```
# 다음과 같이 catch를 사용하면 알 수 없는 에러 발생
if [ catch [ ... ] errorMessage ] {
    ...
}
```

body 를 수행하면서 return, break, continue 로 body 의 수행이 끝난 경우에도 catch 문으로 프로그램 흐름이 넘어 온다. 이 때 catch 문이 리턴하는 값은 다음과 같다. 리턴 값을 보고 적절히 프로그램의 의도에 맞게 적절히 처리해 주어야 한다. 이러한 기능이 실제 프로그램에서 그렇게 유용한 기능으로 생각되진 않는다. 필자는 아직까지 한번도 아래 기능을 사용해서 프로그램해야 하는 경우가 없었다.

```
switch [ catch { ... } message ] {
    0 {                                # 정상적인 종료, 즉 에러 없음 }
    1 {                                # 에러가 났음 }
    2 { return $message # return 으로 body 가 끝난 경우 임}
    3 { break           # break 로 body 가 끝난 경우 임}
    4 { continue        # continue 로 body 가 끝난 경우 임}
    default {           # 알 수 없는 경우 임}
}
```

3.2.9. error 명령어

이전 절에서 `catch` 구문에 대하여 공부하였다. 대부분의 경우에는 `catch` 구문만 어떻게 사용하는지 알고 있어도 충분하다. Tcl 인터프리터가 제공하는 **Built-In** 라이브러리들은 내부적으로 적절한 곳에서 `error` 명령을 사용하여 `error` 를 발생시키도록 프로그램되어 있기 때문이다.

`catch` 구문이 발생한 에러를 처리하는 부분이라면, `error` 명령어는 에러를 처음으로 발생시키는 명령어이다. 발생한 에러는 `catch` 구문으로 처리되어야 하는데 프로그래머가 이를 처리하지 않을 경우 Tcl 인터프리터의 기본 에러 핸들러가 이를 처리하게 된다. Tcl 인터프리터의 기본 에러 핸들러는 발생한 에러에 대한 에러메시지를 출력하고 Script의 실행을 종료시킨다.

```
error message ?info? ?code?
```

`message` 는 `catch` 문에서 맨 마지막 변수에 저장될 에러 메시지이다. 함수를 만드는 방법에 대해서는 아직 공부하지 않았지만 다음을 그대로 입력하여 결과를 보면서 `error` 명령어의 동작을 음미해 보기 바란다.

```
proc babo { } {  
    error "This is Error Message"  
}  
  
catch [ babo ] message  
  
set $message
```

연습문제

1. 구구단을 출력하는 프로그램을 작성 하시오.
2. 다음과 같이 동작하는 Text Mode 게임을 작성 하시오
 - * 컴퓨터는 1 부터 60 중 임의의 숫자를 고른다.
 - * 게임 플레이어는 이 숫자를 맞춘다.
 - * 플레이어가 숫자를 맞출 때까지 컴퓨터는 플레이어에게 숫자를 입력시킨다.
 - * 컴퓨터가 고른 숫자보다 플레이어의 숫자가 크면 숫자가 크다는 메시지 출력
 - * 컴퓨터가 고른 숫자보다 플레이어의 숫자가 작으면 숫자가 작다는 메시지
 - * 맞췄으면 축하 메시지와 점수를 출력
 - * 점수는 반복 횟수가 적을 수록 높게
 - * 임의의 숫자를 구하기 위해서 UNIX 명령어인 `date` 와 `cut` 를 사용 하시오.
`date | cut -c18-19`
 - * 사용자가 입력한 숫자를 `userguess` 라는 변수로 입력받기 위해 다음과 같이 프로그램 하시오
`gets stdin userguess`

3.3. 문자열 조작

“2소개”에서도 언급하였지만 Tcl 과 같은 최신 스크립트 언어들이 프로그래밍 언어에 비하여 각광을 받고 있는 이유 중에 하나는 무엇보다도 문자열을 조작하기 위한 강력한 기능들을 갖추고 있기 때문이다. 일반 프로그래밍 언어로 프로그램을 짜기 위해서는 수백 라인에 걸쳐서 프로그램 해야 할 일들이 Tcl 에서는 단 몇 줄의 프로그램만으로도 가능한 일들 중 하나가 바로 문자열 조작이다. 특히 정규 표현식과 같은 기능을 활용한다면 Tcl 의 강력한 기능을 더욱 실감할 수 있을 것이다.

3.3.1. Tcl 문자열에 대한 정의

Tcl 에서 문자열은 알파벳이나 밑줄문자로 시작하는 글자들의 집합이다. 알파벳이나 밑줄문자로만 시작하면 그 다음에 올 수 있는 문자들에는 제한이 없다. 단, Tcl 인터프리터가 특별한 의미를 부여한 기호를 문자열에 포함시키기 위해서는 백슬래쉬(\)를 사용해야 한다. 여기에 관한 자세한 내용은 “3.1.7 백슬래쉬 문자열(Backslash Sequence)”을 참조하기 바란다.

문자열에 공백 문자를 포함시키기 위해서는 따옴표를 사용하면 된다. 공백문자가 없는 문자열의 경우는 따옴표로 감싸줄 필요가 없다.

3.3.2. string 명령어

string 명령어는 문자열에 관해서 할 수 있는 많은 연산들을 한데 모아 놓은 명령어 이다. 이 명령어의 기능을 요약하면 다음과 같다.

문법	기능
string compare str1 str2	str1 과 str2 를 사전 순으로 비교하여, 같으면 0, str1 이 str2 보다 앞서면 -1, 뒤서면 1 을 리턴
string first str1 str2	str1 이 str2 에서 첫번째로 나타나는 인덱스를 리턴
string index str i	str 의 i 번째 문자를 리턴 ¹²
string last str1 str2	str1 이 str2 에서 마지막으로 나타나는 인덱스를 리턴
string length str	str 의 길이를 리턴
string match pattern str	glob style 패턴 ¹³ 이 str 에 매칭되면 1 을 아니면 0 을 리턴
string range str i j	str 의 i 번째에서 j 번째까지의 문자들을 리턴
string tolower str	str 내의 대문자를 모두 소문자로 바꿈
string toupper str	str 내의 소문자를 모두 대문자로 바꿈
string trim str ?chars?	str 의 시작 부분과 끝 부분에 존재하는 chars 를 없애버린다. chars 는 따로 지정하지 않으면 공백문자들이다.
string trimleft str ?chars?	str 의 시작 부분에 존재하는 chars 를 없애 버린다.
string trimright str ?chars?	str 의 끝 부분에 존재하는 chars 를 없애 버린다.
string wordend str ix	str 내에서 ix 번째 글자가 포함 된 단어가 끝나는 지점의 인덱스를 리턴. 단어 사이는 공백 문자로 구분한다.
string wordstart str ix	str 내에서 ix 번째 글자가 포함 된 단어가 시작하는 지점의 인덱스를 리턴

표 4 string 명령어 기능 요약

¹² 문자열의 인덱스 값은 0 부터 시작한다. 즉, 문자열에서 첫번째 글자의 인덱스는 0 이라는 뜻이다.

¹³ “3.3.2.1 문자열 매칭” 참조

string 명령어의 실제 사용 예를 보이면 다음과 같다.

```
% set str ":::::: won nara is genius :::: nara? really? :::"
:::::: won nara is genius :::: nara? really? ::::
% string first "nara" $str
11
% string last "nara" $str
31
% string index $str 1
:
% string length $str
49
% string match ".*" $str
1
% string trim $str :
  won nara is genius :::: nara? really?
% string trimleft $str :
  won nara is genius :::: nara? really? ::::
% string trimright $str :
:::::: won nara is genius :::: nara? really?
% string toupper $str
:::::: WON NARA IS GENIUS :::: NARA? REALLY? ::::
% string range $str 11 14
nara
% string wordend $str 12
15
% string wordstart $str 12
11
%
```

3.3.2.1. 문자열 매칭

string match 명령어에서 사용되는 패턴은 glob-style 패턴이다. 이는 UNIX 셸에서 사용하는 wild card 문자들을 사용하여 기술한 패턴을 말한다. UNIX 사용자라면 이 패턴에 익숙해 있을 것이다. glob-style 패턴 기술에 사용되는 wild card 문자들은 과 같다.

Wild card	의미
*	아무 글자에나 매칭이 되고 0 개 이상의 글자에 매칭된다.
?	아무 글자나 한 글자에만 매칭 된다.
[]	[와] 사이에 열거된 문자에만 매칭 되고 한 글자만 적용된다. ¹⁴

다음의 사용 예를 통하여 glob-style 패턴 사용법을 정확히 이해하도록 하자. 패턴은 “{ }”로 싸줘야 제대로 동작한다.

```
% string match {a*} about
1
% string match {a?} about
0
% string match {a????} about
1
% string match {[aA]} a
1
% string match {[aA]} A
1
% string match {[aA]} aA
0
% string match {[aA][aA]} aA
1
% string match {a\?} a?
1
```

¹⁴ “[]”의 보다 자세한 사용법은 “3.6.1정규 표현식 Primitives”에 나와 있다.

3.3.2.2. 문자열 비교

Tcl 에서 문자열을 비교하는 가장 확실한 방법은 다음과 같이 `string compare` 명령어를 이용하는 방법이다.

```
if { [ string compare $var MODEL ] == 0 } {  
    ...  
}  
...  
if { [ string compare $var "Test String" ] == 0 } {  
    ...  
}  
...  
if { [ string compare $var1 $var2 ] == 0 } {  
    ...  
}
```

이 방법 말고도 “==” 연산자를 사용하여 다음과 같이 두 문자열이 서로 같은 지 비교할 수 있다.

```
if { "$var" == "MODEL" } {  
    #변수와 문자열을 비교할 때는  
    #변수와 문자열을 반드시 따옴표로 묶어야 Syntax Error 가 발생하지 않음  
    ...  
}  
...  
set a "HaHa HoHo"  
set b "HaHa HoHo"  
if { $a == $b } {  
    #변수끼리 비교할 때는 따옴표로 묶을 필요 없음  
    ....  
}
```

3.3.3. format 명령어

`format` 명령어는 어떤 형식으로 변수를 문자열로 바꿔줄 것인가를 명시하는 명령어이다. C 언어에서 `printf()` 함수를 사용해본 경험이 있다면 이 명령어를 쉽게 이해할 수 있을 것이다.

`format spec value1 ?vlaue2 ...?`

C 언어의 다음 문장을 똑같은 결과가 나오도록 Tcl 로 프로그램한 예는 다음과 같다.

```
/* C 언어 문장 */  
printf( "Sum is %5.2f\n", sum );  
  
# Tcl 문장  
puts [ format "Sum is %5.2f" $sum ]
```

`spec` 은 문자열이며 변수의 값을 찍어 줄 자리에 %기호를 이용하여 변수를 문자열로 변환할 규칙을 명시한다. `spec` 문자열 안에서는 “%”가 특별한 의미를 갖고 있기 때문에 “%” 기호 자체를 문자열에 포함시키고 싶을 경우에는 “%%”를 사용해야 한다. `puts` 명령어를 이용하여 “%”를 찍을 때는 하나만 사용해도 되지만 `format` 명령어의 `spec` 문자열 안에서 %를 찍기 위해서는 “%%”라고 써야 한다. %문은 다음

과 같은 필드들로 구성된다.

%?flags??preceeding??width??precision?type

width 필드는 변수가 문자열로 변환될 공간의 최소 크기를 표시한다. 변환된 문자열이 이 크기 보다 작아도 width 만큼 공간을 차지하도록 한다. 만일 변환된 문자열의 길이가 width 보다 크면 필요한 만큼 공간을 늘린다. width 는 숫자로 명시하는 것이 일반적이지만 그 크기를 프로그램 작성시에는 알 수 없고 프로그램 동작 중에 명확해 지는 경우도 있다. 이를 위하여 width 필드에 “*”를 쓰고 width 값을 뒤에 변수로 줄 수도 있다.

precision 필드는 실수를 문자열로 변환하고자 할 때 필요한 필드이다. 구두점 “.” 다음에 소수점 이하 몇 째 자리까지 변환할 것인가를 명시한다. 명시된 것보다 작은 부분은 반올림 된다.

type 필드는 변수에 저장된 값을 무엇으로 해석해야 할 것인가를 알려주는 필드이다. 이 필드는 “%”기호 다음에 반드시 존재해야 한다. 여기에 가능 값들과 의미를 표 5에 정리하였다.

Type	의미
d	정수
u	양의 정수
I	정수, 변수에 기억된 값이 8 진수(0), 16 진수(0x)형식으로 저장된 문자열일 때 사용한다.
O	양의 8 진수
x	양의 16 진수, 알파벳이 필요할 경우 소문자로 사용
X	양의 16 진수, 알파벳이 필요할 경우 대문자 사용
c	변수에 저장된 숫자를 ASCII 코드로 생각하고 그에 해당하는 글자로 변환 시킴
s	문자열
f	실수, 구두점으로 구분된 형식 (예: 3.14)
e	실수, 과학적 표기법, e 를 사용 (예: 1.23e14)
E	실수, 과학적 표기법, E 를 사용 (예: 1.23E14)
g	실수를 %f 나 %e 중 짧은 쪽으로 변환
G	실수를 %f 나 %E 중 짧은 쪽으로 변환

표 5 format 명령어 %구문 type 필드

flags 필드는 변환된 문자열의 모양을 다듬기 위해 사용되는 필드이다. 이 필드에 사용할 수 있는 플래그들을 표 6에 정리하였다.

Flag	의미
-	변환된 문자열의 길이가 width 보다 작을 경우 왼쪽으로 정렬 한다.
+	숫자의 경우 양, 음수 부호를 반드시 출력한다.
#	이 플래그는 type 필드에 따라서 다른 의미를 갖는다. 아래의 type 들 이외에는 아무런 의미가 없다. ● %o : 맨 앞에 항상 0 을 붙인다. ● %x : 맨 앞에 항상 0x 를 붙인다. ● %g : 소수점 이하에서 뒤에 붙는 0 들을 없애지 않는다.

표 6 format 명령어 %구문 플래그

preceeding 필드에 올 수 있는 것은 0 과 스페이스 문자이다. width 에 정의된 공간 보다 숫자가 차지하는 문자열의 길이가 짧을 경우 왼쪽에 이 문자들로 적당히 채워 width 의 길이가 되도록 한다. 왼쪽으로 정렬시켰을 때는 아무 의미도 없다.

다음에 format 명령어의 사용 예를 음미하면서 format 명령어의 기능을 정확히 이해하도록 하자.

```
% format "%010d" 33
0000000033
% format "%03.3f" 1.23456
1.235
% format "%010.3f" 1.23456
000001.235
% format "%o" 8
10
% format "%#o" 8
010
% format "%x" 10
a
% format "%X" 10
A
% format "%#X" 10
0xA
% format "%e" 1.23456
1.234560e+00
% format "%E" 1.23456
1.234560E+00
% format "%g" 1.234
1.234
% format "%g" 0.00000000000000012343
1.2343e-16
% format "%g" 1.200000
1.2
% format "%#g" 1.20000
1.20000
% format "%*s" 10 aaa
      aaa
% format "%0*d" 5 3
00003
% puts [format "Result : %s = %d %" value 30]
Result : value = 30 %
```

3.3.4. scan 명령어

scan 명령어는 주어진 문자열을 명시한 spec 에 따라 해석하는 기능을 한다. 리턴 값은 해석을 해서 변환한 갯수이다. 문자열로 표현된 숫자를 계산에 사용될 수 있는 숫자로 변환하거나 글자를 ASCII 코드로 변환하는 경우 등에 유용하게 사용될 수 있다. format 명령어와 반대되는 기능을 한다고 볼 수 있다. C 언어에서 scanf() 함수와 비슷한 기능을 한다.

```
scan str spec var1 ?var2 ...?
```

scan 에서 사용하는 spec 은 format 에서 사용하는 spec 과 거의 같다. 단, %u 를 사용할 수 없으며 “[]” 를 사용하여 가능한 문자 집합을 명시할 수 있다.¹⁵ “[]”를 사용할 경우 따옴표 안에서 사용하면 따옴표는 치환을 허용하기 때문에 “[]”를 명령어 netsting 으로 해석하려 한다. 이를 방지하기 위해 치환을 허용하지 않는 중괄호를 사용해서 spec 을 명시해야 한다.¹⁶

¹⁵ “[]”의 자세한 사용법은 “3.6.148정규 표현식 Primitives”에 나와 있다.

¹⁶명령어 nesting 에 관한 설명은 “3.1.8명령어의 nesting”에 있으며, 그룹핑과 치환에 관한 설명은 “3.1.9그룹핑(Grouping)과 변수 치환(Substitution)”에 있다.

```
% scan "0xA 010" "%x %o" hex oct
error
% scan "A 10" "%x %o" hex oct
2
% set hex
10
% set oct
8
% scan "abcABCdef" "[%a-z]" low
invalid command name "a-z"
% scan "abcABCdef" {[%a-z]} low
1
% set low
abc
```

3.3.5. 문자열 덧붙이기와 분할, 병합

문자열을 분할하고 병합하는 함수는 리스트와 밀접한 관계를 갖고 있다. 리스트에 관한 자세한 내용은 “3.4리스트(List)”절을 참조하기 바란다.

3.3.5.1. append 명령어

append 명령어는 첫번째 Argument 로 주어진 문자열 변수에 저장된 문자열과 나머지 Argument 들을 연결하여 하나의 문자열로 만들어 준다.

```
% set fruit "app"
app
% append fruit le
apple
% set fruit
apple
%
```

3.3.5.2. split 명령어

필드 분리자(field separator)를 명시해 주면 그 필드 구분자로 분리된 필드들을 추출하여 리턴한다.

```
% set test_string host:address:phone:name
host:address:phone:name
% split $test_string :
host address phone name
```

3.3.5.3. join 명령어

주어진 리스트의 각 원소들 사이에 주어진 필드 분리자(field separator)를 넣어 하나의 문자열로 병합한다.

```
% set myData { host address phone name }
host address phone name
% join $myData :
host:address:phone:name
%
```

3.4. 리스트(List)

Tcl 에서 리스트는 서로 관련 있는 것을 그룹화 시켜 놓고 한꺼번에 다루고자 할 때 매우 유용한 데이터 구조이다.

foreach 문을 이용하여 리스트의 모든 원소들에 대하여 목적에 맞는 특별한 일을 할 수 있다.

eval 명령어를 이용하면 리스트에 기억된 내용들을 Tcl 명령어로 생각하고 재해석하여 실행하도록 할 수 있다. eval 명령어는 차후에 수행될 명령어를 미리 만들어 기억시켜 두었다가 적당한 시기에 수행시키고자 할 때 유용하다.

3.4.1. 리스트의 생성

리스트는 중괄호를 이용하여 생성할 수도 있으며 list 명령어를 이용하여 생성할 수도 있다. 중괄호를 이용하는 것 보다 list 명령어를 이용하는 편이 수행 속도가 빠르다. 명령어의 Argument 로 리스트를 받아들이는 명령어에서 중괄호를 사용해서 리스트를 만들어 넘길 때 에러가 나는 경우 list 명령어로 리스트를 만들어 넘겨주면 에러를 없앨 수 있다.

```
% set a { 1 2 3 4 }
1 2 3 4
% set b [ list a b c d ]
a b c d
demo
haha
```

공백 문자를 포함하는 리스트 원소를 정의하기 위해서는 중괄호나 따옴표를 이용한다.

```
% set a { "1 2" { 3 4 } 5 }
"1 2" {3 4} 5
% set b [ list a {b c} "d e" ]
a {b c} {d e}
% foreach x $a {
    puts $x
}
1 2
3 4
5
% foreach x $b {
    puts $x
}
a
b c
d e
```

그러나 Tcl 은 공백 문자를 리스트 원소를 구별하는데 이용하기 때문에 공백 문자를 포함하는 리스트 원소는 또 하나의 리스트를 구성한다.

```
% set b [list {a b c} d]
{a b c} d
% foreach x $b {
    foreach c $x {
        puts $c
    }
}
a
b
c
d
```

3.4.2. 리스트 추가 및 연결

리스트 관련 명령어들을 공부할 때 주의해야 할 점은 각 명령어가 원본 변수의 값을 변화 시키는가 아닌가를 눈여겨 볼 필요가 있다. 보통 변수의 이름을 **Argument** 로 받는 경우는 원본 변수의 값을 변화 시키고, 변수의 값을 **Argument** 로 받는 경우엔 원본 변수의 값을 변화시키지 않는다. 이러한 규칙은 모든 Tcl 명령어에 해당된다. 간단히 변수 이름 앞에 **\$**가 붙으면 원본이 변하지 않고, **\$**가 붙지 않으면 원본이 변한다고 생각하면 쉽다. 원본 변수의 값이 변하지 않는 경우에는 명령어의 리턴값을 적당한 변수에 저장하여 사용하여야 한다.

3.4.2.1. lappend 명령어

존재하지 않는 변수가 주어졌을 때는 새로 리스트를 생성하고 이미 존재하는 변수가 주어졌을 때는 그 변수 리스트의 끝에 리스트를 덧붙인다. 원본 변수의 값이 바뀐다.

```
% lappend myList 1 2
1 2
% lappend myList 3 {4 5}
1 2 3 {4 5}
% lappend myList 6 7 8 9
1 2 3 {4 5} 6 7 8 9
% set myList
1 2 3 {4 5} 6 7 8 9
```

3.4.2.2. concat 명령어

주어진 **Argument** 들을 연결하여 하나의 리스트로 만든다. 원본 변수의 값이 바뀌지 않는다.

```
% set a [list 1 2]
1 2
% set b [list 3 4]
3 4
% set c 5
5
% concat $a $b $c 6 7
1 2 3 4 5 6 7
```

3.4.3. 리스트 원소 얻기

본 절에서 설명하고 있는 `lindex` 와 `lrange` 명령어를 사용할 때 리스트에 저장된 데이터가 매우 많은 경우 약간의 주의를 요한다. Tcl 8.0 이전 버전에서는 리스트의 크기가 매우 큰 경우, 이 둘 명령어의 수행 속도가 현저히 떨어지기 때문이다. Tcl 8.0 에 와서 개선 되기는 하였으나 리스트에 저장할 데이터가 매우 많은 경우에 `lindex` 와 `lrange` 명령어를 사용해야 한다면 배열을 사용할 것을 권장한다.¹⁷ 배열에 대한 자세한 내용은 “3.5배열”을 참조하기 바란다.

3.4.3.1. `llength` 명령어

주어진 리스트의 원소 개수를 리턴한다.

```
% llength { a b c d }
4
% llength { {a b} "c d" e }
3
% llength {}
0
% set kk [ list a b c ]
a b c
% llength $kk
3
```

3.4.3.2. `lindex` 명령어

주어진 리스트 중에서 인덱스에 해당하는 원소를 리턴한다. 인덱스는 0 부터 시작한다.

```
% lindex { a b c d } 1
b
% set k [list a b c d ]
a b c d
% lindex $k 0
a
%
```

3.4.3.3. `lrange` 명령어

주어진 리스트 중에서 시작 인덱스와 끝 인덱스 사이의 원소를 리턴한다. 리스트의 끝까지를 나타내는 특별한 인덱스 `end` 가 있다

```
% set k [list a b c d e f g]
a b c d e f g
% lrange $k 1 4
b c d e
% lrange $k 4 end
e f g
%
```

¹⁷ Tcl 8.0 이전 버전에서 내부적으로 배열은 해쉬 테이블을 이용해 구현되어 인덱스에 상관없이 모든 배열 원소를 참조하는 시간이 같지만, 리스트는 순차적인 검색을 하여 원소를 찾기 때문에 뒤쪽에 있는 원소일 수록 검색 시간이 오래 걸린다.

3.4.4. 리스트 원소 삽입 및 치환

리스트 원소 삽입 및 치환 명령어들은 대부분 원본 변수 값을 바꾸지 않는다는 점에 유의하기 바란다. 삽입되거나 치환된 결과를 이용하기 위해서는 명령어의 리턴값을 따로 저장해야 한다.

3.4.4.1. linsert 명령어

리스트에 Argument 들을 주어진 인덱스 위치에 끼워 넣는다.

```
% linsert { 0 1 2 3 } 2 a b c d
0 1 a b c d 2 3
% set k [list a b c d]
a b c d
% linsert $k 0 5 6
5 6 a b c d
%
```

3.4.4.2. lreplace 명령어

리스트에서 시작 인덱스부터 끝 인덱스까지 주어진 Argument 들로 바꾼다.

```
% lreplace { 0 1 2 3 4 } 1 3 a b c d e
0 a b c d e 4
% lreplace { 0 1 2 3 4 } 2 end d
0 1 d
```

3.4.5. 리스트 검색과 정렬

3.4.5.1. lsearch 명령어

리스트의 원소 중에서 패턴에 맞는 첫번째 원소를 찾아 그 원소를 리턴한다. 리스트 내에 패턴에 맞는 원소가 없을 경우 -1 을 리턴한다. 이 명령에 가능한 옵션들은 다음과 같다.

- * -glob : glob 스타일의 패턴 매칭¹⁸을 한다. 옵션이 없는 경우 이 옵션으로 지정되어 있다고 생각한다.
- * -exact : 정확히 모든 글자가 일치하는 경우
- * -regex : 정규 표현식¹⁹을 이용한 패턴 매칭을 한다.

```
% lsearch { apple air snow } a*
0
% lsearch -exact { apple air snow } apple
0
% set kk { test haha hello }
test haha hello
% lsearch $kk apple
-1
```

¹⁸ “3.3.2.1문자열 매칭”에 자세한 설명이 나와 있다.

¹⁹ “3.6정규 표현식”을 참조한다.

3.4.5.2. Lsort 명령어

리스트의 원소들을 정렬하여 리턴한다. 단, 리스트 자체는 변하지 않는다. 가능한 옵션들은 다음과 같다.

- * **-ascii**: 리스트 원소들을 ASCII 값에 따라 정렬한다.
- * **-dictionary**: 리스트 원소들을 사전 순으로 정렬한다.
- * **-integer**: 리스트 원소들이 모두 정수임을 명시한다.
정수가 아닌 경우 에러가 발생한다.
- * **-real**: 리스트 원소들이 모두 실수임을 명시한다.
실수가 아닌 경우 에러가 발생한다.
- * **-increasing**: 오름순으로 리스트를 정렬한다. 디폴트 값이다.
- * **-decreasing**: 내림순으로 리스트를 정렬한다.

```
% lsort -ascii { Z m b B }  
B Z b m  
% lsort -dictionary { Z m b B }  
B b m Z  
% set k [list 7 3 5 1 ]  
7 3 5 1  
% lsort $k  
1 3 5 7  
% set k  
7 3 5 1
```

연습문제

다음과 같은 데이터 파일을 읽어들이고 그래프를 그리고자 한다. 그래프를 그리는 것은 Tk 를 공부한 다음으로 잠시 미루고, 그래프를 그리기 위한 전 단계로 다음과 같은 데이터 파일이 주어졌을 때 이를 해석하여 다음과 같은 출력을 보여 주는 프로그램을 작성한다.

프로그램을 작성할 때는 Tk 공부 후 그래프를 그릴 수 있도록 데이터 구조를 설계하고 구현하여야 한다. 단, 현재까지 File 입출력을 다루지 않았으므로 stdin 에서 데이터를 입력 받는 것으로 프로그램을 작성한 후 UNIX 와 Windows 에서 제공하는 File Redirection 을 이용해서 데이터 파일을 입력 받도록 한다. 예를 들어 작성한 프로그램 이름이 myprog2.tcl 이고 데이터 파일 이름이 mydata.dat 이라면 다음과 같이 실행한다.

```
$ myprog2.tcl < mydata.dat
```

파일의 끝인지 테스트 하는 Tcl 명령어에는 eof 가 있다. stdin 으로 입력되는 파일의 끝인지를 테스트 하기 위해서는 다음과 같이 하면 된다.

```
while { [ eof stdin ] == 0 } {
```

데이터 파일

```
#
# This is sample data file for Tcl/Tk programming project
#

TITLE           : Imaginary data for test
COMMENT         : 1999.2.9

DATA_NUM        :      4
LOT_NUM         :      5

FIELD_NAME      :      LOT1 LOT2 LOT3 LOTP LOTK

MPU              100  200   400 500   20
DSP              10   50    80 90    100
PKG              300  400   320 430   650
TGET             400  210   330 410   200
```

화면 출력

```
title           : Imaginary data for test
comment         : 1999.2.9
data number     : 4
lot number      : 5
field names     : LOT1 LOT2 LOT3 LOTP LOTK
DATA : MPU - 100 200 400 500 20
DATA : DSP - 10 50 80 90 100
DATA : PKG - 300 400 320 430 650
DATA : TGET - 400 210 330 410 200
```

3.5. 배열

배열은 일련의 데이터를 인덱스를 사용하여 저장하고 참조할 수 있도록 해준다. C 언어와 같은 프로그래밍 언어에서와는 달리 인덱스로 문자열을 사용한다. 따라서 Tcl 배열의 문자열 인덱스를 잘 활용하면 이진 트리나 변환 테이블 같은 복잡한 데이터 구조를 쉽게 만들 수 있다.

3.5.1. 배열의 생성과 참조

배열은 배열의 한 원소에 값을 할당할 때 배열이 존재하지 않으면 생성된다. 값을 할당하지 않은 배열 원소를 참조할 경우에는 에러가 발생한다. 다음 예제를 통하여 배열을 생성하고 참조하는 방법을 이해하도록 한다.

```
% set myname(last) won
won
% set lastname $myname(last)
won
% set firstname $myname(first)
can't read "myname(first)": no such element in array
% set myname(first) nara
nara
% set firstname $myname(first)
nara
% set fruit(0) apple
apple
% set fruitname $fruit(0)
apple
% set index last
last
% puts $myname($index)
won
```

배열을 사용할 때 잦은 실수는 다음과 같은 경우이다.

```
% set myname(myindex) won
won
% set index myindex
myindex
% puts $myname( $index )
error
```

여기서 명심해야 할 것은 배열의 인덱스는 문자열이기 때문에 공백 문자도 인덱스를 나타내는 문자열로 해석된다는 점이다. 위의 예제에서 괄호 사이에 공백 문자가 존재하기 때문에 Tcl 인터프리터는 인덱스 문자열을 공백을 포함한 문자열인 “myindex”을 가지고 배열의 원소를 찾게 되므로 에러가 발생하는 것이다. 이러한 에러를 방지하기 위해서는 다음과 같이 수정해야 한다.

```
% set myname(myindex) won
won
% set index myindex
myindex
% puts $myname($index)
error
```

따라서 배열의 인덱스를 표기할 때 괄호와 인덱스 사이에 공백이 존재하지 않도록 해야 한다는 점을 명심해야 한다.

3.5.2. array 명령어

array 명령어는 배열에 대한 여러 가지 정보를 리턴한다. array 명령어의 기능을 요약하면 표 7과 같다.

문법	기능
array exists arr	배열 arr 이 존재하면 1 을 리턴 한다.
array get ?pattern?	배열 내의 모든 원소들에 대해 인덱스, 값의 순서로 리스트를 만들어 리턴한다. 패턴이 주어지면 패턴에 일치하는 인덱스에 대해서만 리스트를 만든다. 예 : { 0 apple 1 orange }
array names arr ?pattern?	배열 내의 모든 인덱스의 리스트를 리턴한다. 패턴이 주어진 경우 패턴에 일치하는 인덱스들만 리스트로 만든다.
array set arr list	array get 에서와 같은 형식으로 된 리스트를 배열로 만든다.
array size arr	배열에 정의된 인덱스의 갯수를 리턴한다.
array startsearch arr	배열 arr 에 대한 검색 작업을 초기화 하고 이 작업을 identify 하는 ID 를 리턴한다.
array nextelement arr id	배열 arr 에 대하여 행해지고 있는 검색작업 id 에서 다음 인덱스를 리턴한다. 더 이상의 인덱스가 없으면 빈 문자열을 리턴한다.
array anymore arr id	배열 arr 에 대하여 행해지고 있는 검색작업 id 에서 남아 있는 인덱스가 더 있으면 1 을 리턴한다.
array donesearch arr id	배열 arr 에 대한 검색 작업 id 를 끝마친다.

표 8 array 명령어의 기능 요약

array 의 모든 인덱스에 대하여 어떤 일을 하는 프로그램을 작성해 보면 다음과 같다.

```
foreach index [array names myArray] {
    puts $myArray($index)
}
```

똑같은 일을 하는 프로그램을 아래와 같이 작성할 수도 있다.

```
set id [array startsearch myArray]
while {[array anymore myArray $id]} {
    set index [array nextelement myArray $id]
    puts $myArray($index)
}
array donesearch myArray $id
```

array names 명령어에는 원하는 패턴을 명시하여 패턴과 매칭되는 인덱스만을 골라 특별한 일을 처리할 수도 있음을 눈여겨 보자. 이를 이용하면 보다 프로그램을 간단히 구현할 수 있다.

```
foreach index [array names myArray ".signal." ] {
    puts $myArray($index)
}
```

3.5.3. 배열을 이용한 복잡한 데이터 구조 구현

배열을 이용해서 구현할 수 있는 데이터 구조는 무궁무진하다. 몇 가지 예를 통하여 배열로 복잡한 데이터 구조를 만들기 위한 힌트를 얻어 보자.

3.5.3.1. 주소록

```
proc AddAddress { name address } {  
    global array TABLE  
    set TABLE($name) $address  
}  
proc GetAddress { name } {  
    global array TABLE  
    return $TABLE($name)  
}
```

3.5.3.2. 이진 트리

다음 프로그램은 그림 2와 같은 이진 트리를 구성한 후 Top-Down 방식으로 왼쪽부터 오른쪽으로 트리를 여행하면서 각 노드의 값을 찍어 주는 프로그램이다.

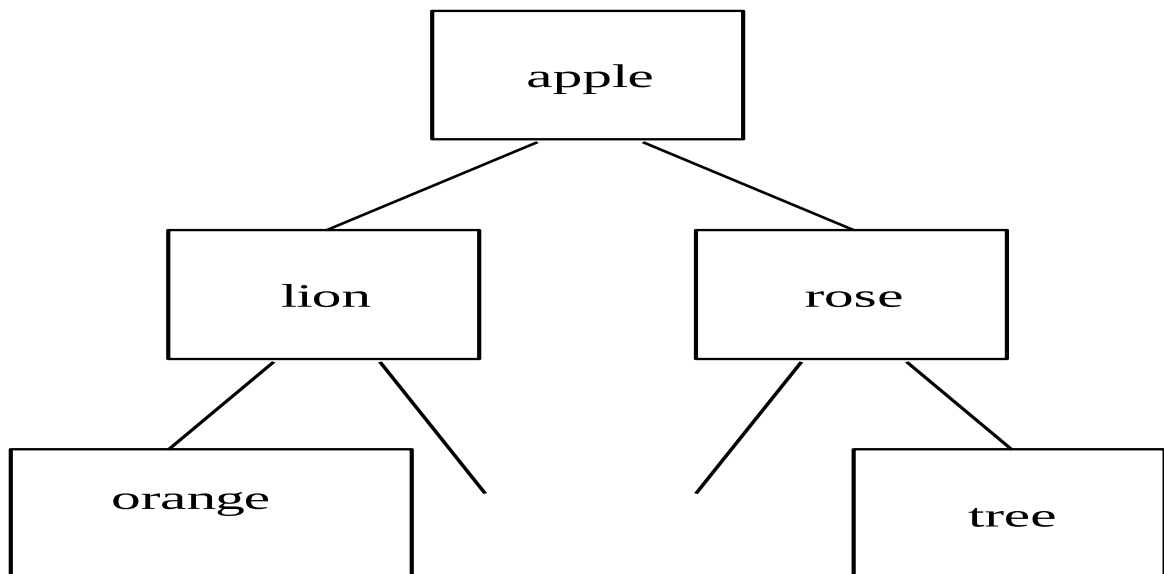


그림 2 트리 구조

```

#!/usr/local/bin/tclsh

proc MakeNewNode { name value } {
    global tree
    set tree($name.value) $value
    set tree($name.left) empty
    set tree($name.right) empty
}

proc AddNodeLeft { parent child } {
    global tree
    set tree($parent.left) $child
}

proc AddNodeRight { parent child } {
    global tree
    set tree($parent.right) $child
}

proc Tour { startnode } {
    global tree

    puts $tree($startnode.value)
    if { "$tree($startnode.left)" != "empty" } {
        Tour $tree($startnode.left)
    }

    if { "$tree($startnode.right)" != "empty" } {
        Tour $tree($startnode.right)
    }
}

MakeNewNode node1 apple
MakeNewNode node2 lion
MakeNewNode node3 rose
MakeNewNode node4 orange
MakeNewNode node5 tree

AddNodeLeft node1 node2
AddNodeRight node1 node3
AddNodeLeft node2 node4
AddNodeRight node3 node5

Tour node1

```

연습문제

1. 배열을 이용하여 Stack 을 구현하시오.
2. 배열을 이용하여 Queue 를 구현하시오.
3. 배열을 이용하여 다차원 배열을 구현하시오.

3.6. 정규 표현식

정규 표현식 (Regular Expression)은 원하는 문자열을 찾아내어 적절히 조작하기 위해 사용되는 강력한 문법이다. 문자열의 Pattern Matching 을 위해 사용되는 문법이라고 이해하면 된다.

3.6.1. 정규 표현식 Primitives

정규 표현식에 사용되는 연산자들을 표 9에 정리하였다. 단, **표시가 된 기능은 Tcl/Tk 8.1 이상의 버전에서만 동작한다.

종류	기호	의미
메타 (Meta) 문자	.	어떤 글자든지 한 글자와 매칭된다.
	^	문자열의 시작 부분임을 나타냄
	\$	문자열의 끝 부분임을 나타냄
반복 연산자	*	바로 앞 패턴이 0 번 이상 반복 됨을 표시
	+	바로 앞 패턴이 1 번 이상 반복 됨을 표시
	?	바로 앞 패턴이 나타나지 않거나 1 번 나타날 수 있음을 표시
	{m}	바로 앞 패턴이 m 번 반복됨을 표시 **
	{m,}	바로 앞 패턴이 m 번 이상 반복됨을 표시 **
	{m,n}	바로 앞 패턴이 m 번 이상 n 번 이하 반복됨을 표시 **
그룹	()	패턴의 한 부분을 그룹화한다. 패턴에 매칭되는 부분을 기억한다.
열거 및 선택	[]	[]안에 열거된 글자 중 하나와 같으면 매칭 된 것으로 본다.
	[^]	[^와]사이에 열거된 글자가 아닌 다른 글자이면 매칭된 것으로 본다.
		“ ” 기호 왼쪽에 명시된 패턴이나 오른쪽에 명시된 패턴 중 하나에 매칭되면 매칭 된 것으로 본다.
복귀	\	연산자로 사용되는 문자를 패턴에 사용하기 위해 바로 앞에 붙인다.
Backslash Escape		공백 문자, 숫자 등을 나타내는 패턴과 같이 자주 사용되는 패턴들을 간단히 표현할 수 있도록 여러 가지 backslash escape 를 제공한다. **

표 9 정규표현식 연산자

구두점 “.”은 어떤 글자이든 한 글자에 매칭된다. alphanumeric 글자 뿐 아니라 탭, 스페이스, 특수 문자 등 모든 한 문자에 매칭된다. 4 글자 이상으로 이루어진 문자열을 찾는 패턴은 다음과 같다. 밑줄 친 부분이 실제로 패턴에 매치되는 부분이다. 문자열안에 패턴과 매치되는 부분이 있기만 하면 그 문자열은 패턴과 매치된 것으로 본다.

패턴	
매치되는 문자열	<u>aaaa</u> <u>1234</u> 567 <u>a13b</u> 456ba is woow <u>cat</u> is yellow
매치되지 않는 문자열	bcd 12 a

소문자 알파벳과 숫자로만 이루어진 2 글자가 들어 있는 패턴을 찾기 위해 필요한 정규 표현식은 다음과 같다.

패턴	[a-z,0-9][a-z,0-9]
매치되는 문자열	<u>b9</u> <u>1a</u> 567 BBA <u>a3</u> 456ba is woow NUMBER <u>is</u> 12 THIS IS NUM <u>12</u>
매치되지 않는 문자열	AB1 aABCD 9 IS NOT ALPHABET

“[]”는 그 사이에 나열된 모든 글자 중 하나와 일치하면 일치하는 것으로 본다. “a-z”는 “a”에서 “z”까지 모든 문자를 나타낸다. 마찬가지로 “0-9”도 “0”에서 “9”까지 모든 문자를 나타낸다. “[]”사이에 정규 표현식에 사용되는 연산자나 기타 의미 있는 기호들은 모두 단순히 패턴으로만 사용된다. 단, “[” 바로 다음에 “^”가 오면 열거된 문자가 아닌 문자와 매칭시킨다는 특별한 의미를 갖는다.

패턴	AA[^a-z,0-9]
매치되는 문자열	<u>AAA</u> <u>AAZ</u> a67 BBA <u>AC</u> 456ba is woow AA9 <u>AD</u> AAA 12 THIS IS AA <u>aAF</u>
매치되지 않는 문자열	AAa AA9

“+”는 바로 앞 패턴이 한 번 이상 반복되는 것을 의미한다. 따라서 대문자 알파벳으로만 이루어진 문자열을 포함하는 패턴을 찾기 위한 정규 표현식은 다음과 같이 쓸 수 있다.

패턴	[A-Z]+
매치되는 문자열	<u>ABCDEFGHJKLMN</u> abcd <u>AAF</u> AAA 12 1234 <u>THISAF</u> abcd is hahaha
매치되지 않는 문자열	abcd 123 this is not matched

“*”연산자는 0 번 이상 패턴이 반복됨을 나타낸다. “+”연산자는 그 패턴이 반드시 나타나야 매칭이 이루어 지지만 “*”연산자는 패턴이 나타나지 않아도 매칭이 이루어진다. 이를 이용하면 앞에 “A”가 반드시 하나 있고 그 뒤에는 대문자 알파벳으로만 이루어진 단어를 포함하는 패턴을 찾기 위한 정규 표현식을 다음과 같이 쓸 수 있다.

패턴	A[A-Z]*
매치되는 문자열	<u>A</u> <u>ABCDEFGHJKLMN</u> abcd <u>AAF</u> AAA 12 1234 <u>AHISAF</u> abcd is hahaha
매치되지 않는 문자열	abcd 123 BCDEF this is not MATCHED

{ }는 반복의 횟수를 지정한다. {m}이라고 쓰면 바로 앞 패턴이 정확히 m 번 반복됨을 나타낸다.

패턴	a{3}
매치되는 문자열	<u>aaa</u> <u>aaaaaaaaa</u> 123aa345 <u>aaaaa</u>
매치되지 않는 문자열	a aa

{m,}이라고 쓰면 바로 앞 패턴이 m 번 이상 반복 됨을 나타낸다.

패턴	a{3,}
매치되는 문자열	<u>aaa</u> <u>aaaaa</u> <u>aaaaaaaaaaaaaaaaa</u> 123aa345 <u>aaaaa</u>
매치되지 않는 문자열	a aa

{m,n}이라고 쓰면 바로 앞 패턴이 m 번 이상 n 번 이하 반복 됨을 나타낸다.

패턴	a{3,5}
매치되는 문자열	<u>aaa</u> <u>aaaa</u> <u>aaaaa</u> <u>aaaaa</u> <u>aaaaa</u> 123aa345 <u>aaaaa</u> aaa
매치되지 않는 문자열	a aa

“^”기호는 라인의 처음을 의미한다. “[^”처럼 사용될 때와는 전혀 다른 의미임에 주의하자.

패턴	^#.*
매치되는 문자열	<u>#</u> <u># This is all matched</u> <u>#AC comment</u>
매치되지 않는 문자열	# AAa aaa # this

“\$”기호는 라인의 끝을 의미한다.

패턴	end\$
매치되는 문자열	<u>end</u> aaa # this <u>end</u>
매치되지 않는 문자열	end is hahaha end.

“?”는 바로 앞 패턴이 없거나 한번만 나타나는 경우를 나타낸다. 이것을 이용하여 양의 정수를 찾는

정규 표현식을 써보면 다음과 같다. 여기서 “\”는 “+”를 정규 표현식의 연산자로 사용하는 것이 아니라\n는 것을 명시하기 위해 사용되었다.

패턴	\+?[0-9]+
매치되는 문자열	<u>2</u> <u>23456</u> <u>+123456897</u> <u>-87.99</u> <u>-1234</u> 123 is ----- <u>+123454</u> abcd is hahaha ++++ <u>+123454</u> abcd abcd <u>123454</u> abcd is hahaha
매치되지 않는 문자열	abcd BCD abCD

“()”는 패턴을 그룹으로 만들어 패턴 전체에 대한 연산이 가능하도록 한다. 그리고 “()”안의 정규 표현식에 매칭된 패턴을 기억 시켜 두었다가 프로그램의 다른 부분에서 이용할 수 있도록 해 준다. 이것에 관해서는 다음 절에서 자세히 살펴 보기로 한다. “()”를 이용하여 실수를 찾아 내는 정규 표현식을 써보면 다음과 같다. 구두점 “.” 앞의 “\”도 구두점이 정규 표현식의 메타 문자로 사용된 것이 아니라는 것을 나타내고 있다. 조금 복잡하지만 그 의미를 잘 따져 보면 실수의 여러 가지 표현 방식을 모두 나타내고 있음을 알 수 있을 것이다.

패턴	[+-]?[0-9]*\.[0-9]+([eE][+-]?[0-9]+)?
매치되는 문자열	<u>23456.12</u> <u>+123456897.34</u> <u>-456.124</u> <u>+.42</u> <u>-.64</u> <u>.23</u> <u>45.24e98</u> <u>32.34e-134</u> <u>.23e+54</u> <u>436.34E-99</u> <u>-22.54E+77</u> +3243 <u>+234.34e-19</u> +1239 abcde <u>154.34</u> -1239AAA Mean is <u>-456.124</u> and Sum is 345.34
매치되지 않는 문자열	345 -21

“|”는 “OR”의 의미로 사용된다. 즉, 왼쪽의 정규식 혹은 오른쪽의 정규식과 일치하면 일치하는 것으로 간주한다는 의미이다. “|”를 사용하여 음수와 양수를 모두 찾을 수 있는 정규 표현식을 써보면 다음과 같다.

패턴	(\+ -)?[0-9]+
매치되는 문자열	<u>234</u> <u>+12345</u> <u>-45</u> aaa <u>+234</u> +1239 Sum is <u>+54</u> and
매치되지 않는 문자열	AAA abCD

정규 표현식을 적용하는데는 2 가지 규칙이 있다.

First Rule
Greedy Rule

First Rule 이란 한 문자열 내에서 패턴과 매칭되는 부분이 여러 개 있을 경우 가장 먼저 나타나는 부분을 매칭 부분으로 규칙이다. 그리고 Greedy Rule 이란 패턴에 매칭되는 부분을 가능한 한 많이 포함시킨다는 규칙이다. 다음의 예에서 실제로 매칭된 부분을 보면서 이 2 가지 규칙의 의미를 음미해 보기 바란다.

패턴	a+
매치되는 문자열	<u>aa</u> is aaaaaa and aaaa <u>aaaaaa</u> 12 or aaaaaaaaaaaaaa13 <u>AAAaaaaaa</u> + 1243 aaa

3.6.2. Backslash Escape

자주 사용되는 패턴을 미리 정의하여 간편하게 이용할 수 있도록 한 기능이다. 이 기능은 Tcl/Tk 8.1 이상에서만 사용 가능하다는 점에 유의하기 바란다. 제공되는 Backslash Escape 중에 자주 사용될 수 있는 것들은 다음과 같다. **표시 된 것들은 Tcl/Tk 8.0 이하 버전에서도 제공되던 것들이다. 모두 글자 하나를 의미함을 명심하자.

Backslash Escape	의미
\d	숫자
\D	숫자가 아닌 모든 것
\s	공백 문자 (스페이스 혹은 탭)
\S	공백 문자가 아닌 모든 것
\w	워드 구성 성분 중 하나 (알파벳과 숫자 혹은 밑줄 문자)
\W	워드 구성 성분이 아닌 모든 것
\n	개행 문자 (new line) **
\t	탭 문자 (tab) **
\a	Alert
\b	backspace
\e	ESC 키

그림 3 주요 Backslash Escapes

Backslash Escape 를 사용하면 한 단어를 찾는 패턴을 다음과 같이 간단히 작성할 수 있다.

패턴	\w+
매치되는 문자열	<u>a123 a</u> is aaaaaa and aaaa <u>+!- 12 b 34</u> * 1243 aaa

3.6.3. regexp 명령어

regexp 명령어는 정규 표현식에 주어진 문자열이 매칭될 때 1 을 리턴하고 매칭되지 않으면 0 을 리턴한다. 그리고 원하는 문자열을 지정한 변수에 저장할 수 있는 방법을 제공한다. regexp 의 정확한 문법은 다음과 같다.

```
regexp ?flags? expression string ?allmatch variable1 variable2 ... ?
```

expression 에는 정규 표현식이 오고 string 에는 정규 표현식으로 매칭 여부를 판별할 문자열이 오는 자리이다. 여기까지만 명시하면 매칭 여부만을 알 수 있다.

만일 그 다음에 변수를 지정하면 첫번째 지정된 변수에는 매칭되는 패턴 전체가 복사된다. 그리고 그 다음 변수에는 사용자가 “()”를 써서 매칭된 패턴 중에서 특정한 부분을 지정했을 경우 차례대로 저장된다. 다음에 이에 대한 예제가 있다. 여기서 주의할 점은 정규 표현식의 가독성(readability)을 높이기 위하여 공백문자를 임의로 삽입해서는 안 된다는 점이다. 공백 문자도 엄연히 하나의 패턴으로 작용하기 때문에 원치 않는 결과를 초래하게 된다.

regexp 명령어에 적용 가능한 flag 들은 다음과 같다.

- * -nocase : 정규 표현식에 나타난 소문자들은 문자열의 같은 문자이지만 하면 소문자와도 대문자와도 매칭된다.
- * -indices : 문자열 안에서 매칭된 패턴의 시작 인덱스와 끝 인덱스의 쌍을 주어진 변수에 기억 시킨다. 문자열 인덱스는 0 부터 시작한다.
- * -expanded : 패턴을 여러 줄에 걸쳐서 기술할 수 있도록 해준다. 그리고 각 패턴에 대하여 주석을 달 수 있도록 해준다.²⁰
- * -- : 정규표현식이 “-”로 시작할 경우 이 플래그를 적용하여 플래그로 오해할 수 있는 소지를 없앤다.

```
% set test_string host:address:phone:name
host:address:phone:name
% regexp {([a-z]+):([a-z]+):([a-z]+)} $test_string all p1 p2 p3
1
%set all
host:address:phone
% set p1
host
% set p2
address
% set p3
phone
% regexp -indices {([a-z]+):([a-z]+):([a-z]+)} $test_string all p1
1
% set all
0 17
% set p1
0 3
```

위에서 보는 바와 같이 regexp 를 사용하여 문자열에서 원하는 부분을 뽑아낼 수 있다. 뽑아내고자 하는 패턴을 “()”로 감싸고 난 후 “()”의 순서대로 변수의 이름을 배열하면 된다. 단, 여기서 주의할 점은 변수 all 의 값에서 보는 바와 같이 첫번째 변수에는 매칭된 모든 부분이 저장된다는 것이다.

²⁰ Tcl/Tk 8.1 버전 이상에서만 이 옵션을 제공한다.

regexp 를 사용할 때 잦은 실수 중의 하나는 다음과 같은 경우이다.

```
% set string yahoo
yahoo
% regexp { yahoo } $string
0
```

위의 예제에서 regexp 의 실행 결과가 1 이 나와야 하는데 0 이라고 이상하게 생각하는 사람이 많다. 여기서 0 이 나올 수 밖에 없는 이유는 매칭 패턴에 공백 문자가 포함되어 있기 때문이다. 따라서 다음과 같이 고쳐야만 원하는 결과를 얻을 수 있다.

```
% set string yahoo
yahoo
% regexp {yahoo} $string
1
```

이 예에서 보는 바와 같이 공백문자도 패턴이라는 점을 명심하여야 한다. 정규표현식의 패턴을 쓸 때는 반드시 필요한 패턴만을 써야한 다는 점을 잊지 말아야 한다.

-expanded 옵션을 사용하면 다음과 같이 패턴에 대한 주석을 달 수가 있다. 보통의 경우 정규 표현식은 매우 복잡하여 해석하기가 매우 곤란한 경우가 많은데 이 옵션을 이용하면 이러한 문제를 극복할 수 있다. 다음의 두가지 명령어는 똑같은 기능을 수행한다. -expanded 옵션을 사용했을 때 어떤 장점이 있는지 눈으로 확인해 보기 바란다. -expanded 옵션을 사용했을 때 패턴내에 삽입된 모든 공백 문자와 개행 문자 그리고 주석은 무시된다. -expanded 옵션을 사용했을 때 공백문자나 개행문자를 패턴에 포함 시키고자 할 때는 backslash escape 중 \s 와 \n 을 사용해야 한다.

```
regexp {[+-]? \d* \. \d+} $string matchpart
```

```
regexp -expanded { [+-]?      # 실수의 부호가 나와도 좋고 안나와도 좋고
                    \d*        # 실수의 정수부분은 있어도 좋고 없어도 좋고
                    \.         # 소수점은 반드시 있어야 되고
                    \d+        # 실수에서 소수부분은 반드시 있어야 된다.
                    } $string matchpart
```

3.6.4. regsub 명령어

regsub 는 정규 표현식을 이용하여 치환 연산 (substitution)을 수행하도록 하는 명령어이다. 이 명령어의 정확한 문법은 다음과 같다.

```
regsub ?flags? expression string subspec variable
```

regexp 에서와 마찬가지로 expression 과 string 에는 각각 정규 표현식과 매칭 여부를 검사할 문자열이 온다. 다음에 어떻게 치환을 행할 것인지를 명시하는 subspec 이 온다. 그리고 마지막으로 치환 결과를 저장할 변수가 온다. 이 명령은 수행 후 치환이 일어난 개수를 리턴한다.

regsub 에 적용 가능한 플래그들은 다음과 같다.

- * -nocase : 정규 표현식의 소문자는 같은 문자라면 문자열의 대문자, 소문자 모두와 매칭 된다.
- * -all : 매칭이 여러 번 일어나면 모두 치환 하라
- * -- : 정규 표현식이 “ - ” 문자로 시작할 경우 플래그로 오해하지 않도록 하기 위한 플래그 이다.

```
% set test_string host:address:phone:name
host:address:phone:name
% regsub {([a-z]+):([a-z]+)} $test_string {\1 + \2} sub_string
1
% set sub_string
host + address:phone:name
% set test_string everybody
% regsub {e} $test_string {E} sub_string
1
% set sub_string
Everybody
% regsub -all {e} $test_string {E} sub_string
2
% set sub_string
EvEverybody
```

regexp 에서 처럼 뽑아내고 싶은 부분이 있으면 이 부분을 “()”로 감싸준다. “()”의 순서에 따라 각 괄호안의 패턴에 매칭된 부분은 \1, \2, \3 ... 등의 변수에 저장된다. 이를 이용하여 치환 패턴에 사용할 수 있다. 단, 여기서 주의할 점은 \1, \2, \3 ... 등과 같은 변수는 치환 패턴 안에서만 사용할 수 있고 다른 곳에서는 사용할 수 없다는 점이다.

regsub 명령어에서 정규 표현식 패턴을 쓸 때 유의할 점은 “\t”가 Tab 문자로 매칭이 되지 않는다는 점이다. 현재 버전의 버그인지 다른 이유로 인한 것인지는 알 수 없다. Tab 문자를 패턴에 넣기 위해서는 실제로 Tab 문자를 치는 수 밖에 없다.

연 습 문 제

1. 다음과 같은 데이터를 입력 받아 출력 결과와 같이 변환하시오. 데이터의 입력은 데이터 분석 연습 문제에서와 같은 방법으로 하시오.

입력 데이터

```
DAT_LOT3
DAT_LOT1
DTT_PHDA4
HHUI_EE2
```

출력 결과

```
LOT-3DAT
LOT-1DAT
PHDA-4DTT
EE-2HHUI
```


2. 다음의 데이터 파일은 간략화된 Verilog 디자인 파일이다. 이를 분석하여 입출력의 종류를 출력하는 프로그램을 작성하시오. 데이터 파일의 입력은 데이터 분석 연습문제에서와 같은 방법으로 하시오.

입력 데이터

```
module aa(a, b, c, d);
input a, b;
output c;
inout d;

....;

endmodule;

module bb(a1, a2, a3, a4, a5, a6, a7,a8);
input a1;
input a2;
input a3;
output a4,
        a5,
        a6 ;
inout a7;
inout a8;

....

endmodule;
```

출력 결과

```
MODULE NAME : aa
  inputs : a b
  outputs : c
  inouts : d
MODULE NAME : bb
  inputs : a1 a2 a3
  outputs : a4 a5 a6
  inouts : a7 a8
```

3.7. 함수

Tcl 에서 함수를 정의하고 사용하는 방법을 알아본다. 그리고 함수 내에서 사용하는 변수의 참조 범위와 관련된 여러 가지 사항들에 관하여 알아 본다.

3.7.1. 함수의 정의

함수를 정의하는 문법은 다음과 같다.

```
proc procName parameterNames procBody
```

예를 보이면 다음과 같다. 여기서 주의할 점은 함수를 사용하기 전에 반드시 먼저 함수를 정의해야 한다는 점이다. 다시 말해서 함수 정의 부분이 파일의 앞 쪽에 오고 파일의 뒤 쪽에서 함수 호출 부분이 있어야 한다²¹.

```
proc factorial { a } {  
    set result 1  
    for { set t 1 } { $t <= $a } { incr t } {  
        set result [ expr $result * $t ]  
    }  
    return $result  
}
```

3.7.2. 함수의 사용

함수를 사용하는 방법은 일반 명령어를 사용하는 방법과 같다. 보는 바와 같이 함수에 넘겨 줄 Argument 가 있을 경우 이를 공백문자를 사이에 두고 나열해 주면 된다. 공백이 포함된 문자열을 함수의 Argument 로 넘겨줄 때에는 “”로 감싸 주면 된다.

```
% factorial 4  
24
```

3.7.3. Default Parameter Value

함수의 Argument 을 미리 정해 놓고 함수 호출 시 Argument 의 값을 명시하지 않는 경우에는 미리 정해진 값을 사용하도록 할 수 있다.

```
% proc yahoo { {a 2} {b 3} {c 4} } {  
    puts [expr $a + $b + $c]  
}  
% yahoo  
9  
% yahoo 4  
11  
% yahoo 4 5  
13  
% yahoo 4 5 6  
15  
% yahoo 4 5 6 7  
called "yahoo" with too many arguments
```

²¹ 이러한 제약은 Tcl/Tk 버전이 올라 가면서 없어질 것이라고 예상되지만 현재는 이러한 제약이 있다.

3.7.4. Parameter 갯수의 유동성

일반적인 경우에 함수의 Parameter 갯수는 일정하게 정해져 있다. 따라서 함수 호출시 정해진 Parameter 의 갯수보다 많은 인자를 넘겨줄 때는 다음과 같이 에러가 발생한다.

```
% proc haha { a } {  
    puts $a  
}  
% haha 1  
1  
% haha 1 2 3  
called "haha" with too many arguments  
%
```

그러나 어떤 경우에는 함수의 Parameter 갯수를 명확히 정하기 힘든 경우가 있다. 이런 경우 args 라는 특수한 Parameter 를 이용하여 다음과 같이 프로그램할 수 있다. args 라는 Parameter 가 존재하는 경우는 함수에 정의 된 Parameter 의 갯수보다 많은 인자가 넘어올 경우 나머지는 모두 args 라는 변수에 리스트 형태로 저장된다. args 는 반드시 맨 마지막에 와야 한다.

```
% proc hoho { a args } {  
    puts "a : $a"  
    puts "args : $args"  
}  
% hoho  
no value given for parameter "a" to "hoho"  
% hoho 1  
a : 1  
args :  
% hoho 1 2  
a : 1  
args : 2  
% hoho 1 2 3 4 5 6  
a : 1  
args : 2 3 4 5 6
```

3.7.5. 변수 참조 범위

Tcl에서는 특별한 선언을 하지 않은 변수들은 모두 local 변수로 만들어 진다. local 변수란 함수가 실행되는 동안만 존재하고 함수의 실행이 끝나면 사라지는 변수를 말한다. Global 변수를 함수내에서 참조하기 위해서는 반드시 사용하기 전에 global 변수임을 다음과 같이 명시하여야 한다.

```
proc mySub { } {  
    global a      ; #변수 a는 global 변수임을 선언  
    set a 5  
    set b 5      ; #변수 b에 대한 특별한 선언이 없으므로 b는 local 변수로 생성됨  
}
```

Tcl 함수 내에 존재하는 변수는 이름과 상관 없이 독립적으로 존재한다. 따라서 여러 함수 안에서 같은 이름의 변수를 서로 간섭 받지 않고 사용할 수 있다. 그러나 같은 함수 내에서는 같은 이름의 변수는 같은 변수로 본다. 다음의 예에서 보는 바와 같이 global 변수와 같은 이름의 local 변수가 사용되는 경우 global 변수는 함수내에서 보이지 않게 된다.

```
% set a 5  
5  
% proc ttt {} {  
    set a 2  
    puts $a  
}  
% proc kkk {} {  
    set a 4  
    ttt  
    puts $a  
}  
% kkk  
2  
4  
% set a  
5
```

Tcl 함수의 매개변수로 변수가 주어졌을 때 변수 자체를 넘겨 주지 않고 변수의 값을 함수에게 넘겨 준다. 따라서 함수 내에서 함수 Argument의 값을 변경하여도 매개변수로 주어진 변수의 값에는 변함이 없다.²² 변수의 값을 바꾸어 주는 함수를 작성하기 위해서는 upvar 명령어를 사용하여야 한다.

```
% set a 5  
5  
% proc ttt { a } {  
    set a 2  
}  
% ttt a  
2  
% set a  
5
```

²² 이러한 방식을 call by value 라고 한다.

3.7.5.1. 전역 변수 (Global Variable)

전역 변수를 함수 안에서 사용하기 위해서는 `global` 이라는 명령어를 사용한다. `global` 이라는 명령어 다음에 주어지는 변수들은 모두 전역 변수라는 것을 Tcl 인터프리터에 알린다. 전역 변수는 어떤 함수에서도 같은 이름으로 참조 가능 하다.

다음의 예제를 음미하면서 전역 변수 사용법을 이해하도록 한다.

```
% set a 10
10
% set b 20
20
% proc ttt {} {
    puts "$a $b"
}
% ttt
can't read "a": no such variable
% proc hhh {} {
    global a b
    puts "$a $b"
}
% hhh
10 20
% proc kkk {} {
    global a
    set a 5
}
% kkk
% set a
5
```

버전 8.0 이전 버전에서는 다음과 같이 배열을 전역 변수로 이용하기 위해서 `global` 다음에 `array` 를 써서 배열임을 명시해야 한다.²³ 그러나, 버전 8.0 이후 부터는 배열 변수에 대해서도 `array` 라는 키워드를 명시하지 않아도 되도록 바뀌었다.

```
% set TABLE("my name") nara
nara
% proc ttt {} {
    # 버전 8.0 이전에는 다음과 같이 array 임을 명시해야 했음
    global array TABLE
    puts $TABLE("my name")
}
% ttt
nara
```

²³ Tcl 8.0 에서는 굳이 `array` 를 명시하지 않고 `global` 만 써줘도 잘 동작한다.

3.7.5.2. upvar

upvar 명령어는 변수의 참조 범위를 변경시켜주는 명령어이다. upvar 명령어의 사용 형식은 다음과 같다.

```
upvar ?stackLevel? varName localVar
```

upvar 명령어를 이용하여 Call by Name 형식의 매개변수를 구현할 수 있다. Call by Name 이란 함수의 매개변수가 변수일 때 변수 자체를 함수로 넘겨 주는 방식을 말한다. Call by Name 방식에서는 함수 내에서 매개변수로 넘어온 변수의 값을 바꿀 수 있다. 다음에 그 예제가 있다.

```
#!/usr/local/bin/tclsh

proc MyIncr { varName } {

    upvar $varName a
    set a [ expr $a + 1 ]

}

set a 5

MyIncr a ; #MyIncr will change the value of a

puts $a
```

위에서 MyIncr 을 호출할 때 변수명 앞에 “\$” 기호가 없을 주의하자. 서브루틴으로 변수의 값을 넘기는 것이 아니라 변수의 이름을 넘기는 것이다. 이러한 서브루틴 호출 방식을 Call by Name 이라고 한다. 서브루틴으로 넘겨진 변수 이름을 가지고 메모리 상에 존재하는 변수값 저장 위치를 찾아내어 그 변수의 값을 바꾸거나 값을 참조할 수 있도록 해주는 명령어가 바로 upvar 인 것이다.

위와 같은 프로그램을 작성하라고 했을 때 보통 다음과 같은 실수를 하기 쉽기때문에 각별한 주의를 해야 한다.

```
proc MyIncr { varName } {

    # This is not valid programming
    # below statement has no effects

    set varName [ expr $varName + 1 ]

}
```

함수가 호출되면 스택에 그 함수에서 사용할 지역 변수 (local variable)들을 저장하게 된다. 따라서 함수가 함수를 부르고 그 함수에서 또 함수를 부르고 이런 식으로 반복되면 점점 스택의 깊이는 깊어지게 된다. upvar 명령어에서 stackLevel이라는 것은 이것을 말한다.

stackLevel 이 0 이면 바로 자신의 스택을 말하고 1 이면 자신을 호출한 함수의 스택을 말한다. 다음 예제를 통하여 그 의미를 이해하도록 하자.

```
% proc p1 {} {
    upvar 1 a a1
    upvar 2 a a2
    upvar 3 a a3

    set a 1

    puts "a0 = $a"
    puts "a1 = $a1"
    puts "a2 = $a2"
    puts "a3 = $a3"
}
% proc p2 {} {
    set a 2
    p1
}
% proc p3 {} {
    set a 3
    p2
}
% set a 4
4
% p3
a0 = 1
a1 = 2
a2 = 3
a3 = 4
%
```

위의 예제를 실행 시켰을 때 서브루틴 p1 이 실행되고 있는 순간 스택의 모양을 간단하게 그려 보면 다음과 같다. 각 함수의 실행이 끝나면 각 함수가 stack 상에서 차지하고 있던 공간은 사라진다.

P1 의 데이터 a : 1	Stack Level 0
P2 의 데이터 a : 2	Stack Level 1
P3 의 데이터 a : 3	Stack Level 2
Global 데이터 a : 4	Stack Level 3

그림 4 stack 의 snap shot

전역 변수들이 존재하는 스택을 가리키는 특별한 코드는 “#0”이다.

```
% proc p1 {} {  
    upvar #0 a aglobal  
    puts "a = $aglobal"  
}  
% proc p2 {} {  
    p1  
}  
% proc p3 {} {  
    p2  
}  
% set a 4  
4  
% p3  
a = 4  
%
```

stackLevel 을 0 으로 하면 변수 별명(Variable Alias)을 만들 수 있다.

```
% proc ttt {} {  
    set a 5  
    upvar 0 a b  
    set b 1  
    puts "a = $a"  
}  
% ttt  
1
```


3.8. 파일 및 디렉토리 조작

스크립트 프로그래밍에서 가장 많이 사용되는 것이 파일을 열고 그 파일의 내용을 읽거나 그 파일에 결과를 출력하는 것일 것이다. 또한 파일과 디렉토리를 생성, 복사, 이동하고 그 속성(Attribute)을 바꾸는 일도 매우 자주 한다. Tcl/Tk 는 이러한 일들을 수행할 수 있는 강력한 기능을 제공한다. 또한 일반적인 파일 뿐 아니라 Pipe 까지도 파일처럼 사용할 수 있도록 해주고 있다.

3.8.1. OS 마다 다른 파일 경로 표기법

파일의 경로를 표현하기 위해 사용되는 표기법이 OS 마다 제각각이다. 하지만 Tcl/Tk에서는 모든 파일명을 UNIX 스타일로 사용하면 Tcl/Tk 인터프리터가 현재 사용하고 있는 OS에 맞게 자동적으로 변환해 준다. 따라서 스크립트 작성시 Relative Path Name 을 이용하면 파일 경로명을 OS에 따라 달리 줄 필요가 없어 진다.

```
% dir d:/wonnr/tcl
```

드라이브 D에 이름이 없습니다

일련 번호 17FE-0C66

디렉토리 D:\wonnr\TCL\교재\예제

```
.                <DIR>          98-09-29  10:53 .
..               <DIR>          98-09-29  10:53 ..
TREE    TCL           978  98-09-19   9:12 tree.tcl
MOVE    TCL           592  98-09-28  10:47 move.tcl
RES     TCL           201  98-09-28  15:36 res.tcl
TEST    RES           313  98-09-28  15:35 test.res
MENU    TCL          2,665  98-09-28  13:52 menu.tcl
TEST    TCL            19  98-09-29  14:13 test.tcl
        6 개 파일              4,768 바이트
        2 개 디렉토리          606,081,024 바이트 사용가능
%
```

그러나 Path Name 을 다른 프로그램의 Argument 로 넘겨 줘야 할 경우에는 각 OS에 따라 OS가 정하는 규칙에 맞는 경로명을 만들어 줘야 한다. 이러한 경우를 대비하여 Tcl/Tk에서는 file join 명령어와 file split 명령어를 제공하고 있다.²⁴

3.8.2. 파일 입출력

파일로부터 자료를 입력받거나 자료를 출력하기 위해서는 다음과 같은 3 가지 단계를 거쳐야 한다.

파일 열기

파일로 데이터를 입출력

파일 닫기

²⁴ file join 과 file split 에 관한 내용은 “3.8.3file 명령어”에 나와 있다.

3.8.2.1. 파일 열기

파일을 열기 위해서 사용하는 명령어는 `open` 명령어이다. `open` 명령어는 파일을 연 후 그 파일에 대한 핸들러를 리턴한다.

```
open fileName ?accessType? ?permissions?
```

`fileName` 에는 열고자 하는 파일의 이름이나 경로명을 주면 된다. `accessType` 에 가능한 값은 표 10에 정리하였다. 사용할 수 있는 값의 종류는 간략형과 POSIX 형으로 나눌 수 있는데 이 둘을 동시에 같이 사용할 수는 없다. 간략형은 C 언어의 라이브러리 함수인 `fopen()`에서 사용하는 값과 의미가 같고, POSIX 형은 `open()`함수에서 사용하는 값과 의미가 같다. POSIX 형은 리스트를 사용하여 여러 값을 조합할 수 있다.

```
set dataFileIN [ open ../DATA/result01.dat r ]  
  
set dataFileOUT [ open ../DATA/result02.dat { RDWR CREATE } ]
```

파일을 열 때는 항상 예상치 않은 오류가 발생할 수 있다는 점에 유의하여야 한다. 이에 대비하여 `catch` 구문을 이용 해 다음과 같은 조치를 취하여 주는 것이 좋다.

```
if [ catch { set fileID [ open "my.dat" r ] } errorMessage ] {  
    puts stderr "Error : $errorMessage"  
}  
...  
close $fileID
```

이 경우 Open 된 File 의 핸들러는 `fileID` 라는 변수에 저장이 되고 File Open 시 에러가 발생하면 그에 대한 에러 메시지가 `errorMessage` 라는 변수에 저장되게 된다. 위와 같은 코드는 File Open 의 Idiom 이라고 할 수 있으므로 꼭 익혀 두도록 하자. 보다 간단하게 다음과 같이 프로그램할 수도 있다.

```
if [ catch { open "my.dat" r } fileID ] {  
    puts stderr "Error : $fileID"  
} else {  
    ...  
    close $fileID  
}
```

이 경우에는 에러 없이 파일이 열린 경우에는 파일 핸들러가 `fileID` 에 저장되고 에러가 발생한 경우에는 `fileID` 라는 변수에 에러 메시지가 저장된다. 따라서 위와 같이 에러가 발생한 경우와 발생하지 않은 경우 `fileID` 의 내용이 다르므로 사용에 유의하여야 한다.

Access Type		의미
간략형	r	읽기 전용. 파일은 반드시 존재해야 함
	r+	읽고 쓰기 가능. 파일은 반드시 존재해야 함
	w	쓰기 전용. 파일이 이미 존재하면 덮어 쓴다.
	w+	읽고 쓰기 가능. 파일이 이미 존재하면 덮어 쓴다.
	a	쓰기전용. 존재하는 파일의 내용이 보존됨. 파일이 반드시 존재해야 함
	a+	읽고 쓰기. 존재하는 파일의 내용이 보존됨. 파일이 반드시 존재해야 함
POSIX형	RDONLY	읽기 전용
	WRONLY	쓰기 전용
	RDWR	읽기 쓰기
	APPEND	기존 파일 내용 보존
	CREATE	파일이 없다면 새로 만든다.
	EXCL	CREATE 과 함께 사용될 경우, 이미 파일 존재하면 에러를 발생
	NOCTTY	파일명이 terminal device 인 경우 terminal 제어를 허용하지 않는다.
	NONBLOCK	파일을 여는 모든 과정이 끝날 때 까지 스크립트의 실행을 멈추고 있는 것을 방지한다. 파일을 여는 과정이 끝나지 않았더라도 스크립트의 수행은 계속 된다.
	TRUNC	파일이 이미 존재하면 내용을 삭제하고 길이를 0 으로 만든다.

표 10 open access types

3.8.2.2. 파일 닫기

파일을 닫기 위해 사용되는 명령어는 close 이다. open 명령어가 리턴한 파일 핸들러를 주면 된다.

```
set dataFileIN [ open ../DATA/result01.dat r ]
close $dataFileIn
```

파일을 열고 close 를 이용하여 파일을 닫지 않았을 경우 스크립트가 종료되면 자동으로 파일은 닫힌다. 하지만 close 를 하지 않은 상태에서 많은 파일을 계속 열 경우 시스템 자원이 고갈되어 에러가 발생할 가능성이 있다.

3.8.2.3. 파일 쓰기

파일에 데이터를 쓰는 명령어는 puts 명령어가 있다. puts 명령어는 주어진 문자열과 새줄문자(new line character)를 출력한다.

```
puts ?-newline? ?channelID? stringToWrite
```

channelID 에는 open 명령어가 리턴한 파일 핸들러 값을 주면 된다. channelID 를 명시하지 않으면 기본적으로 puts 는 stdout 으로 출력한다. stdout 은 항상 열려 있으므로 따로 open 할 필요는 없다. stdout 외에 stdin, stderr 가 기본적으로 항상 열려 있다. stdout 과 stderr 는 출력 채널이며 stdin 은 입력 채널이다.

기본적으로 출력되는 새줄문자가 출력되지 않게 하기 위해서 -newline 옵션을 사용할 수 있다.

```
puts "Program Started"
set handle [open MyFileName w]
puts $handle "This line will be printed in MyFileName"
close $handle
puts "Program was over"
```

3.8.2.4. flush 명령어

파일에 출력을 실행할 때에는 기본적으로 버퍼에 관하여 동작하기 때문에 프로그램이 계속 동작 중인 동안에는 출력 파일을 열어 봐도 아무런 내용을 찾을 수 없는 경우가 대부분이다. 따라서 만일 서로 다른 프로그램이 같은 파일을 공유하면서 서로 데이터를 주고 받고자 할 경우 파일에 출력하는 프로그램이 완전히 끝나야지만 다른 프로그램이 데이터를 받아갈 수 있게 되는 문제점이 발생한다.

이러한 경우에 즉시 파일에 출력하기 위하여 flush 라는 명령어를 호출하면 출력하는 즉시 실제 파일에 출력 내용을 저장할 수 있다.

```
set handle [open MyFileName w]
puts $handle "This line will be printed in MyFileName"
flush $handle
close $handle
puts "Program was over"
```

3.8.2.5. 파일 읽기

파일을 읽는 명령어는 gets 와 read 가 있다.

gets ?channelID? ?variableName?

gets 명령어는 한 번에 한 줄씩 명시된 채널에서 읽어 들인다. channelID 를 명시하지 않으면 기본적으로 stdin 에서 한 줄을 읽어 들인다. 변수 이름을 variableName 에 명시하면 읽어들인 문자열은 변수에 저장하고 읽어들인 문자의 갯수를 리턴한다. 변수 이름을 명시하지 않으면 읽어들인 문자열을 리턴한다.

```
read ?-nonewline? channelID
read channelID nuberOfByte
```

read 명령어는 명시된 채널의 전체 내용을 읽어 들이거나 주어진 바이트 수 만큼의 내용을 읽어들인다. -nonewline 옵션이 주어지면 마지막 글자는 새줄 문자로 간주하고 버린다. read 명령어는 읽어들인 내용을 리턴한다.

다음에 gets 와 read 명령을 이용하여 “/etc/passwd”파일의 내용을 출력하는 프로그램이 있다.

```
#!/usr/local/bin/tclsh
set input [open /etc/passwd r]
while { [gets $input line] >= 0 } {
    puts $line
}
close $input
```

```
#!/usr/local/bin/tclsh
set input [open /etc/passwd r]
puts [read $input]
close $input
```

3.8.2.6. 파이프 열기

Tcl/Tk 에서는 device 파일을 포함한 일반적인 파일 뿐 아니라 Pipe 도 open 명령을 통하여 열 수 있다. 파일명을 “|”로 시작하면 파이프로 인식한다.

다음 프로그램은 “sort /etc/passwd”의 결과를 출력하는 프로그램이다. Pipe 를 읽기 전용으로 열어서 사용하고 있다.

```
#!/usr/local/bin/tclsh
set inputData [open "| sort /etc/passwd" r]
puts [read $inputData]
close $inputData
```

다음 프로그램은 “/etc/passwd”를 UNIX 명령어인 “sort”를 이용해 정렬하는 프로그램이다. 위의 프로그램과 다른 점은 Pipe 를 쓰기 전용으로 열었다는 데에 있다.

```
#!/usr/local/bin/tclsh
set input [open /etc/passwd r]
set output [open "| sort" w]
puts $output [read $input]
close $input
close $output
```

3.8.3. file 명령어

Tcl/Tk 에서는 파일과 관련된 많은 기능들을 file 명령어를 통하여 제공한다. file 명령어의 여러 가지 사용법을 표 11에 정리하였다.

문법	의미
file atime name	name 이 최종적으로 access 된 시간을 정수로 리턴한다. 이 정수는 1970 년 1 월 1 일부터 경과된 시간을 초로 환산한 시간이다.
file attributes name ?option?	파일 속성에 대한 여러 가지 값을 리턴한다.
file copy ?-force? src dest	src 를 dest 로 복사한다. -force 옵션은 dest 가 이미 존재하는 경우 덮어쓰도록 한다.
file delete ?-force? name	name 을 삭제 한다. -force 옵션은 비어 있지 않은 디렉토리도 삭제할 수 있도록 해준다.
file dirname name	name 의 문자열을 분석하여 디렉토리 path 에 해당하는 부분만을 추출하여 리턴한다. 디렉토리 path 에 해당하는 부분이 없으면 “.”을 리턴한다.
file executable name	실행 가능한 파일일 경우 1 을 리턴한다.
file exists name	name 이라는 이름을 가진 파일이 존재하는 경우 1 을 리턴한다.
file extension name	name 의 문자열 중 마지막 구두점 이후의 문자열을 리턴한다. 구두점이 없으면 빈 문자열을 리턴한다.
file isdirectory name	name 이 디렉토리이면 1 을 리턴한다.
file join path path ...	각 path 들을 하나의 path 로 합친다. path 를 합칠 때 사용되는 경로 분리자(path separator)는 각 OS 에 알맞은 것을 사용 한다.
file lstat name var	symbolic link 파일이 link 하고 있는 파일에 대하여 file stat 기능을 수행한다.
file mkdir name	name 이라는 디렉토리를 만든다.
file mtime name	name 이 마지막으로 수정된 시간을 정수로 리턴한다. 이 정수는 1970 년 1 월 1 일부터 경과된 시간을 초로 환산한 시간이다.
file nativename name	name 에 주어진 경로명을 각 OS 에 알맞는 형태로 바꾸어 리턴한다.
file owned name	name 의 소유자가 현재 사용자일 때 1 을 리턴한다.

file path type name	absolute, relative, volumerelative 중에 하나를 리턴한다. Absoute 은 name 이 절대 경로 일때, relative 는 상대 경로일때 리턴한다. volumerelative 는 “C:\wonnr”과 같이 volume 이 명시된 경우에 리턴한다.
file readable name	name 을 읽을 수 있는 권한이 있으면 1 을 리턴한다.
file readlink name	symbolic link 가 가리키고 있는 파일의 이름을 리턴한다.
file rename ?-force? old new	파일 이름을 바꾼다. -force 옵션은 new 에 명시된 이름과 같은 파일이 있는 경우 덮어 쓴다.
file rootname name	name 의 문자열이 구두점으로 구분되어 있을 때 구두점 왼쪽의 모든 값을 리턴한다. 구두점이 없을 경우에는 name 의 문자열을 그대로 리턴한다.
file size name	파일의 크기를 바이트수로 리턴한다.
file split name	name 에 주어진 경로명에서 경로 분리자(path separator)로 나뉘어진 요소들로 분리하여 리스트로 만든 후 그 리스트를 리턴한다. name 에 주어 지는 문자열에는 각 OS 에 알맞는 경로명을 사용해도 된다.
file stat name var	Kernel system call 인 stat 을 호출하여 결과로 리턴되는 파일에 관한 정보를 var 에 저장해 준다.
file tail name	name 에 주어진 경로명 중에서 맨 마지막 경로 분리자 (path separator)다음에 있는 문자열을 리턴한다.
file type name	파일의 type 을 검사하여 다음 값 중에 하나를 리턴 한다. file, directory, characterSpecial, blockSpecial, fifo, link, socket
file volume	현재 시스템에 사용 가능한 volume 들을 리스트로 만들어 리턴한다. (예 : { a:/ c:/ })
file writable name	쓸 수 있는 권한이 있으면 1 을 리턴 한다.

표 11 file 명령어 사용법

3.8.4. cd 와 pwd 명령어

UNIX 명령어와 마찬가지로 cd 는 현재 디렉토리를 다른 곳으로 옮기는 기능을 하고 pwd 는 현재 작업 디렉토리의 경로명을 리턴한다.

연 습 문 제

1. 현재 디렉토리 내에 존재하는 모든 파일들의 라인수의 합을 구하여 출력하는 프로그램을 작성하시오.
이 프로그램의 결과는 UNIX 명령어 `wc -l *` 를 실행한 것과 같은 결과임

3.9. 외부 프로그램과의 연동

Tcl 프로그램 내에서 외부 프로그램을 호출하는 방법과 Tcl 프로그램의 수행 종료 시 종료 상태를 외부 프로그램에게 알리는 방법에 대하여 알아 본다.

3.9.1. exec 명령어

exec 명령어는 외부 프로그램을 Tcl 스크립트 안에서 실행시켜 준다.

```
exec ?-keepnewline? unixCommand
```

exec 는 기본적으로 명령이 한 줄로 끝난다고 생각한다. 그러나 -keepnewline 을 명시하면 여러 줄에 걸친 외부 명령을 exec 를 이용하여 실행할 수 있다. 외부 명령어인 ls -al 을 수행하고 그 결과를 출력하는 스크립트는 다음과 같다. Windows 95/NT 인 경우는 ls 대신 dir 명령을 실행해 보도록 한다.

```
puts [exec ls -al]; # for UNIX
```

```
puts [exec dir]; # for Windows 95/NT
```

외부 프로그램이 실행되다가 에러가 발생한 경우 에러 메시지를 화면에 찍히지 않게 하고 스크립트 내부에서 처리하고자 한다면 다음과 같이 프로그램 하면 된다.

```
if { [ catch { exec haha } errMsg ] } {  
    puts "Error : $errMsg"  
} else {  
    puts "Result : $errMsg"  
}
```

위와 같이 프로그램하면 외부 명령어가 stdout 으로 출력하는 결과와 stderr 로 출력하는 결과가 모두 errMsg 에 저장되게 된다. stdout 으로 출력되는 결과와 stderr 로 출력되는 결과를 분리하고 싶은 경우 다음과 같이 프로그램하면 된다.

```
if { [ catch { set result [ exec haha ] } errMsg ] } {  
    puts "Error : $errMsg"  
} else {  
    puts "Result : $result"  
}
```

위와 같이 프로그램하면 stdout 으로 출력되는 메시지는 result 라는 변수에 저장되고 stderr 로 출력되는 메시지는 errMsg 라는 변수에 저장되게 된다.

만일 exec 로 실행하려는 프로그램이 Tcl 로 작성한 프로그램일 경우 위와 같이 catch 가 제대로 동작하기 위해서는 다음과 같은 조건을 만족해야 한다. 정상적으로 수행을 마쳤을 때는 Command Shell²⁵에 종료코드로 0 을 리턴하고 에러가 발생한 경우는 0 이 아닌 값을 리턴 한다.²⁶ 그리고 모든 에러 메시지는 stderr 로 출력한다. 종료 코드의 값은 exit 명령어의 Argument 로 주면 된다.

exec 에서도 UNIX 에서 제공하는 IO redirection 기능을 모두 사용할 수 있다. 다음에 그 예가 있다.

```
% exec sort < myDataFile | uniq >> mySortedData
```

exec 를 통하여 외부 명령을 수행하면 외부 명령이 끝날 때까지 기다린다. 이를 방지하기 위하여 명령

²⁵ UNIX 의 경우는 csh, bsh 등이 있고 Windows 는 command.com 이다.

²⁶ Tcl 에서는 exit 명령어를 사용하여 종료 코드 값을 정할 수 있다. C 언어에서는 exit() 함수를 제공한다.

어 맨 끝에 “ & ”를 붙여 외부명령을 백그라운드로 수행시킬 수 있다. 이 때 `exec` 는 process ID 를 리턴한다.

3.9.2. `exit` 명령어

`exit` ?exitCode?

`exit` 명령어는 Tcl 셸을 종료한다. Argument 을 지정하지 않으면 정상적인 종료 코드인 0 을 command shell 에 리턴한다. 매개 변수 값을 지정하여 원하는 종료 코드를 리턴할 수 있다. `exit` 을 사용하지 않고 Tcl 셸이 종료된 경우에는 정상적인 종료 코드를 리턴한다.

```
if { ... } {  
    # error case  
    exit 1  
}
```

스크립트가 정상적으로 종료하는 경우에는 종료코드 0 을 리턴하고 에러가 발생한 경우에는 0 이 아닌 값을 종료 코드로 리턴하여야 한다.

3.10. 기타

앞의 장에서 언급되지 않았지만 중요한 내용들에 대하여 간략히 기술하였다.

3.10.1. 명령행 인수 (Command Line Argument)

Tcl 프로그램에서 명령행 인수는 `argv` 라는 리스트에 차례대로 저장되어 있다. C 언어에서는 `argv` 의 첫번째 원소는 프로그램의 이름이 저장되어 있으나 Tcl 에서는 첫번째 인수가 저장되어 있다. 프로그램의 이름을 알고 싶으면 `argv0` 라는 전역 변수를 참조 하면 된다. 인수의 갯수는 `argc` 라는 변수에 저장되어 있다.

다음을 실행시켜 보면서 그 의미를 확실히 이해 하도록 하자.

```
#!/usr/local/bin/tclsh

puts "Program name is : $argv0"
puts "Number of arguments : $argc"

foreach arg [ $argv ] {
    puts "Command Line Argument : $arg"
}
```

이 프로그램을 `comarg.tcl` 이라는 파일로 저장하였다면 UNIX 셸이나 DOSBOX 안에서 다음과 같이 실행하면 다음과 같은 결과가 나온다.

```
$ comarg.tcl haha hehe hoho
Program name is : comarg.tcl
Number of arguments : 3
Command Line Argument : haha
Command Line Argument : hehe
Command Line Argument : hoho
$ comarg.tcl 1 2
Program name is : comarg.tcl
Number of arguments : 2
Command Line Argument : 1
Command Line Argument : 2
$
```

3.10.2. 환경 변수 (environment Variable)

Command Shell 에는 여러 프로그램이 정보를 공유할 수 있도록 해주는 환경 변수가 있다. UNIX 와 Windows 의 대표적인 환경 변수로 실행 프로그램의 위치를 지정해 주는 `PATH` 를 들 수 있겠다. Tcl 인터프리터는 이러한 환경 변수들을 `env` 라는 전역 변수에 저장하고 있다. 다음을 `tclsh` 에 입력하여 환경 변수가 어떻게 세팅되어 있는지 각자 확인해 보기 바란다.

```
#!/usr/local/bin/tclsh

foreach varName [array names env] {
    puts "$varName = $env($varName)"
}
```

3.10.3. incr 명령어

incr 명령어는 제어구조는 아니지만 for 문에 매우 유용하게 사용되는 명령어이다. incr 명령어의 형식은 다음과 같다.

```
incr variableName ?deltaValue?
```

deltaValue 를 생략하고 사용하면 deltaValue 는 기본적으로 1 로 설정된다. 명령어가 수행되면 deltaValue 값 만큼 variableName 이 갖고 있는 값을 증가시킨다.

```
%set a 2
2
%incr a
3
%set a
3
```

deltaValue 를 명시하면 명시된 deltaValue 값 만큼 variableName 의 값을 증가시킨다.

```
%set a 2
2
%incr a -1
1
%set a 2
1
```

단, incr 명령어가 취할 수 있는 deltaValue 의 값은 정수값에 한정된다.

3.10.4. console 명령어

UNIX 의 경우는 기본 환경이 Text 모드이므로 이 명령어가 필요 없으나, Windows 나 매킨토시 같은 경우는 기본 환경이 그래픽 유저 인터페이스 이므로 메시지의 출력이나 사용자로부터 입력을 받을 때는 Text Mode 의 console 이 필요하다.

특히 Windows 와 매킨토시의 경우 console 명령어를 이용하여 console 박스를 보이지 않은 경우에는 puts 를 사용하여 메시지를 출력하여도 출력이 되지 않음에 유의 하여야 한다.

UNIX 의 경우는 console 이란 명령어 자체가 없어서 console 이란 명령어를 만나면 invalid command 라는 에러가 발생되어 스크립트가 실행되지 않음에 유의한다.

```
#!/usr/local/bin/tclsh

# 다음과 같이 콘솔 박스를 보이게 한다.
if { "$tcl_platform(platform)" != "unix" } {
    console show
}

# 다음과 같이 콘솔 박스를 감춘다.
if { "$tcl_platform(platform)" != "unix" } {
    console hide
}
```

3.10.5. after 명령어

일정한 시간 후에 어떤 명령을 실행하기 위하여 `after` 명령어를 사용한다. `after` 명령어의 사용 형식은 다음과 같다.

```
after miliSecond ?command?
```

10 초 간격으로 화면에 Hello 를 10 번 찍는 프로그램을 다음과 같이 작성할 수 있다. 이 때 사용되는 인터프리터는 `tclsh` 가 아니라 `wish` 임에 유의하도록 하자. `tclsh` 를 사용하지 않고 `wish` 를 사용하는 이유는 적어도 100 초 동안 프로그램이 끝나지않고 계속 실행하고 있어야 하기 때문이다.

```
#!/usr/local/bin/wish

proc PrintHello { } {
    global count

    puts "Hello"
    incr count

    if { $count < 10 } {
        after 1000 { PrintHello }
    }
}

set count 0

after 1000 { PrintHello }

if { "$tcl_platform(platform)" == "windows" } {
    console show
}
```

맨 마지막의 `command` 를 생략할 경우 지정한 시간만큼 기다렸다가 다음의 명령을 실행한다. 다음의 예제를 실행하면 Haha 가 찍히고 난 후 30 초 후에 HoHo 가 찍히게 된다.

```
#!/usr/local/bin/tclsh

puts Haha

after 3000

puts HoHo
```

3.10.6. 시스템 변수들

Tcl 인터프리터에 의해 미리 값이 할당되어 있는 시스템 변수들은 다음과 같다.

변수	의미
argc	command line argument 갯수
argv	command line argument 를 담고 있는 리스트
argv0	실행되고 있는 스크립트의 이름
env	환경 변수들의 값을 담고 있는 배열
tcl_interactive	스크립트를 실행하고 있으면 0, interactive 모드이면 1
tcl_library	실행 가능한 파일일 경우 1 을 리턴한다.
tcl_patchLevel	인터프리터의 정확한 버전 (패치된 버전)
tcl_platform	OS 와 컴퓨터에 관련된 정보를 담고 있는 배열 tcl_platform(byteOrder) : bigEndian or littleEndian tcl_platform(os) : 정확한 OS 이름 tcl_platform(osVersion) : OS 의 버전 tcl_platform(platform) : OS 의 계열 tcl_platform(machine) : 컴퓨터 종류
tcl_pkgPath	패키지를 찾기 위해 탐색하는 경로를 담은 리스트
tcl_prompt1	인터프리터의 프롬프트를 사용자가 정의할 때 사용 프롬프트를 찍는 데 사용되는 명령어를 할당해야 함
tcl_prompt2	인터프리터의 2 수준 프롬프트를 사용자가 정의할 때 프롬프트를 찍는데 사용되는 명령어를 할당해야 함 끝나지 않은 명령어를 여러줄에 걸쳐 입력할 때 나타나는 프롬프트임
tcl_version	인터프리터의 버전 정보
auto_path	스크립트 라이브러리를 찾는 경로를 담은 리스트
auto_index	OS 커맨드를 Tcl 명령어로 aliasing 하는데 사용 된다. 예) % set auto_index(haha) ls % haha
auto_noload	1 로 설정되면 라이브러리 기능을 사용할 수 없다.
auto_noexec	1 로 설정되면 자동 실행 기능을 사용할 수 없다. OS 커맨드들을 바로 사용할 수 없고 exec 를 사용해서만 실행 시킬 수 있게 된다.
geometry	wish 를 사용할 때만 의미가 있으며, -geometry 로 주어진 값을 갖고 있다. -geometry 옵션을 쓰지 않은 경우는 변수 없음

표 12 시스템 변수

3.11. Package

작성하고 있는 프로그램이 작을 경우에는 모든 프로그램에서 사용되는 모든 함수들을 한 파일에 넣어 놓아도 된다. 하지만 규모가 좀 더 커질 경우에는 서로 연관 있는 함수들끼리 모아 따로 다른 파일에 저장해 둔다면 한결 프로그램 소스를 관리하기가 간편해질 것이다. 그리고 자주 사용되는 함수들은 라이브러리화 해서 다른 프로그램에서도 쉽게 사용할 수 있도록 하면 좋을 것이다. Tcl/Tk 에서 제공하는 Package 기능을 이용하면 이러한 요구를 쉽게 충족시킬 수 있다.

3.11.1. package 만들기

package 를 만들기 위해서는 먼저 package 를 구성할 요소들을 만들고 그 다음에 package 인덱스를 만들어야 한다.

package 로 사용할 수 있는 것에는 Tcl 스크립트와 Tcl C API 를 이용해 C 언어로 구현된 binary file 이 있다. binary 파일은 UNIX 의 경우 *.so, Windows 의 경우 *.dll, 매킨토시의 경우 *.shlib 라는 확장자를 가진다. 이들은 모두 각 운영체제에서 제공하는 Dynamic link library 의 확장자들이다. 여기서는 Tcl 스크립트에 국한하여 설명하도록 한다.

3.11.1.1. package 구성 요소 만들기

package 를 구성하는 모든 Tcl 스크립트 파일은 자신이 속하는 패키지의 이름과 버전을 선언해야 한다. 이를 위해서 사용하는 Tcl 명령어는 package provide 이다.

```
package provide packageName MajorVersion.MinorVersion
```

이렇게 선언된 파일 내에 존재하는 모든 함수들은 같은 패키지에 속하고 그 패키지의 같은 버전에 속하게 된다. 한 패키지를 구성하는 함수들을 여러 개의 파일로 만들 수도 있다. 이 때는 각 파일마다 같은 패키지 이름과 같은 버전을 사용하면 된다.

```
#!/usr/local/bin/tclsh
package provide MyPackage 1.0
...
```

위의 명령이 수행된 다음 줄부터 나오는 함수들은 선언된 패키지에 속하게 된다. 한 파일 내에서 위의 선언을 여러 번 하게 되면 각각의 선언 다음에 나오는 함수들은 각각 다른 패키지에 속하게 된다.

패키지를 제공하는 파일에는 반드시 서브 루틴들만 들어 있어야 한다. 서브루틴의 내부가 아닌 곳에 존재하는 Tcl 명령이 있는 경우에는 원치 않는 결과가 발생하거나 Index 파일이 전혀 안만들어 질 수도 있음에 유의해야 한다.

3.11.1.2. package Index 만들기

package 의 구성 요소들을 적당한 디렉토리에 모아 놓고 pkg_mkIndex 라는 명령을 사용하여 package 인덱스를 만든다. 이 인덱스에는 패키지의 종류와 패키지를 구성하는 함수와 파일들에 대한 정보가 저장된다. package 를 이용하는 Tcl 스크립트를 수행할 때 원하는 함수를 효율적으로 찾기 위한 것이다. 같은 package 의 같은 버전을 구성하는 파일들은 될 수 있으면 한 디렉토리 내에 존재하는 것이 좋다. 물론 한 디렉토리 내에 여러 개의 패키지가 존재해도 된다.

예를 들어 “~/tcl/lib/GUI” 디렉토리 밑에 있는 Tcl 스크립트 파일들에 대해 모두 패키지를 만들고자 할 때 tclsh 를 실행시키고 다음과 같이 pkg_mkIndex 를 실행시킨다.

```
% pkg_mkIndex ~/tcl/lib/GUI *.tcl
```

이 명령을 실행하고 나면 지정한 디렉토리에 pkgIndex.tcl 이라는 파일이 생긴다. 이 파일을 열어 보면 지정한 파일에서 package 를 구성하는 서브루틴들에 대한 정보가 저장되어 있음을 알 수 있다.

pkg_mkIndex 명령어의 사용 형식은 다음과 같다. 첫번째에는 패키지가 존재하는 디렉토리 Path 를 주

야하고, 두번째에는 패키지를 구성할 함수들을 담고 있는 파일들에 매칭될 수 있는 패턴을 줘야 한다.

```
pkg_mkIndex dirPath filePattern ?filePattern ...?
```

이미 존재하는 package 의 내용을 수정할 경우 어떤 경우에 `pkg_mkIndex` 를 실행해야 할까? `pkg_mkIndex` 를 실행해야하는 경우는 새로운 서브루틴을 추가하였거나 이미 존재하고 있던 서브루틴을 삭제한 경우이다. 서브루틴의 추가나 삭제 없이 서브루틴의 내용만 수정한 경우에는 `pkg_mkIndex` 를 해 줄 필요가 없다.

3.11.2. package 사용 하기

`package` 를 사용하기 위해서는 먼저 `package` 를 찾기 위해 검색해야 할 디렉토리를 알려줘야하고 어떤 패키지의 어떤 버전을 사용할 것인지 명시해야 한다.

3.11.2.1. 패키지 탐색 경로 지정

Tcl 인터프리터는 패키지를 찾기 위해 검색해야 할 디렉토리들을 `auto_path` 라는 변수에 기억하고 있다. 만일 자신이 사용할 패키지가 Tcl 설치 시에 기본적으로 제공하는 패키지 디렉토리에 존재하지 않고 다른 곳에 존재한다면, 자신의 패키지 디렉토리를 이 변수에 추가 해야 한다.

`auto_path` 변수는 디렉토리 경로명들을 리스트형태로 보관하고 있으므로 다음과 같이 자신의 패키지 디렉토리를 `auto_path` 에 추가할 수 있다.

```
lappend auto_path ~/tcl/lib/GUI
```

3.11.2.2. 사용할 패키지 선언

`package` 에 대한 인덱스를 만들고 나면 패키지를 사용할 모든 준비는 끝났다. 이제 패키지를 사용하기 위하여 `package require` 라는 Tcl 명령어를 사용한다.

```
package require packageName ?version? ?-exact?
```

위와 같이 선언하면 스크립트 실행 중 그 파일 안에서 찾을 수 없는 함수들을 선언된 패키지 안에서 찾아 수행 한다. 한 스크립트 내에서 여러 개의 패키지를 사용하는 것도 물론 가능 하다.

패키지의 버전을 명시하지 않으면 같은 이름의 패키지들 중에서 가장 최신 버전의 패키지를 사용한다. 최신 버전은 버전 번호가 가장 큰 것이다. 버전 번호는 먼저 Major Version 을 비교하고 Major Version 이 같을 경우에만 Minor Version 을 비교 한다.

```
package require GUI 1.0
```

버전을 명시했을 때는 정확히 일치하는 버전의 패키지를 사용한다. 만일 정확히 일치하는 버전이 없다면 명시된 Major 버전 번호와 같은 Major 버전 번호를 갖는 패키지들 중에서 Minor Version 번호가 가장 큰 패키지를 사용한다. 하지만 Major 버전이 같더라도 명시된 Minor Version 번호보다 작은 Minor Version 번호를 가진 패키지는 사용할 수 없다. 예를 들어 1.1 버전을 사용하겠다고 했는데 1.1 버전이 없다면, 1.2 버전은 사용할 수 있지만 1.0 버전은 사용할 수 없다는 뜻이다.

`-exact` 옵션이 주어지면 정확히 버전 번호가 일치하는 패키지가 없을 경우 에러를 발생시킨다.

연 습 문 제

앞선 연습문제에서 구현한 Stack, Queue 을 Package 로 작성하고 이를 사용하는 예를 보이시오.

4. Tk

Tk 는 GUI 를 간편하게 제작할 수 있도록 도와주는 Tool Kit 이다. 처음에는 UNIX 에서 실행되는 X Windows 를 기반으로 설계되었으나, 현재는 매킨토시를 비롯하여 Windows, OS2 등 GUI 환경을 사용할 수 있는 거의 모든 OS 에 포팅되어 사용되고 있다.

Tk 는 X Windows 나 Windows95/NT, OS2 와 같은 Window 시스템을 기반으로 하고 있기 때문에 Window 많은 개념이 Window 프로그래밍과 일치한다. 그러나 Tk 를 사용하기 위해 Window 시스템에 대한 지식이 반드시 필요하지는 않다.

4.1. Tk 기본

4.1.1. 위젯(Widget)

GUI 는 여러 개의 윈도우들로 구성된다. Widget 은 GUI 를 구성하는 각각의 Window 하나에 해당한다고 생각하면 된다. Tk 는 이처럼 GUI 를 구성하는 Widget 을 다룰 수 있도록 여러가지 Tcl 명령어들을 제공하고 있다.

“2.4Tcl/Tk 맛보기”의 예제의 경우 위젯의 구조를 살펴 보면 그림 5 과 같다.

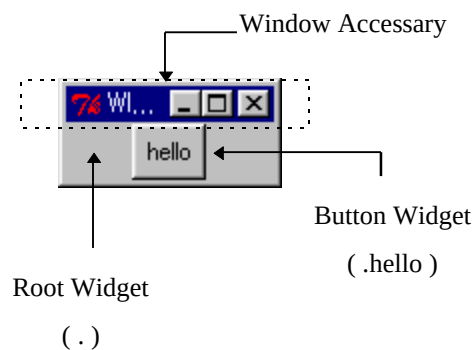


그림 5 Widget 구조

4.1.2. 위젯 클래스 (Widget Class)

모든 Widget 들은 자신의 Type 을 결정해 주는 Class Name 을 가지고 있다. 예를 들어 모든 button 위젯의 class name 은 “Button”이다. 그리고 모든 frame 위젯의 Class name 은 “Frame”이다.

이러한 클래스 명이 유용하게 사용되는 경우는 Binding 을 만들 때이다. 여기에 대한 자세한 내용은 “4.3Binding Commands to Events”에 설명하였다.

4.1.3. 위젯의 Instance Name

보통 위젯의 이름이라고 하면 위젯의 Instance Name 을 말한다. 위젯의 이름은 반드시 소문자로 시작해야 한다. 일반적으로 위젯의 이름에 대문자를 쓰지 않는 것이 관례화 되어 있다. 이는 위젯 클래스 이름과 구분하기 위해서이다. 위젯 클래스 이름은 모두 대문자로 시작한다.

위젯의 Instance Name 은 특별한 선언을 하지 않아도 모든 서브루틴내에서 참조 가능하다.

4.1.4. 위젯의 계층 구조

그림 5과 같이 간단한 예제의 경우에는 별 의미가 없지만 복잡한 GUI 를 구성하기 위해서는 위젯의 계층 구조에 대한 이해가 반드시 필요하다. Tk 에서 위젯의 계층구조는 구두점을 사용하여 표현된다.

위젯의 계층 구조 표현 방법은 UNIX 에서 디렉토리 path 표현 방법에서 “ / ” 대신 “ . ”을 사용한다고 생각하면 이해가 빠를 것이다. 최상위의 루트 위젯은 “ . ”으로 표현된다. 이것은 UNIX 에서 최상위 디렉토리를 “ / ”로 표현하는 것과 같다. 루트 위젯 위에 존재하는 서브 위젯들은 위젯의 이름들 사이에 “ . ”을 넣어 그 계층을 표시한다. 따라서 위의 예에서

```
.hello
```

은 루트 위젯의 Child 로 hello 위젯이 존재한다는 것을 나타낸다. hello 위젯이 haha 위젯을 포함한다고 가정하면 다음과 같이 나타낼 수 있다.

```
.hello.haha
```

여기서 hello 과 haha 는 프로그래머가 정한 위젯의 이름이다. 위젯의 이름을 붙이는데 있어서 “4.1.3 위젯의 Instance Name”절에서 설명한 것 외에 다른 제약은 없다.

4.1.5. Event Driven Programming

GUI 를 제공하는 프로그램을 작성할 때는 반드시 Event Driven Programming 을 해야 한다. Tk 의 경우도 예외는 아니다. Event Driven Programming 이란 어떤 Event 가 발생하면 그 Event 에 맞는 적절한 조치를 취할 수 있는 루틴들을 작성하는 것을 말한다. 이러한 프로그램은 계속 사용자가 어떤 Event 를 발생시키는 지를 감시하고 있다가 사용자가 발생시키는 Event 에 따라 적절한 루틴을 실행한다.

Tk 에서는 bind 명령어를 사용하여 Event 와 프로그래머가 정의한 루틴을 연관시킬 수 있다. Tk 에서 대부분의 Event 를 자동으로 처리해 주기 때문에 프로그래머는 자신이 관심이 있는 Event 에 대해서만 관심을 집중할 수 있다.

4.1.6. 위젯의 생성과 파괴

위젯의 생성은 각 위젯 클래스의 생성 명령어에 의해서 생성된다. 위젯 생성 명령어는 그 위젯이 속하는 클래스 이름에서 유추할 수 있다. 대문자로 시작하는 클래스 이름에서 맨 앞의 대문자를 소문자로 바꾸기만 하면 그 클래스에 속하는 위젯에 대한 생성 명령어가 된다. 예를 들어 Button 클래스에 속하는 위젯에 대한 생성 명령어는 button 이 되는 것이다.

생성된 위젯을 파괴하기 위해서는 destroy 명령어를 사용하면 된다. 각 사용 예를 보이면 다음과 같다.

```
button .b1
pack .b1
destroy .b1
```

4.1.7. 위젯의 배치

위젯을 생성하는 Tcl 명령어를 사용하여 위젯을 만들지만 한 상태에서는 윈도우 상에 아무 것도 나타나지 않는다.²⁷ 생성된 위젯을 화면에 나타나게 하기 위해서는 그 위젯을 어떻게 배치해야 하는 지 명시한 후에야 생성된 위젯이 화면 상에 나타나게 된다.²⁸ 위젯의 위치와 크기는 기하학 관리자를 사용하여 명시할 수 있다. 기하학 관리자에 대한 자세한 내용은 “ 4.2Geometry Manager”에서 자세히 설명할 것이다.

²⁷ 위젯을 생성하는 명령어들에 대해서는 “4.1.1위젯(Widget)”절에서 상세히 설명할 것이다. “2.4Tcl/Tk 맛보기”에서 사용한 button 명령어는 버튼 위젯을 생성하는 Tcl 명령어이다.

²⁸ 이러한 기능을 하는 것이 Geometry Manager 이고 “4.2Geometry Manager”절에서 상세히 설명한다. “2.4Tcl/Tk 맛보기”에서 pack 명령어가 .hello 위젯의 배치 방법을 알려 주고 있다.

4.1.8. Widget Attributes

Widget 의 모양은 Attribute 에 의해 결정된다. Widget 의 Attribute 에는 width, height 등과 같이 모든 Widget 에 공통인 Attribute 도 있고 특정 Widget Class 에만 존재하는 Attribute 도 있다. 각 Widget Class 의 Attribute 들에 대한 자세한 내용은 Online 매뉴얼을 참고하기 바란다.

Widget 의 Attribute 를 설정하는 방법에는 다음과 같은 3 가지 방법이 있다.

- Widget 생성시에 명시하는 방법
- config 명령을 사용하는 방법
- 리소스를 이용하는 방법²⁹

4.1.8.1. 생성시 Widget Attribute 값 정하기

위젯 생성 명령어는 위젯의 Attribute 값을 옵션으로 받아들인다. 버튼 생성시에 다음과 같은 방법으로 버튼의 각종 Attribute 값을 명시할 수 있다.

```
button .b1 -text hello -foreground blue -background red
```

4.1.8.2. config 명령어

config 명령어를 사용하면 이미 생성된 Widget 의 Attribute 를 바꿀 수 있다. config 명령어는 위젯의 Instance Name 다음에 오며 그 사용 형식은 다음과 같다.

```
widgetName config attributes ...
```

```
button .b1 -text hello
pack .b1
.b1 config -text haha -background red
```

4.1.8.3. cget 명령어

cget 명령어는 위젯의 현재 Attribute 값을 리턴한다.

```
widgetName cget attributes ...
```

```
% button .b1 -text hello
.b1
% .b1 cget -text
hello
%
```

²⁹ “4.8Resource”절 참조

4.2. Geometry Manager

위젯이 여러 개 존재하는 복잡한 GUI 를 구성하기 위해서는 위젯을 원하는 위치에 적절히 배치할 수 있는 기능이 필수적이다. Geometry Manager 는 바로 이러한 기능을 담당하고 있다.

Geometry Manager 를 사용하기 위해서는 frame 위젯을 먼저 만들어야 한다. 루트 위젯은 frame 위젯이기 때문에 루트 위젯에서 Geometry Manager 를 사용하기 위해 따로 frame 위젯을 만들 필요는 없다. 그러나 메뉴 bar 나 툴 bar 와 같이 Widget 들의 위치를 독립적으로 관리하기 위해서는 frame widget 을 따로 만들 필요가 있다. frame widget 을 따로 만들고 이 frame widget 의 children 으로 widget 들을 만들면 이들은 frame widget 안에서 따로 위치를 관리할 수 있게 된다.

4.2.1. pack 명령어

```
pack widgetPath ?widgetPath ...? ?option ...?
```

pack 명령어는 자식 위젯들을 부모 위젯의 가장자리를 기준으로 배열해 준다. widgetPath 만 보면 어떤 위젯이 부모이고 자식임을 쉽게 알 수 있다. 아무 옵션도 주지 않았을 때는 부모 위젯의 위쪽부터 아래로 자식 위젯들을 배열해 간다.

4.2.1.1. -side 옵션

아래에 예제는 스크립트 파일로 저장하여 실행하면 최종 결과밖에 볼 수 없다. 따라서 될 수 있으면 wish 를 실행시키고 Interactive Mode 에서 직접 입력하여 각 명령어의 실행 결과가 어떤 형태로 반영되는지 살펴보는 것이 좋다. 그리고 될 수 있으면 wish 를 실행했을 때 생성되는 윈도우의 크기를 크게 늘린 후에 다음의 예제를 입력하면 이해에 도움이 된다.

<pre>% for { set num 1 } { \$num <= 8 } \ { incr num } { button ".but\$num" -text \$num } % pack .but1 -side left % pack .but2 -side right % pack .but3 -side top % pack .but4 -side bottom % pack .but5 -side left</pre>	<pre>% pack .but6 -side right % pack .but7 -side top % pack .but8 -side bottom</pre>
--	--

4.2.1.2. frame 위젯과 pack 명령어 사용 예제

다음과 같이 frame 위젯을 사용하여 보다 복잡한 위젯의 배치를 만들어 낼 수 있다.

```
frame .fr1
frame .fr2
button .fr1.but1 -text .fr1.but1
button .fr1.but2 -text .fr1.but2
button .fr1.but3 -text .fr1.but3
button .fr2.but1 -text .fr2.but1
button .fr2.but2 -text .fr2.but2
pack .fr1.but1 .fr1.but2 .fr1.but3 -side left
pack .fr2.but1 .fr2.but2 -side top
pack .fr1 .fr2 -side top
```



4.2.1.3. Internal Padding & External Padding

internal padding 은 버튼 안쪽에 여백을 주는 것이고, external padding 은 버튼의 바깥쪽에 여백을 주는 것이다.

```
button .one -text one
button .two -text two
pack .one .two -side left
```



```
button .one -text one
button .two -text two
pack .one .two -side left \
    -ipadx 30 -ipady 5
```



```
button .one -text one
button .two -text two
pack .one .two -side left \
    -padx 5 -pady 30
```



4.2.1.4. -fill 옵션

가능한 값에는 x, y, both 가 있다.

<pre>Button .one -text one pack .one</pre> 	<pre>button .one -text one pack .one -fill x</pre> 
--	---

4.2.1.5. Resizing And Expanding

윈도우의 크기가 바뀔 경우 위젯들의 배치를 어떻게 바꿀 것인지를 정하는 옵션이다.

Original	After Resizing
<pre>button .one -text one button .two -text two pack .one .two -side left</pre> 	
<pre>button .one -text one button .two -text two pack .one .two -side left\ -expand true</pre> 	

4.2.1.6. Anchoring

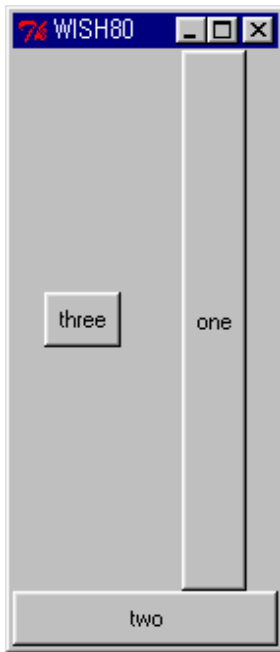
Display Size 는 Widget 을 Display 하기 위해 필요한 최소 크기 이며 Packing Size 는 Widget 을 Packing Rule 에 따라 배치하고 난후 그 Widget 에 할당된 공간을 말한다. 이제까지의 예제에서는 모두 Display Size 와 Paking Size 가 같았으나 다음의 예제에서 .three 의 Display Size 는 Packing Size 보다 작아지게 된다.

anchoring 은 Packing Size 보다 Display Size 가 작을 때 Widget 을 어디에 출력할 것인지를 정하는 옵션이다. 가능한 값들은 center, s (south), n (north), e (east), w (west), se, sw, ne, nw 이다. 지정하지 않았을 때 기본 값은 center 이다.

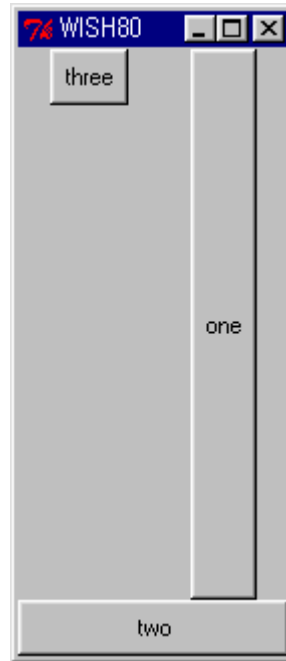
```

Button .one -text one \
    -height 20
button .two -text two -width 20
pack .two -side bottom
button .three -text three
pack .one .three -side right \
    -expand true

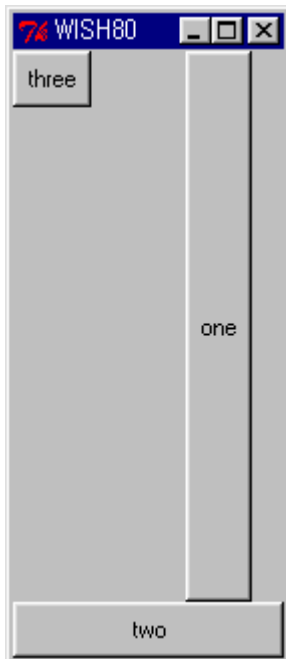
```



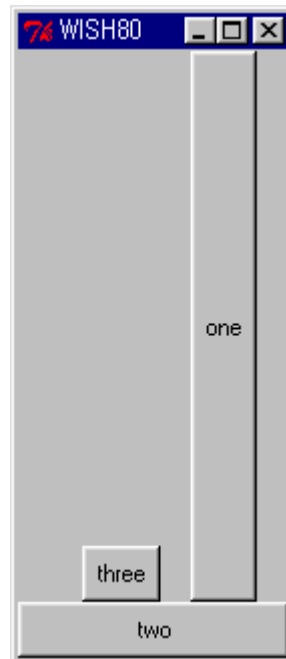
```
pack .three -anchor n
```



```
pack .three -anchor nw
```



```
pack .three -anchor se
```



4.2.2. Grid 명령어

grid 명령어는 위젯을 배치하기 위하여 부모 위젯을 행과 열로 나눈 후 이 행과 열의 상대적인 값으로 위젯의 위치를 정한다. 행과 열의 크기는 부모 위젯의 크기에 따라 동적으로 바뀐다.

grid 명령이 한번 불릴 때마다 행이 하나 생기고 grid 명령어 호출 시 주어진 Widget 들은 열의 수와 순서를 정의 한다.

```
for {set num 1} {$num <= 9 } {incr num } {  
    button .but$num -text $num  
}  
grid .but1 .but2 .but3  
grid .but4 .but5  
grid .but6 .but7 .but8 .but9
```



4.2.2.1. -sticky 옵션

sticky 옵션은 위젯에 할당된 공간이 위젯 크기 보다 클때 위젯을 어떤 쪽으로 늘일지를 지정해 주는 옵션이다. 가능한 값으로는 n (north), w (west), e (east), s (south)가 있고 이 값들을 조합해서 지정할 수 있다. 지정한 값에 따라 위젯이 그 방향으로 늘어나게 된다.

```
button .b1 -text a  
button .b2 -text ab  
button .b3 -text abc  
button .b4 -text abcd  
button .b5 -text abcde  
button .b6 -text abcdef  
grid .b1 .b2  
grid .b3 .b4  
grid .b5 .b6
```



```
button .b1 -text a  
button .b2 -text ab  
button .b3 -text abc  
button .b4 -text abcd  
button .b5 -text abcde  
button .b6 -text abcdef  
grid .b1 .b2 -sticky w  
grid .b3 .b4 -sticky e  
grid .b5 .b6 -sticky ew
```



4.2.2.2. Spanning Rows and Columns

column 옵션과 row 옵션을 사용하여 위젯의 위치를 명시해 줄 수도 있다. 이때 row index 와 column index 는 0 부터 시작한다.

그리고 rowspan 옵션을 사용하여 위젯이 몇 개의 행에 걸쳐서 존재할 지를 명시하게 된다. 마찬가지로 colspan 이라는 옵션을 사용하여 위젯이 몇 개의 열에 걸쳐서 존재할 지를 명시하게 된다.

```
for {set num 1} {$num <= 7 } {incr num } {  
    button .but$num -text $num  
}  
grid .but1 .but2 .but3  
grid .but4  
grid .but5 -row 1 -column 1 -colspan 2 -sticky news  
grid .but6 -row 2 -column 0 -colspan 3 -sticky news  
grid .but7 -row 0 -column 3 -rowspan 3 -sticky news
```



4.2.3. Place 명령어

place 기하학 관리자는 상대적인 좌표와 상대적인 크기로 위젯의 위치와 크기를 명시하기 위해 사용된다.

다음의 예제는 .b1 에 대한 .b2 의 상대적인 위치를 지정하는 예를 보여 주고 있다. .b1 의 가장 왼쪽을 0 으로 보고 .b1 의 오른쪽을 1 로 봤을 때 .b2 가 어디 쯤 위치하는 지를 나타내기 위해 relx 라는 옵션을 사용한다.

마찬가지로 .b1 의 아래 쪽을 0 으로 보고 위쪽을 1 로 봤을 때 .b2 가 어디에 위치해야 할 지를 지정하기 위해 rely 라는 옵션을 사용한다.

anchor 옵션에 의해 지정된 위치에 .b2 의 어떤 부분이 위치해야 할지가 정해 진다. 현재 center 라고 지정되어 있으므로 .b2 의 중앙이 지정한 좌표에 위치하게 된다.

```
button .b1 -width 20 -height 10
button .b2 -text hello
pack .b1
place .b2 -in .b1 -relx 0.2 -rely -0.7 -anchor center
```



다음은 .b1 에 대한 .b2 의 상대적인 크기를 지정하는 예를 보여주고 있다. .b1 의 가로 크기를 1 로 봤을 때 .b2 의 가로 크기가 어느 정도 되어야 할 지를 relwidth 라는 옵션을 사용하여 지정하고 있다. 마찬가지로 .b1 의 높이를 1 로 봤을 때 .b2 의 높이가 어느 정도 되어야 할지를 relheight 라는 옵션을 사용하여 지정하고 있다. x 와 y 는 .b1 에 대한 상대 좌표를 지정하고 있다.

```
button .b1 -width 20 -height 10
button .b2 -text hello
pack .b1
place .b2 -in .b1 -relwidth 0.5 -relheight 0.5 -x 0 -y 0
```



4.2.4. 위젯 감추기

화면에 배치된 위젯을 필요에 따라 감추기 위해 각 기하학 관리자 명령어는 `forget` 이라는 부 명령어를 제공한다. `forget` 명령어는 `destroy` 명령어와 달리 위젯을 파괴하지는 않고 화면에 나타나지 않도록 감출 뿐이다.

다음에 `forget` 명령어의 각 사용예를 보인다.

```
% button .b1 -text haha
% pack .b1
% destroy .b1
% pack .b1
error
% button .b1 -text haha
% pack .b1
% pack forget .b1
% grid .b1
% grid forget .b1
% place .b1 -in .
% place forget .b1
```

`pack` 으로 화면에 배치한 위젯은 반드시 `pack forget` 을 사용해서 감춰야 한다. 다른 기하학 관리자에 대해서도 마찬가지이다. 즉, 반드시 같은 종류의 기하학 관리자 명령어를 사용하여 위젯을 배치하거나 감추어야 한다.

4.3. Binding Commands to Events

Window System 에서 어떤 Event 가 발생하면 원하는 Tcl 명령어를 실행하도록 연관되어 있는 Event 와 Command 의 쌍을 Binding 이라 한다. Window System 에서 발생하는 Event 들에는 Key Press, Mouse Button Press, Mouse Button Release, Mouse Move 등이 있다. 이러한 실제적인 Event 외에도 프로그래머가 의도하는 바에 따라 가상적인 Event 도 정의할 수 있다.

4.3.1. bind 명령어

bind 명령어를 이용하여 binding 을 만든다.

```
bind bindingTag ?event? ?command?
```

bindingTag 는 Widget 의 instance name 이나 class name 이 올 수 있다. Class name 이 오면 그 클래스에 속하는 모든 Widget instance 들에 대한 binding 이 만들어 진다.

```
button .b1 -text one
pack .b1
bind .b1 <Enter> { %W config -text enter }
bind .b1 <Leave> { %W config -text leave }
```

```
button .b1 -text one
button .b2 -text two
pack .b1 .b2
bind Button <Enter> { %W config -text enter }
bind Button <Leave> { %W config -text leave }
```

binding 을 없애기 위해서는 “{ }”를 명령어로 주면 된다.

```
button .b1 -text one
button .b2 -text two
pack .b1 .b2
bind .b1 <Enter> { %W config -text enter }
bind .b2 <Leave> { %W config -text leave }
bind .b1 <Enter> { }
```

command 는 수행하고자 하는 Tcl 명령어들이 올 수 있다. Tcl 명령어는 반드시 “{ }”로 감싸야 한다. 여기서 %W, %x, %y 등과 같이 이벤트의 종류에 따라 이벤트에 관련된 정보를 알려 주는 변수 값을 활용할 수 있다.³⁰ %W 에는 Event 가 일어난 Widget 의 경로명 (or instance name)이 들어 있고 %x, %y 는 마우스 이벤트인 경우 마우스 포인터의 좌표가 들어있다.

```
button .b1 -text one
button .b2 -text two
pack .b1 .b2
bind .b1 <Enter> { %W config -text enter }
bind .b2 <Leave> { %W config -text leave }
```

```
button .b1 -text one
button .b2 -text two
pack .b1 .b2
bind Button <Enter> { %W config -text enter }
bind Button <Leave> { %W config -text leave }
```

command 가 “{+ ”로 시작하면 이미 binding 이 존재하는 경우 그 binding 의 명령에 현재 명령을 덧붙이

³⁰ 이벤트 변수에 관해서는 “4.4.1실제 이벤트” 참조

는 기능을 한다.

```
button .b1 -text one
button .b2 -text two
pack .b1 .b2
bind Button <Enter> { %W config -text enter }
bind Button <Leave> { %W config -text leave }
bind .b1 <Enter> {+ %W config -text hello }
bind .b1 <Leave> {+ %W config -text one }
```

특정한 순서로 발생하는 이벤트들에 대해서도 binding 을 만들 수 있다. 다음 예제를 스크립트로 만들어 실행한 후 생기는 윈도우에 포커스를 맞춘 다음 차례로 ‘a’, ‘b’를 쳐보면 stdout 에 haha 라고 찍히는 것을 볼 수 있을 것이다³¹.

```
#!/usr/local/bin/wish

bind . <Key-a><Key-b> { puts haha }
```

4.3.2. Binding Tags

한 위젯에 대하여 기본적으로 5 가지의 binding 이 존재한다. 위젯은 이 모든 Binding 들 중 하나에 Event 가 발생하면 이에 해당하는 binding 명령을 수행한다. 각각은 다음과 같다.

1. **위젯 고유의 Binding** : 특정한 위젯에 대해서만 정의된 Binding 이다. 이 binding 의 Tag 는 위젯 경로 명과 같다.
2. **위젯 클래스 Binding** : 특정 클래스에 속하는 위젯들에 공통적으로 정의된 Binding 이다. 이 binding 의 Tag 는 위젯 클래스명과 같다.
3. **Top level Widget Binding** : “.” 위젯에 정의된 Binding 이다. Binding Tag 도 “.”이다. 기본적으로 Hot Key 등을 정의하는데 사용된다.
4. **global Binding** : 모든 위젯에 대하여 정의된 Binding 이다. Binding Tag 는 “all”이다. 기본적으로 위젯의 Focus 를 바꾸기 위해 사용된다.
5. **사용자 정의 Binding** : bindtags 명령어로 사용자가 Binding Tag 이름을 새로 만들고 그 Binding Tag 에 속하는 위젯들을 정의할 수 있다. 사용자가 원하는 위젯 그룹들이 특정한 Event 에 대하여 똑같은 반응을 하도록 만들 수 있다.

이렇게 한 위젯에는 기본적으로 5 개의 Binding Tag 가 주어져 있고, 각 Tag 가 대표하고 있는 위젯들의 집합에 Binding 된 Event 에 대하여 반응하게 된다.

사용자 정의 Binding 을 잘 사용하면 특정한 위젯들의 집합을 사용자가 정의하고 이 위젯들은 어떤 Event 에 대하여 똑같은 반응을 보이도록 만들 수 있다. 이 때 사용되는 명령어가 bindtags 라는 명령어이다.

³¹ MS-Windows 나 매킨토시를 사용하는 사용자들은 stdout 의 출력을 보기 위해서는 console 명령을 사용하여 console 을 띄워야 한다.

4.3.2.1. bindtags 명령어

bindtags widget ?tagList?

tagList 를 명시하지 않으면 현재 widget 이 Binding 되어 있는 Binding Tags 의 리스트를 리턴한다. tagList 를 명시하면 widget 의 Binding Tag List 를 명시한 List 로 대체한다.

```
% button .b1
.b1
% bindtags .b1
.b1 Button . all
% bindtags .b1 [list .b1 Button]
% bindtags .b1
.b1 Button
```

이러한 성질을 이용하여 bindtags 를 실제로 사용하는 예를 보이면 다음과 같다.

```
#!/usr/local/wish

# 버튼 10 개를 만든다.
for { set ii 0 } { $ii < 10 } { incr ii } {
    button .b$ii -text $ii
}

# 사용자 정의 Tag 인 MyGroup 을 버튼 5 개에만 등록한다.
for { set ii 0 } { $ii < 5 } { incr ii } {
    set tagList [ bindtags .b$ii ]
    lappend tagList MyGroup
    bindtags .b$ii $tagList
}

# MyGroup 이라는 Tag 에 원하는 이벤트 핸들러를 bind 한다.
bind MyGroup <Enter> { %W config -text enter }
bind MyGroup <Leave> { %W config -text leave }

# 버튼을 화면에 보이게 한다.
for { set ii 0 } { $ii < 10 } { incr ii } {
    pack .b$ii -side left
}
```

위의 예제를 실행하면 MyGroup 에 속한 버튼들은 마우스가 버튼에 들어오고 나감에 따라 반응하지만 다른 버튼들은 전혀 변화가 없음을 알 수 있다.

이와 같이 사용자가 원하는 위젯들을 같은 Tag 에 등록시켜 놓고 어떤 이벤트에 대하여 똑같이 반응하도록 하기 위하여 bindtags 명령어를 주로 사용한다.

4.4. Event

Event 의 종류에는 실제 Event 와 가상 Event 가 있다. 실제 이벤트는 키보드나 마우스 등으로부터 발생되는 실질적인 이벤트이고, 가상 이벤트는 Copy, Paste 등과 같이 프로그래머가 필요에 따라 따로 정의한 이벤트이다.

4.4.1. 실제 이벤트

이벤트는 다음과 같은 형식으로 표현된다.

<modifier-modifier-type-detail>

위에서 가장 중요한 필드는 type 필드이다. Button 이나 Key 과 같이 이벤트의 종류를 명시한다. detail 은 Key-a 나 Button-1 과 같이 특정한 키나 버튼을 명시하는데 사용된다. modifier 는 Control-Key-a 와 기능 키가 더 눌러진 상태를 표시한다. 가능한 이벤트 타입들을 표 13에 정리하였다.

이벤트 타입	의미
Activate	프로그램이 활성화 되었음 (매킨토시 전용)
ButtonPress, Button	마우스 버튼이 눌러 졌음 (Down)
ButtonRelease	마우스 버튼을 눌렀음 (Up)
Circulate	윈도우 쌓인 순서가 바뀌었음
Colormap	색상표가 바뀌었음
Deactivate	프로그램이 비활성화 되었음 (매킨토시 전용)
Destroy	윈도우가 파괴 되었음
Enter	마우스가 윈도우로 들어왔음
Expose	가려졌던 윈도우가 보이게 되었음
FocusIn	축점이 이 윈도우에 맞춰졌음
FocusOut	축점이 다른 윈도우에 맞춰졌음
Gravity	부모 윈도우의 크기가 변경되어 윈도우가 움직였음
KeyPress, Key	키보드의 키가 눌러졌음 (Down)
KeyRelease	키보드의 키가 눌렀음 (Up)
Motion	마우스가 윈도우 안에서 움직이고 있음
Leave	마우스가 윈도우 밖으로 나갔음
Map	윈도우가 열렸음
Property	윈도우의 성질(모양)이 바뀌었음
Reparent	윈도우의 부모가 바뀌었음
Unmap	윈도우가 아이콘으로 바뀌었음
Visibility	윈도우가 보이거나 안보이게 되었음

표 13 Event Types

4.4.1.1. 키보드 이벤트

키보드 이벤트의 경우 KeyPress 와 KeyRelease 를 구별하여 키가 눌러질 때와 놓여질 때 다른 명령을 수행할 수 있도록 하고 있다. 특히 KeyPress 의 경우 축약형을 사용할 수 있다. 다음에 열거된 것들은 모두 같은 이벤트를 가리키고 있다.

```
<KeyPress-a>
<Key-a>
<a>
a
```

여기서 주의할 점은 숫자 키보드의 경우 다음과 같이 Key 를 명시해 주어야 한다. 그렇지 않으면 마우스 버튼 이벤트와 구별할 수 없게 된다.

<Key-1>

키보드에 대한 detail 은 keysym 이라고 한다. 특별한 키들을 위하여 정의된 keysym 에는 다음과 같은 것들이 있다.

Return, Escape, BackSpace, Tab, Up, Down, Left, Right, comma, period, dollar, asciicircum, numbersign, exclam, space

위에 열거된 키들은 다음과 같은 형태로 사용된다.

<Key-Return>

키의 조합은 다음과 같이 Modifier 를 사용하여 나타 낸다. 첫 라인은 Shift 키와 Control 키를 누른 상태에서 Return 키를 누른 경우를 나타낸다. 그 다음 라인은 Shift 키를 누른 상태에서 a 키를 누른 경우를 나타낸다.

<Shift-Control-Key-Return>

<Shift-Key-a>

4.4.1.2. 마우스 이벤트

마우스 이벤트 역시 ButtonPress 와 ButtonRelease 를 구별하여 다른 이벤트로 처리한다. ButtonPress 의 경우에는 다음과 같이 간단히 쓸 수 있다. 다음의 3 가지는 모두 같은 뜻이다.

<ButtonPress-1>

<Button-1>

<1>

Button1 이 눌러진 상태에서 마우스가 움직이는 경우는 다음과 같이 나타낼 수 있다.

<B1-Motion>

Double Click 은 다음과 같이 나타낸다.

<Double-1>

Modifier 를 사용하여 키와 마우스 버튼의 조합을 다음과 같이 나타낼 수 있다.

<Shift-Button-1>

<Shift-Control-Button-1>

4.4.1.3. Modifiers

Modifier	의미
Configure	위젯의 configuration 을 바꿀 때
Control	Control Key
Shift	Shift Key
Lock	CapsLock Key
Command	매킨토시의 Command Key
Meta, M	Meta Key : M1 에서 M5 는 모두 Meta_L, Meta_R 에 매핑될 수 있다. ³²
Alt	Alt Key : Alt_L, Alt_R 에 매핑 될 수 있다.
Mod1, M1	1 st Modifier
Mod2, M2, Alt	2 nd Modifier
Mod3, M3	3 rd Modifier
Mod4, M4	4 th Modifier
Mod5, M5	5 th Modifier
Button1, B1	1 st Mouse Button
Button2, B2	2 nd Mouse Button
Button3, B3	3 rd Mouse Button
Button4, B4	4 th Mouse Button
Button5, B5	5 th Mouse Button
Double	마우스 버튼을 짧게 두 번 누른 이벤트
Triple	마우스 버튼을 짧게 세 번 누른 이벤트
Any	어떠한 Modifier 의 조합도 적용됨

표 14 Event Modifiers

4.4.2. 가상이벤트

가상 이벤트는 실제로 존재하는 이벤트가 아니고, 프로그래머가 정의한 이벤트를 말한다. 가상 이벤트는 다음과 같이 표시하고 사용한다.

<<Event>>

```
bind <<Cut>> { ... }
```

가상 이벤트는 event add 명령어를 이용하여 만들어 질 수 있다.

```
event add virtualEvent realEvent ?realEvent ...?
event delete virtualEvent
event info ?virtualEvent?
event generate win event ?option value ... ?
```

```
event add <<Cut>> <Control-Key-x><Key-F20>
```

event generate 를 이용하여 특정 윈도우에 이벤트를 임의로 발생시킬 수 있다. 이 때 option 을 이용하여 여러 가지 Field Value 를 정의할 수 있다. “표 15 이벤트 변수”에 event generate 에 사용할 수 있는 option 들을 정리하였다.

³² UNIX 에서는 xmodmap 이라는 프로그램으로 Modifier 가 어떤 Key 에 매핑되어 있는지 알아 보거나 Mapping 을 바꿀 수 있다.

4.4.3. 이벤트 정보

binding command 에서 이벤트 변수값을 참조하면 이벤트에 대한 여러 가지 정보를 알 수 있다. 이 값들은 자동으로 세팅될 수도 있고 event generate 명령을 이용하여 발생시킨 이벤트의 경우에는 option 을 이용하여 값을 정의할 수도 있다.

이벤트 변수	event generate option	의미	해당 이벤트
%%		%기호를 사용하기 위해서	All
%#	-serial num	이벤트의 Serial Number	All
%a	-above win	above window	Configure
%b	-button num	마우스 버튼 번호	ButtonPress, ButtonRelease
%c	-count num	count Field	Expose, Map
%d	-detail value	가능한 값들은 다음과 같다. NotifyAncestor NotifyNonlinearVirtual NotifyDetailNone NotifyPointer NotifyInferior NotifyPointerRoot NotifyNonlinear NotifyVirtual	Enter, Leave, FocusIn, FocusOut
%f	-focus boolean	초점이 맞추어져 있으면 1 아니면 0	Enter, Leave
%h	-height num	height	Configure, Expose
%k	-keycode num	key code	KeyPress, KeyRelease
%m	-mode value	가능한 값들은 다음과 같다. NotifyNormal NotifyGrab NotifyUngrab NotifyWhileGrabbed	Enter, Leave, FocusIn, FocusOut
%o	-override boolean	override field 가 참이면 1 아니면 0	Map, Reparent, Configure
%p	-place value	가능한 값들 PlaceOnTop PlaceOnBottom	Circulate
%s	-state value	state field, decimal string	ButtonPress, ButtonRelease Enter, Leave, Motion KeyPress, KeyRelease
		가능한 값들 VisibilityUnobscured VisibilityPartiallyObscured VisibilityFullyObscured	Visibility
%t	-time num	time	All
%w	-width num	width	Configure, Expose
%x	-x pixel	위젯 내에서 X 좌표	Mouse 관련 Events
%y	-y pixel	위젯 내에서 Y 좌표	Mouse 관련 Events
%A		프린트할 수 있는 글자 프린트할 수 없다면 아무 값도 없음	KeyPress, KeyRelease
%B	-borderwidth num	border width	Configure
%E	-sendevent bool	send_event field	All
%K	-keysym symbol	keysym value	KeyPress, KeyRelease
%N		keysym as decimal number	KeyPress, KeyRelease

%R	-root win	root window ID	All
%S	-subwindow win	subwindow ID	All
%T		type field	All
%W		위젯 instance name	All
%X	-rootx pixel	루트 위젯에 대한 X 좌표	ButtonPress ButtonRelease KeyPress, KeyRelease, Motion
%Y	-rooty pixel	루트 위젯에 대한 Y 좌표	ButtonPress ButtonRelease KeyPress, KeyRelease, Motion

표 15 이벤트 변수

4.5. Widgets

Tk 에서 제공하는 기본적인 위젯들의 모양과 간단한 사용법들을 알아 본다. 각각의 위젯들은 위젯의 특성에 맞게 여러 가지 operation 을 제공한다. 예를 들어 entry 위젯의 경우 입력된 문자열을 리턴해 주는 operation, 선택된 문자열을 리턴해 주는 operation 등을 제공한다. 이와 같은 operation 들을 사용하기 위한 일반적인 문법은 다음과 같다.

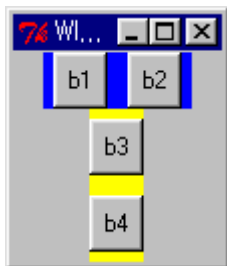
```
pathName operation [arguments ... ]
```

각 위젯별로 operation 에 관한 자세한 내용은 online 매뉴얼에 잘 나와 있으므로 본 교재에서는 생략하기로 하고, 필요한 경우 중요한 operation 에 대하여 잠깐 언급하겠다.

4.5.1. Frame 위젯

프레임 위젯은 위젯들의 위치를 따로 관리하기 위한 위젯이다. 한 프레임 내의 위젯들은 프레임 밖의 위젯과 상관없이 위치를 지정할 수 있다. 루트 위젯은 Frame 위젯이다.

```
frame .f1 -background blue
button .f1.b1 -text b1
button .f1.b2 -text b2
frame .f2 -background yellow
button .f2.b1 -text b3
button .f2.b2 -text b4
pack .f1.b1 .f1.b2 -side left -padx 5
pack .f2.b1 .f2.b2 -pady 5
pack .f1 .f2
```



4.5.2. Button 위젯

글자로 된 버튼 뿐 아니라 bitmap 파일을 이용하거나 GIF, PPM, PGM 형식으로 되어 있는 그림 파일을 이용해서도 버튼을 만들 수 있다. 이러한 기능은 모든 버튼에 적용 가능하며 label 의 경우도 마찬가지이다.

```
button .b -text "push me" -command { puts "Hello" }
pack .b
```



```
button .b -bitmap question
pack .b
```



```
image create photo myimage -file d:/wonnr/tcl/board.gif
button .b -image myimage
pack .b
```



4.5.3. Label 위젯

Label 위젯은 단순히 텍스트를 디스플레이하는 역할을 하는 위젯이다.

```
label .la -text "Nara" -background yellow
pack .la
```



변수를 써서 디스플레이되는 텍스트를 동적으로 바꿀 수도 있다.

```
set message "Hello, World"
label .la -textvariable message -background yellow
pack .la
set message "Nara Won"
```

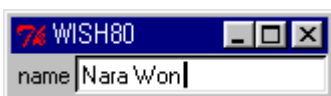
Button 위젯과 마찬가지로 방법으로 비트맵이나 Gif 파일을 보여주는 Label 을 만들 수 있다.

4.5.4. Entry 위젯

한 줄의 텍스트를 사용자로부터 입력받을 때 사용할 수 있는 위젯이다. 간단한 편집 기능과 스크롤, 선택기능을 제공한다. 입력된 문자열을 특정 변수에 자동으로 저장하는 기능도 제공 한다. 입력된 문자열을 다루기 위한 여러 가지 유용한 operation 을 제공하고 있다.

Label 위젯에서와 마찬가지로 textvariable 이라는 옵션을 사용하여 사용자가 입력한 문자열을 임의의 변수에 자동으로 저장되도록 할 수 있다.

```
label .title -text name
entry .titleIn -textvariable inputText -relief sunken
pack .title .titleIn -side left
bind .titleIn <Return> { puts $inputText }
```



연습문제

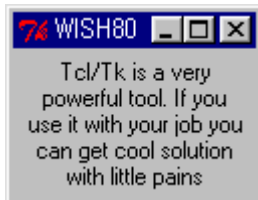
- 숫자 맞추기 게임의 사용자 인터페이스를 다음의 예를 참조하여 GUI로 구현 하시오.



4.5.5. Message 위젯

여러 줄에 걸친 긴 메시지를 화면에 보여줄 때 사용한다. 정의에 따라 줄을 맞추는 기능을 제공한다. 메시지 위젯의 특징은 긴 문자열의 정렬 형태를 justify 라는 옵션을 사용하여 지정할 수 있다는 것이다.

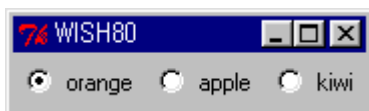
```
set message "Hello, World"
message .msg -justify center -text "Tcl/Tk is a very powerful tool. If you
use it with your job you can get cool solution with little pains"
pack .msg
```



4.5.5.1. Radio Button 위젯

라디오 버튼은 한 순간에 하나의 버튼만 선택할 수 있다. 따라서 라디오 버튼은 하나의 변수를 모든 버튼들이 공유한다. 공유된 변수에는 현재 선택된 버튼의 value 가 저장 된다.

```
set choice 1
radiobutton .b1 -text orange -value 1 -variable choice \
    -command { puts orange }
radiobutton .b2 -text apple -value 2 -variable choice \
    -command { puts apple }
radiobutton .b3 -text kiwi -value 3 -variable choice \
    -command { puts kiwi }
pack .b1 .b2 .b3 -side left
```



4.5.5.2. Check Button 위젯

체크 버튼은 한 순간에 여러 개의 버튼을 선택할 수 있다. 따라서 각 버튼은 각각 따로 자신의 상태를 저장할 변수가 필요하다. 각 변수에는 선택 상태에 따라 선택되었으면 1 이 선택되지 않았으면 0 이 저장된다.

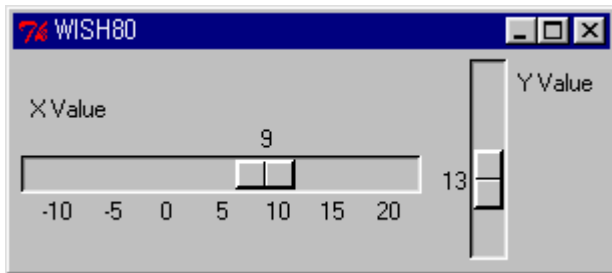
```
checkboxbutton .b1 -text orange -variable stateOrange \
    -command { puts orange }
checkboxbutton .b2 -text apple -variable stateApple \
    -command { puts apple }
checkboxbutton .b3 -text kiwi -variable stateKiwi \
    -command { puts kiwi }
pack .b1 .b2 .b3 -side left
```



4.5.6. Scale 위젯

일정한 범위 내의 연속적인 값을 선택할 수 있도록 해준다. 스케일 위치를 `variable` 이라는 옵션을 사용하여 임의의 변수에 자동으로 저장되도록 할 수 있다.

```
scale .s1 -from -10 -to 20 -length 200 -variable x \  
    -orient horizontal -label "X Value" -tickinterval 5 \  
    -showvalue true  
scale .s2 -from 0 -to 20 -length 100 -variable y \  
    -orient vertical -label "Y Value"  
pack .s1 .s2 -side left
```



4.5.7. listbox 위젯

여러 개의 항목들 중에서 사용자가 원하는 항목을 선택할 수 있도록 해준다. 사용자가 선택한 항목의 index 를 쉽게 알아낼 수 있는 여러가지 operation 들을 제공한다.

```
listbox .lbox  
.lbox insert end apple  
.lbox insert end orange  
.lbox insert end kiwi  
.lbox insert end peach  
pack .lbox
```



현재 선택된 항목의 index 를 알아내기 위해서는 다음과 같이 하면 된다. 첫번째 항목의 인덱스는 0 이다.

```
#!/usr/local/bin/wish  
proc PrintItem { } {  
    puts [ .lbox curselection ]  
}  
listbox .lbox  
.lbox insert end apple  
.lbox insert end kiwi  
pack .lbox  
bind .lbox <Button-1> { PrintItem }
```


4.5.8. scrollbar 위젯

윈도우 안에 원하는 내용을 다 표시할 수 없을 경우 내용을 스크롤시켜 볼 수 있게 해준다.

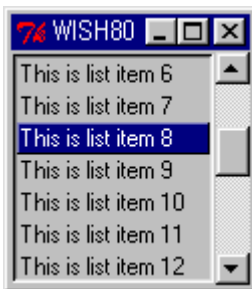
```
scrollbar .s1 -orient horizontal
scrollbar .s2 -orient vertical
pack .s1 .s2
```



스크롤바는 단독으로 사용되기 보다는 스크롤이 가능한 위젯과 같이 사용된다. 스크롤이 가능한 위젯들은 `-xscrollcommand`, `-yscrollcommand` 옵션을 제공한다. 스크롤이 가능한 위젯은 사용자가 스크롤바를 움직였을 때 `-xscrollcommand`, `-yscrollcommand` 에 정의된 명령어 다음에 스크롤바의 상대적인 위치와 View 의 상대적인 크기를 0 부터 1 사이의 실수로 덧붙여서 그 명령어를 실행한다.

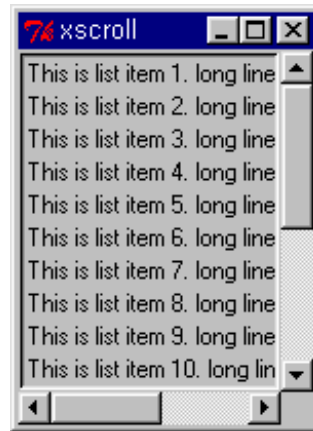
그리고 스크롤이 가능한 위젯들은 스크롤이 되었을 때 자신의 View 를 업데이트하기 위한 명령을 제공하는데 보통 `xview` 와 `yview` 이다. 이것을 각 스크롤바에 `-command` 를 써서 명시해 주어야 한다. 스크롤바에 움직임이 있을 경우 스크롤바는 `-command` 에 정의된 명령을 실행한다.

```
scrollbar .sy -orient vertical -command { .lbox yview }
listbox .lbox -yscrollcommand { .sy set }
for {set num 1 } {$num <= 20} {incr num} {
    .lbox insert end "This is list item $num"
}
grid .lbox .sy -sticky news
```



연습문제

다음과 같이 리스트 박스에 가로 스크롤 바를 붙여 보시오.



4.5.9. toplevel

독립된 윈도우를 하나 더 만든다.

`toplevel .kk`

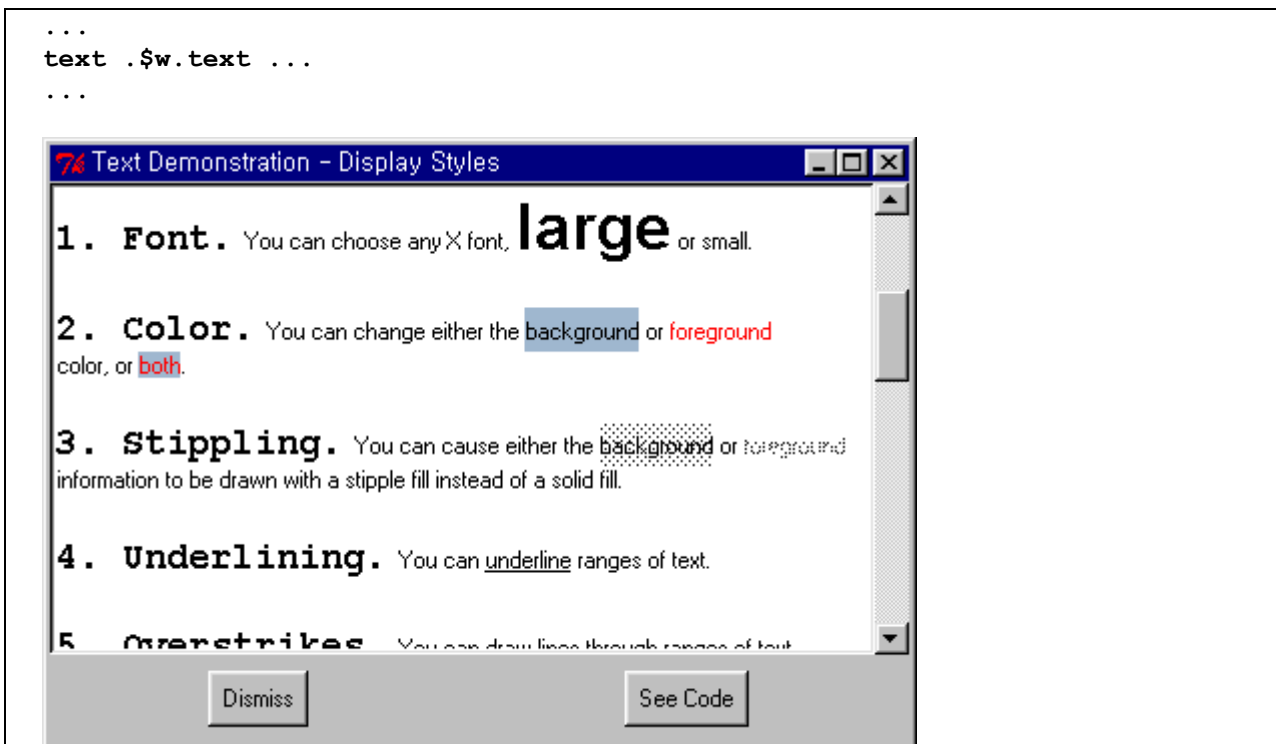


4.5.10. Text 위젯

text 위젯은 여러 줄에 걸친 문자열을 입력 받기 위해 사용할 수 있다. text 위젯이 제공하는 operation 을 잘 활용하면 Note Pad 같은 간단한 에디터 프로그램을 작성할 수도 있다.

Text 위젯은 여러 개의 Tag 를 정의해 두고 서식이 다른 단락을 관리할 수 있도록 해주는 기능을 제공한다. 그리고 줄간격, 줄맞춤 등을 제어할 수 있을 뿐 아니라 문자열 검색 기능도 제공한다. 또한 Text 위젯 내에 다른 위젯도 디스플레이할 수 있다. 그리고 Tag 에 binding 을 만들어 HyperText 를 만들 수 있는 기능도 제공한다.

여기에 관한 자세한 내용은 Online 매뉴얼과 Tcl/Tk 설치 프로그램과 함께 제공되는 데모 프로그램을 참고하기 바란다. 여기에서는 그 중에서 Text 위젯에서 다양한 출력형태를 표시하는 데모 프로그램을 예시한다.



연 습 문 제

1. 구구단 출력이 다음과 같이 되도록 프로그램을 작성하시오.
 - * text 위젯의 사용법은 OnLine Manual 을 활용하여 text 위젯 사용법을 알아내도록 하시오.
 - * 구구단 출력 후 맨 마지막 부분이 사용자에게 보이도록 하시오.
 - * 사용자가 구구단의 내용을 수정할 수 없도록 하시오.



```
nara_99G
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
-----
8 x 2 = 16
8 x 3 = 24
8 x 4 = 32
8 x 5 = 40
8 x 6 = 48
8 x 7 = 56
8 x 8 = 64
8 x 9 = 72
-----
9 x 2 = 18
9 x 3 = 27
9 x 4 = 36
9 x 5 = 45
9 x 6 = 54
9 x 7 = 63
9 x 8 = 72
9 x 9 = 81
-----
```

4.5.11. canvas 위젯

canvas 위젯은 선, 사각형, 원 등을 그리거나 Bitmap 이미지를 디스플레이하는데 사용된다. 그리고 canvas 위젯에 그려진 모든 Object 는 마우스로 쉽게 선택할 수 있으며 마우스의 움직임에 따라 쉽게 위치나 모양을 조정할 수 있다. 뿐만 아니라 canvas 위젯에 그려진 내용을 Postscript 파일로 출력시켜주는 기능도 제공한다.

canvas 위젯의 원점은 왼쪽 위 모서리이다. 오른쪽 방향이 양의 x 축 방향이고 아래쪽 방향이 양의 y 축 방향이다.

canvas 위젯의 보다 실제적인 사용 예는 “6예제 프로그램”의 서브루틴들을 참조하기 바란다.

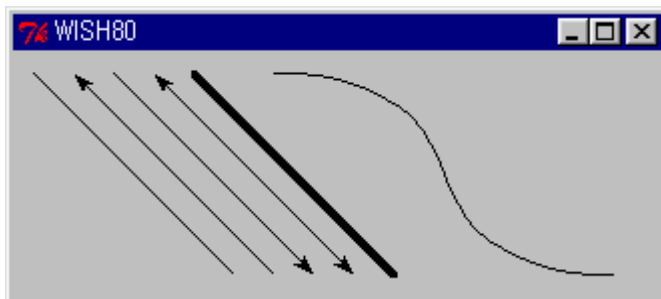
4.5.11.1. canvas items

캔버스 위젯에 도형을 그리기 위해서는 다음과 같은 명령어 형식을 사용한다. 여기서 item 은 canvas 위젯에 그릴 수 있는 도형의 종류이다. 이 명령의 리턴 값은 생성된 도형에 대한 핸들러이다.

```
pathName create item x y ?x y ...? ?option value ...?
```

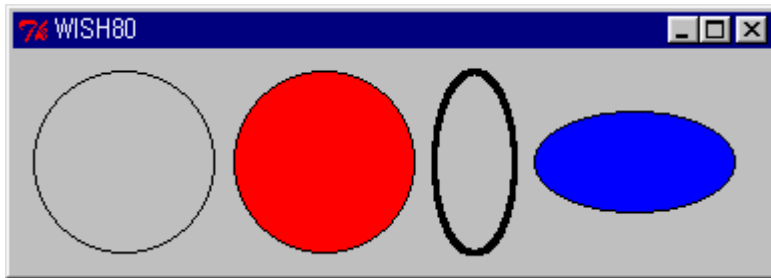
다음은 직선을 그리는 예제이다. 베지어 곡선을 그리는 예제가 마지막에 나와 있다. 베지어 곡선을 그리기 위해서는 4 개의 점에 대한 좌표를 명시해야 한다.

```
canvas .can
pack .can
.can create line 10 10 110 110
.can create line 30 10 130 110 -arrow first
.can create line 50 10 150 110 -arrow last
.can create line 70 10 170 110 -arrow both
.can create line 90 10 190 110 -width 3
.can create line 130 10 200 10 230 110 300 110 -smooth true
```



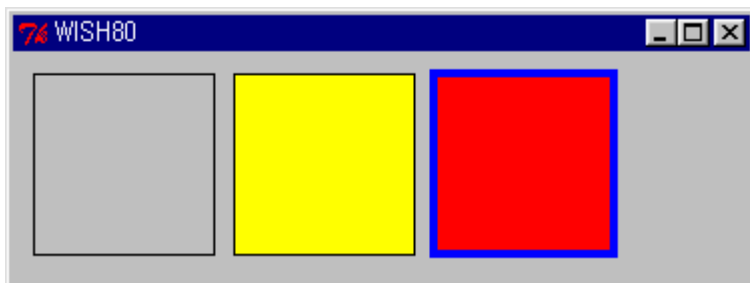
타원을 그리기 위해서는 oval 을 사용하면 된다. 타원을 그리기 위해서는 2 개 점의 좌표를 명시해 주어야 한다. 처음 숫자가 x 좌표이고 그 다음 숫자가 y 좌표이다. 명시된 2 개의 점을 대각선으로 하는 직사각형의 내접 타원을 그려 준다.

```
canvas .can
pack .can
.can create oval 10 10 100 100
.can create oval 110 10 200 100 -fill red
.can create oval 210 10 250 100 -width 3
.can create oval 260 30 360 80 -fill blue
```



사각형도 원과 마찬가지로 2 개 점의 좌표를 명시하면 된다. 2 개의 점을 대각선으로 하는 직사각형이 그려진다.

```
canvas .can
pack .can
.can create rectangle 10 10 100 100
.can create rectangle 110 10 200 100 -fill yellow
.can create rectangle 210 10 300 100 -fill red \
    -outline blue -width 4
```



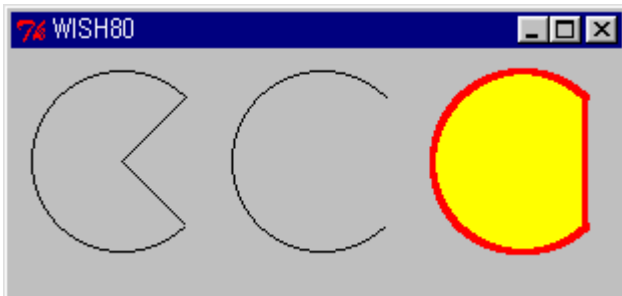
다각형을 그리기 위해서는 그리고자 하는 다각형의 꼭지점 수만큼 점의 좌표를 명시하면 된다. 명시된 각 점을 꼭지점으로 하는 다각형이 그려진다.

```
canvas .can
pack .can
.can create poly 10 30 10 50 \
    30 70 50 70 70 50 70 30 50 10 30 10 \
    -outline white -fill red -width 5
```



부분원을 그리기 위해서는 다음과 같이 2 개 점의 좌표와 시작각 그리고 부분원의 범위 각도를 명시해주면 된다. 옵션에 따라 파이모양, 활 모양, 원호 모양의 부분 원이 그려 진다.

```
canvas .can
pack .can
.can create arc 10 10 100 100 \
    -start 45 -extent 270 -style pieslice
.can create arc 110 10 200 100 -start 45 -extent 270 \
    -style arc
.can create arc 210 10 300 100 -start 45 -extent 270 \
    -style chord -fill yellow -width 3 -outline red
```



OS 상에 미리 정의된 비트맵을 canvas 위젯위에 그릴 수 있다. 각 OS 마다 정의된 비트맵이 다르므로 사용에 주의를 기울여야 한다.

```
canvas .can
pack .can
.can create bitmap 10 10 -bitmap error -anchor nw
.can create bitmap 40 10 -bitmap hourglass -anchor nw
.can create bitmap 80 10 -bitmap question -anchor nw
.can create bitmap 120 10 -bitmap warning -anchor nw
.can create bitmap 160 10 -bitmap info -anchor nw
```



Gif 파일을 canvas 위에 표시하기 위해서는 다음과 같이 하면 된다.

```
image create photo logo -file d:/wonnr/tcl/logoMed.gif
canvas .can
.can create image 10 10 -image logo -anchor nw
pack .can
```



4.5.11.2. Item 지우기

canvas 위젯에 그려진 Item 을 지우기 위해서는 다음과 같이 하면 된다.

```
canvas .can
pack .can
set lineId [ .can create line 10 10 100 100 ]
.can delete $lineId
```

canvas 위젯에 그려진 모든 Item 을 한꺼번에 모두 지우는 방법은 다음과 같다.

```
canvas .can
pack .can
.can create line 10 10 100 100
.can create oval 110 10 210 100
.can create rectangle 310 10 410 100
foreach item [ .can find all ] {
    .can delete $item
}
```

4.5.11.3. item 움직이기

canvas 위젯에 그려진 item 을 움직이는 방법은 다음과 같다.

```
canvas .can
pack .can
set ovalId [ .can create oval 110 10 210 100 ]
.can move $ovalId 100 10
```

4.5.11.4. canvas 내용 프린트하기

canvas 위젯에 그려진 내용은 postscript 파일로 바로 만들 수 있다. 단, 보이는 부분만 postscript 로 출력된다는 점에 유의해야 한다. 보다 자세한 내용은 OnLine 매뉴얼을 참조하기 바란다.

```
canvas .can
pack .can
create oval 110 10 210 100
.can postscript -file "my.ps"
```

4.5.11.5. 확대 및 축소

canvas 위젯에 그려진 item 을 2 배 확대하기 위해서는 다음과 같이 하면 된다.

```
canvas .can
pack .can
set ovalId [ .can create oval 110 10 210 100 ]
.can scale $ovalId 0 0 2 2
```

canvas 위젯에 그려진 모든 item 을 반으로 축소하기 위해서는 다음과 같이 하면 된다.

```
canvas .can
pack .can
.can create oval 110 10 210 100
.can create rectangle 220 10 320 100
foreach item [ .can find all ] {
    .can scale $item 0 0 0.5 0.5
}
```

4.5.11.6. item 좌표 찾기

canvas 위젯에 그려진 item 의 좌표를 얻기 위해서는 다음과 같이 하면 된다.

```
canvas .can
pack .can
set ovalId [ .can create oval 110 10 210 100 ]
set coordinate [ .can coord $ovalId ]
puts $coordinate
```

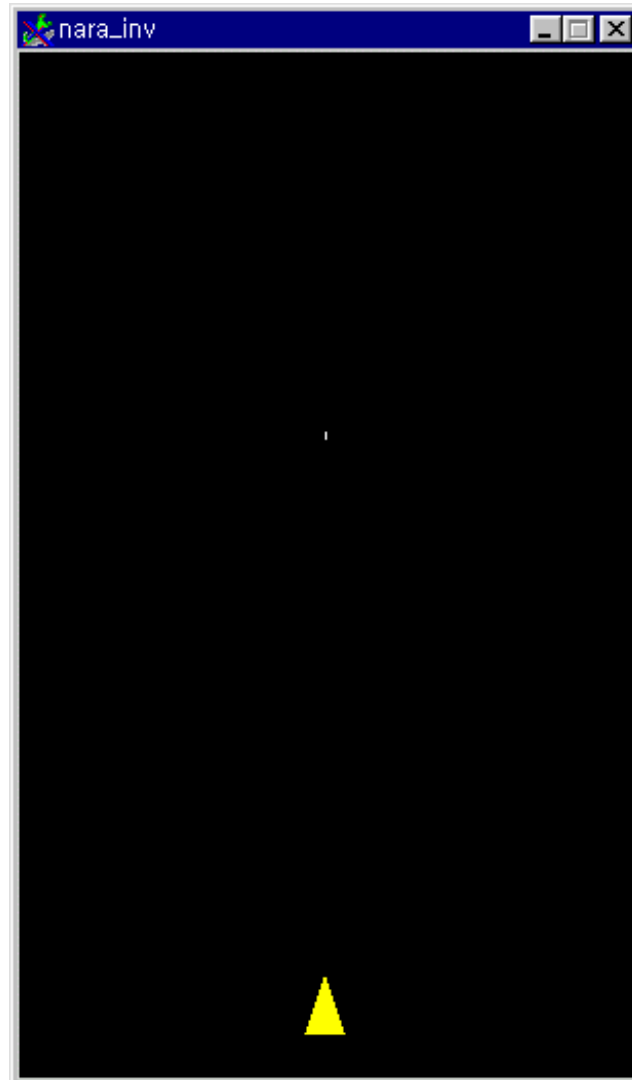
4.5.11.7. item 찾기

특정 좌표의 위치에 가장 가까운 item 을 찾기 위해서는 다음과 같이 하면 된다.

```
canvas .can
pack .can
set ovalItem [ .can create oval 110 10 210 100 ]
set foundItem [ .can find closest 120 50 ]
if { $ovalItem == $foundItem } {
    puts haha
}
```

연 습 문 제

1. 다음과 같은 화면이 나타나고 화살표 키를 누르면 세모가 좌우로 움직이고 스페이스 바를 누르면 포탄이 발사되는 프로그램을 작성하시오.



4.5.11.8. Selection & Tag Binding

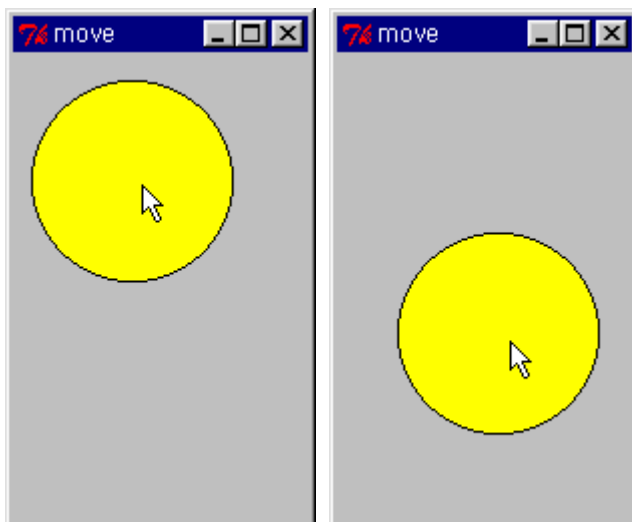
canvas 위젯 내의 특정 Item 그룹에 대하여 프로그래머가 임의의 Tag 를 할당할 수 있다. item 생성시에 tag 옵션을 사용하여 Tag 의 이름을 명시하면 된다. 이 Tag 는 canvas 위젯이 제공하는 bind 라는 부명령어에서 사용된다. 이를 이용하면 canvas 내의 특정 item 그룹은 어떤 이벤트에 대하여 똑같은 방식으로 반응하도록 만들 수 있다.

canvas 위젯내에서 item 을 생성시키면서 정의한 Tag 는 canvas 위젯 내에서만 의미가 있음에 유의해야 한다. 따라서 일반적인 위젯용 bind 명령에는 이 Tag 를 사용할 수 없다.

다음 프로그램을 수행하면 노란색 원이 그려지고 이 원을 마우스로 선택하여 마우스를 움직이면 원의 위치가 바뀐다. 이 예제에서는 원 밖에 생성시키지 않았지만 다른 item 을 생성시키면서 MyMovable 이라는 tag 를 지정하면 그 item 도 역시 마우스를 이용하여 위치를 변경할 수 있게 된다. MyMovable 은 프로그래머가 임의로 정한 사용자 정의 Tag 이다.

```
proc MoveStart { canvasWidget x y } {
    global selectedItem prevX prevY
    set prevX $x
    set prevY $y
    set selectedItem [ $canvasWidget find closest $x $y ]
}
proc MoveItem { canvasWidget x y } {
    global selectedItem prevX prevY
    set dx [ expr $x - $prevX ]
    set dy [ expr $y - $prevY ]
    $canvasWidget move $selectedItem $dx $dy
    set prevX $x
    set prevY $y
}

canvas .can
.can bind MyMovable <Button-1> { MoveStart %W %x %y }
.can bind MyMovable <B1-Motion> { MoveItem %W %x %y }
.can create oval 50 50 150 150 -fill yellow -tag MyMovable
pack .can
```



4.5.12. update 명령어

update 명령어는 프로그램이 widget 에 가한 변화를 실제로 눈에 보이게 하거나 사용자가 widget 에 입력한 내용을 그 widget 과 연관된 변수의 값에 반영하도록 명시하는 함수 이다.

프로그램내에서 매우 오랜 시간이 걸리는 연산을 수행하는 동안 GUI 는 거의 반응하지 않는 상태로 머물러 있다가, 연산이 모두 끝난 후에야 프로그램에서 변경한 내용이 실제로 GUI 에 반영된다. 더욱 중요한 사실은 연산이 진행되는 동안 GUI 가 제대로 반응하지 않음으로 인하여 사용자로 하여금 프로그램이 오동작을 하고 있다는 착각을 불러일으키게 된다는 점이다. 다음의 프로그램을 수행하여 for 루프를 돌고 있는 동안 GUI 가 어떻게 되는지 관찰해 보자.

```
#!/usr/local/bin/wish
proc Start { } {
    global talk
    set sum 0
    for {set num 1} { $num < 1000000 } {incr num} {
        set talk "$sum"
        incr sum $num
    }
}
button .s -text Start -command { Start }
label .t -textvariable talk -bg white
pack .s
pack .t
```

위의 프로그램을 실행시킨 후 start 버튼을 누르고 나면 프로그램의 GUI 는 아무런 반응도 하지 않는다. 다른 윈도우로 GUI 를 가렸다 치우면 GUI 가 지워진 채 다시 그려지지도 않는다. 프로그램 수행시 이러한 현상이 나타나면 프로그램 사용자들 중 대부분은 프로그램이 오동작하고 있는 것으로 판단하고 프로그램 수행을 정지시키려 할 것이다.

이러한 문제를 해결해 주는 명령어가 바로 update 이다. 위의 프로그램을 아래와 같이 바꾸면 위와 같은 문제점을 해결할 수 있다.

```
#!/usr/local/bin/wish
proc Start { } {
    global talk
    set sum 0
    for {set num 1} { $num < 1000000 } {incr num} {
        set talk "$sum"
        update
        incr sum $num
    }
}
button .s -text Start -command { Start }
label .t -textvariable talk -bg white
pack .s
pack .t
```

4.6. Dialogs

사용자에게 파일을 선택하거나 색깔을 선택하게 하는 등의 대화상자는 여러 프로그램에서 공통적으로 많이 사용되는 대화 상자이다. Tk 에서는 이처럼 공통적으로 사용할 수 있는 대화상자를 미리 만들어 제공하고 있다.

4.6.1. Message Box 다이얼로그

원하는 간단한 메시지를 출력하는 대화 상자를 만들 수 있다. `-type option` 을 이용하여 `ok`, `okcancel`, `retrycancel`, `yesno`, `yesnocancel`, `abortretrycancel` 등의 다양한 버튼을 갖는 대화상자를 만들 수 있다. 사용자가 선택한 파일의 Full Path Name 을 리턴해 준다. 또한 `-icon` 을 이용하여 사용되는 icon 의 모양도 바꿀 수 있다.

```
% set choice [ tk_messageBox -message "This is Message Box" ]
```

```
ok
```

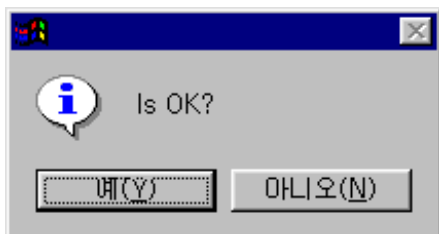
```
% set choice
```

```
ok
```



```
% set choice [ tk_messageBox -message "Is OK?" -type yesno ]
```

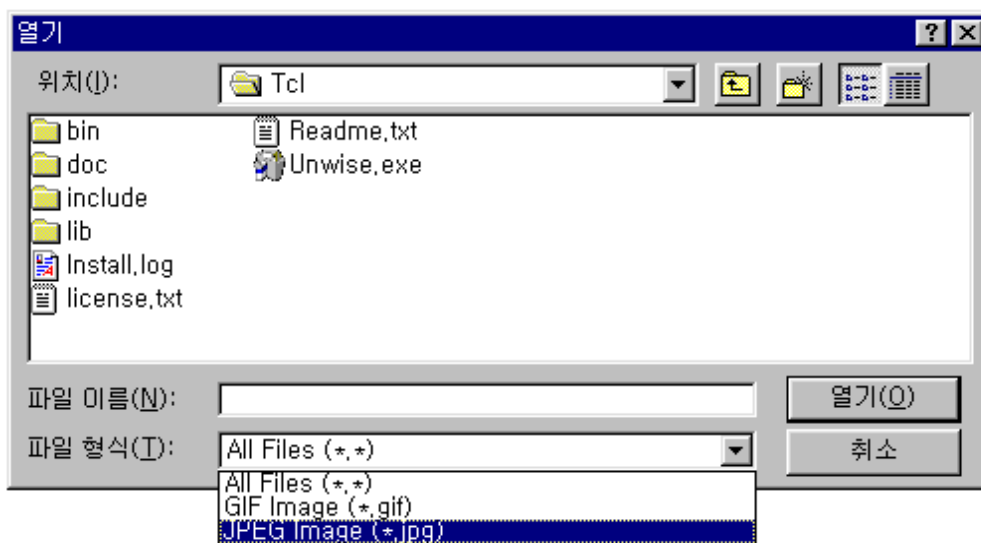
```
yes
```



4.6.2. File 선택 다이얼로그

Tk에서는 파일 선택을 위해 `tk_getOpenFile`, `tk_getSaveFile` 을 제공하고 있다. 초기 디렉토리와 디폴트 파일 이름도 정할 수 있다. `open` 다이얼로그는 반드시 존재하는 파일만을 선택할 수 있다. `save` 다이얼로그는 존재하지 않는 파일이름을 선택할 수 있으며, 이미 존재하는 파일 이름을 선택하는 경우 덮어쓸 것인지를 사용자에게 물어 본다.

```
set typelist {  
    {"All Files" {"*"} }  
    {"GIF Image" {"*.gif"}}  
    {"JPEG Image" {"*.jpg"}}  
}  
set fileName [tk_getOpenFile -filetypes $typelist]
```



프로그램을 작성하다 보면 디렉토리를 고르는 다이얼로그가 필요한 경우가 있다. 그러나 Tk에서 기본적으로 제공하는 다이얼로그에는 디렉토리를 고르기 위한 다이얼로그가 없다. 하지만 자주 사용되는 다이얼로그이다 보니

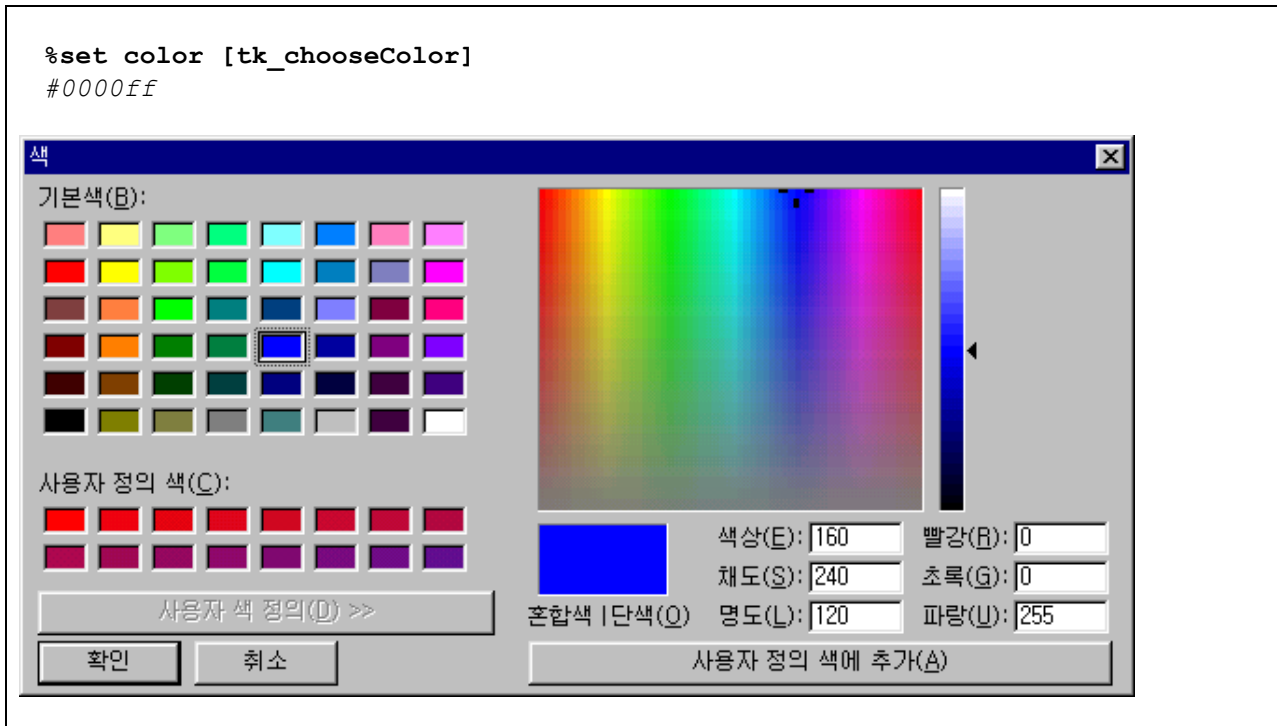
<http://www.scriptics.com>

에서 디렉토리 선택 다이얼로그를 Tcl로 구현한 소스를 배포하고 있다. 이를 다운 받아 사용하면 된다.

4.6.3. Color 선택 다이얼로그

Tcl 에서 색깔은 red, yellow 와 같이 기본적으로 미리 정의된 색이름을 통해 색깔을 지정할 수도 있지만 RGB 코드를 이용하여 색깔을 지정할 수도 있다. RGB 코드를 이용한 색깔 지정코드를 #으로 시작하며 6 자리의 16 진수로 이루어져 있다. 앞쪽 부터 2 자리씩 각각 빨강, 녹색, 파랑에 대한 밝기를 나타낸다. 0 은 꺼져있는 것을 나타내며 ff 는 가장 밝게 켜진 것을 나타낸다.

다음예에서 사용자가 선택한 색깔은 밝은 파란색이다.

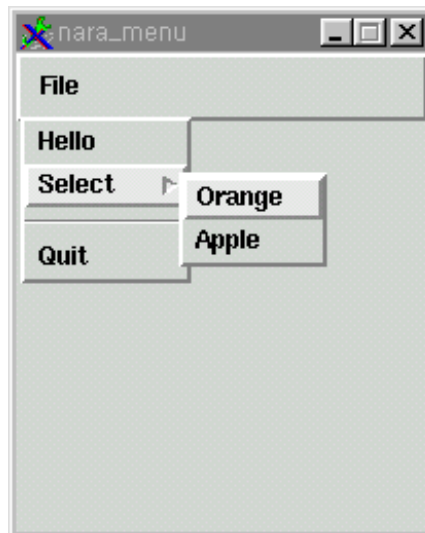


4.7. Menu

폴다운 메뉴는 menu 위젯을 사용하여 만들어 진다. 다음과 같은 메뉴를 만드는 소스를 통하여 폴다운 메뉴를 만드는 방법을 알아 보도록 하자.

```
#!/usr/local/bin/wish
```

```
menu .menubar
. config -menu .menubar
menu .menubar.m1 -tearoff 0
.menubar add cascade -label File -menu .menubar.m1
.menubar.m1 add command -label Hello -command { puts "Hello Menu" }
menu .menubar.m1.m1 -tearoff 0
.menubar.m1 add cascade -label Select -menu .menubar.m1.m1
.menubar.m1.m1 add command -label Orange -command { puts "Orange Menu" }
.menubar.m1.m1 add command -label Apple -command { puts "Apple Menu" }
.menubar.m1 add separator
.menubar.m1 add command -label Quit -command { exit }
```



최상위 메뉴는 루트 위젯에 menu 라는 옵션을 사용하여 등록 된다.

선택했을 때 서브 메뉴가 펼쳐지는 메뉴를 cascade 메뉴라고 한다. 따라서 File 메뉴와 Select 메뉴는 cascade 로 만들어 졌다. cascade 메뉴는 선택되었을 때 보여 줄 서브 메뉴에 대한 정보를 갖고 있어야 한다.

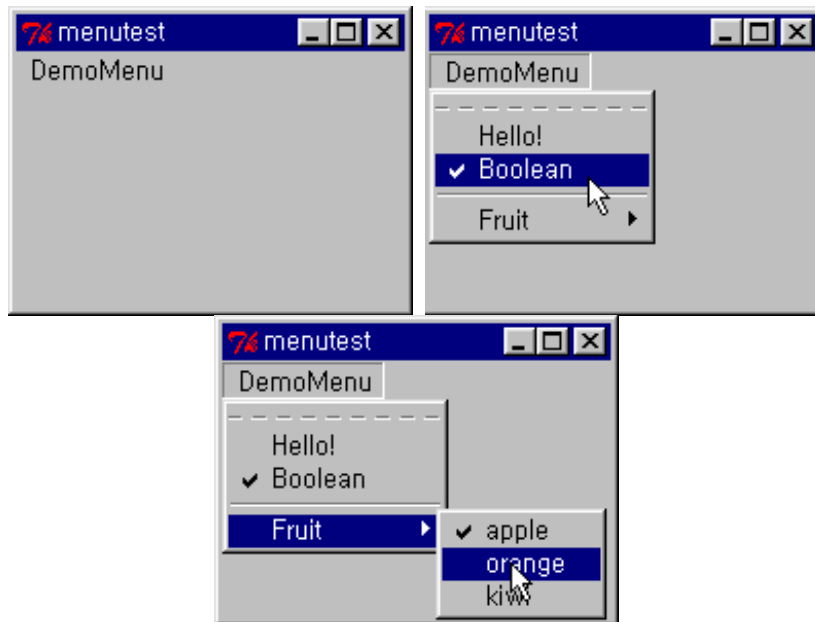
그리고 선택되었을 때 할당된 명령을 실행하는 메뉴를 command 메뉴라고 한다. 따라서 Hello 메뉴와 Orange, Apple, Quit 메뉴는 command 메뉴로 만들어 졌다.

separator 는 메뉴의 종류를 구분해 주기 위해서 삽입되는 가는 가로 줄이다.

연 습 문 제

1. 다음과 같은 간단한 코드를 통하여 다음과 같은 폴다운 메뉴를 만드는 라이브러리를 작성해 보시오.

```
Menu_Setup .menubar
Menu_DemoMenu
Menu_Command DemoMenu Hello! { puts "Hello, World!" }
Menu_Check DemoMenu Boolean foo { puts "foo = $foo" }
Menu_Separator DemoMenu
Menu_Cascade DemoMenu Fruit
Menu_Radio Fruit apple fruit
Menu_Radio Fruit orange fruit
Menu_Radio Fruit kiwi fruit
```



4.8. Resource

Resource 는 Tk 위젯의 속성 값을 정해 주기 위해 사용된다. Tk 위젯의 속성을 스크립트 내에서 hard coding 하지 않고 리소스 데이터 파일에 따로 모아 둬으로써, 사용자가 자신의 취향에 맞게 Tk 위젯의 속성을 쉽게 바꿀 수 있게 해준다. 그리고 Tk 위젯 속성값 할당을 리소스에서 하도록 함으로써 스크립트 소스가 보다 간결해 질 수 있다.

Tk 위젯이 만들어졌을 때 그 속성은 다음의 3 가지 중에 하나로 정해 진다.

- * Tcl 명령어에 직접 주어진 option 이 있으면 그 값을 가장 우선하여 적용한다.
- * Resource 에서 주어진 값이 있으면 그 값을 적용한다.
- * 위의 2 가지 중 어디에도 해당되지 않으면 기본 값을 적용한다.

4.8.1. 리소스 파일 로딩

스크립트에서 사용할 리소스 파일을 로딩하기 위해서 option 명령어를 사용한다.

```
option readfile resourceName ?priority?
```

resourceFileName 자리에 읽어들일 리소스 파일명을 써준다. priority 는 스크립트 내에서 여러 개의 리소스 파일을 읽어들일 때 충돌하는 속성값이 있을 때 적용할 우선 순위이다. 우선 순위에 적용할 수 있는 값은 0 부터 100 까지의 수이며, 높을 수록 우선 순위도 높다. 기본 값은 80 이다.

```
if [ file exists $resourceFile ] {  
    if[ catch { option readfile $resourceFile } err ] {  
        puts stderr "error in $resourceFile: $err"  
    }  
}
```

4.8.2. 리소스 파일 형식

“!”를 줄의 맨 앞에 써서 주석을 쓸 수 있다. 주석 이외의 줄들은 모두 다음과 같은 형식으로 되어 있다.

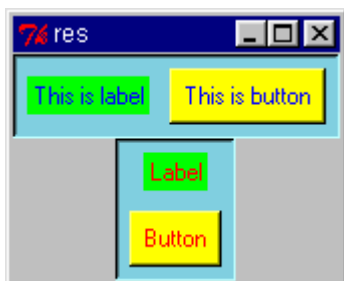
Key : Value

Key 값은 Tk 위젯을 가리키는 값이다. 특정 위젯 instance 를 가리킬 수도 있고 위젯 클래스 전체를 가리킬 수도 있다. 자세한 경로명이 불필요할 경우에는 “*”를 써서 생략할 수도 있다. 클래스명과 “*”를 이용하여 여러 위젯의 값을 한꺼번에 지정할 수 있다. Value 값은 Key 값에 매칭되는 위젯의 속성 값이다.

리소스 파일과 이 리소스 파일을 이용하는 예제는 다음과 같다.

```
!  
! Resource file for test  
!  
  
*f2.l.text : Label  
*f2.b.text : Button  
*f1.l.text : This is label  
*f1.b.text : This is button  
  
*f1*foreground : blue  
*f2*foreground : red  
*Label.background : green  
*Button.background : yellow  
*Frame.background : sky blue  
*Frame.borderWidth : 2  
*Frame.relief : sunken
```

```
#!/usr/local/bin/wish  
option readfile test.res  
frame .f1  
frame .f2  
label .f1.l  
button .f1.b  
label .f2.l  
button .f2.b  
pack .f1.l .f1.b -side left -padx 5 -pady 5  
pack .f2.l .f2.b -padx 5 -pady 5  
pack .f1 .f2
```



5. 관련 프로그램

Tcl/Tk 프로그래밍과 관련된 몇 가지 툴들에 대하여 개략적으로 살펴 본다. 필자가 직접 사용해 본 툴들에 한해 설명함을 이해해 주기 바란다.

5.1. vtcl

GUI 프로그래밍에서 위젯의 크기, 위치, 색상 등을 조절하는 일은 프로그램의 외양을 결정하는 매우 중요한 작업이다. 그러나 한 번에 만족할만한 결과를 얻기는 매우 힘들다. 따라서 만족할만한 결과를 얻을 때 까지 수차례 수정을 반복해야 하는 매우 지루한 작업이다.

Visual Tcl 은 이러한 작업을 매우 쉽게 도와 주는 도구 이다. 위젯의 크기, 위치를 마우스 드래그를 사용하여 조절할 수 있도록 해주기 때문이다. Tcl/Tk 가 기본적으로 제공하는 모든 위젯을 생성할 수 있으며 이들의 성질을 마음대로 변경할 수 있으며, 변경된 내용을 즉시 확인할 수 있다.

Visual Tcl 의 공식 웹사이트는 다음과 같다.

<http://www.sco.com/products/vtcl/vtcl.html>

Visual Tcl 프로그램 자체도 모두 Tcl/Tk 로만 작성되어 있기 때문에 설치가 매우 간단하고 어떤 Tcl/Tk 를 사용할 수 있는 OS 라면 어떤 OS 상에서도 사용할 수 있다.

vtcl 을 사용할 때 유의할 점은 한번 vtcl 을 작성하여 소스를 생성한 후 수정을 위하여 vtcl 로 다시 읽어 들였다가 저장하게 되면, 손으로 고친 대부분의 코드들이 사라진다는 점이다. 따라서 손으로 작성해야 할 부분은 다른 파일로 만들고 package 화 하는 것이 좋다. 이렇게 package 화 된 서브루틴들을 vtcl 내에서 attribute editor 상에서 호출하도록 프로그램하여야 한다.

5.2. TclPro

TclPro 는 Tcl/Tk 스크립트 프로그램을 컴파일하여 이진 코드로 만들어 주는 프로그램이다. 따라서 이 프로그램을 사용하면 Tcl/Tk 로 작성된 프로그램을 사용자에게 배포할 때 소스 코드를 공개하지 않을 수 있다. 이에 더하여 시스템에 Tcl/Tk 인터프리터나 기타 라이브러리가 전혀 설치되어 있지 않더라도 단독으로 실행되는 프로그램을 만들 수 있도록 해준다. 다시말해서 스크립트 프로그램 수행에 필요한 모든 라이브러리와 리소스들을 실행파일내에 포함시켜 독립된 프로그램을 만들어 준다.

Tcl/Tk 를 사용할 수 있는 모든 OS 에 대하여 각각 사용 가능한 TclPro 버전이 제공 되고 있으며, 일정 기간 동안 시험적으로 사용해 볼 수 있다.

<http://www.scriptics.com>

에서 프로그램을 다운 받을 수 있다.

6. 예제 프로그램

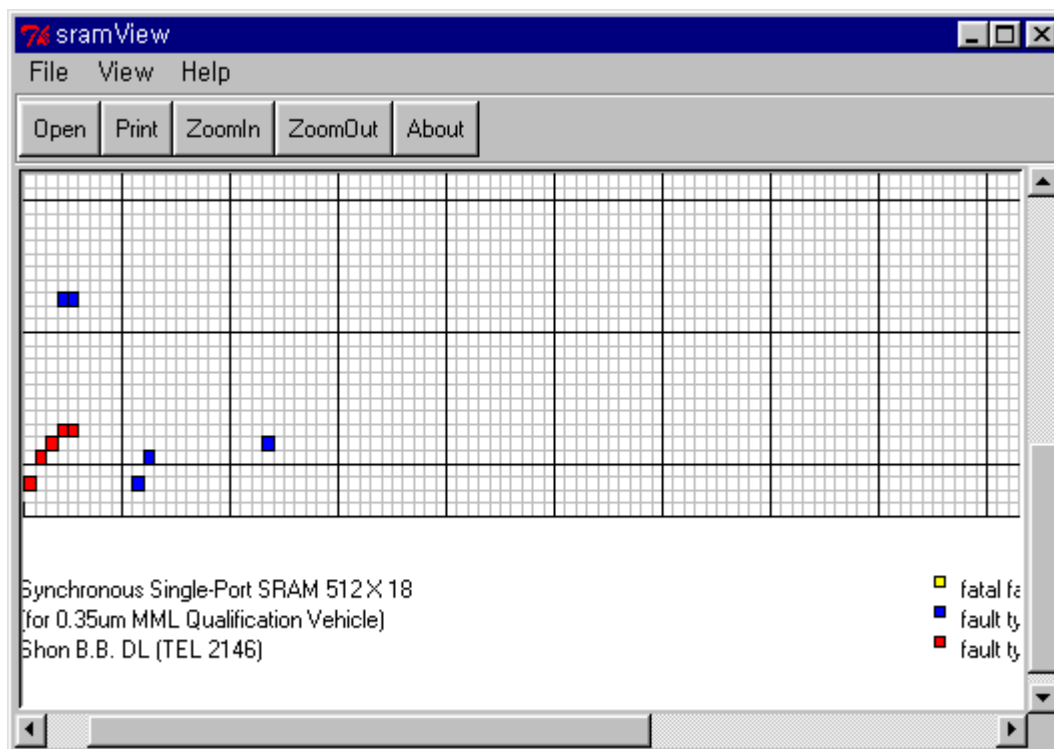
Tcl/Tk 를 이용하여 할 수 있는 일은 무궁 무진하다. 그 중에서 필자가 Tcl/Tk 를 사용하여 작성한 간단한 예제 프로그램을 통하여 Tcl/Tk 를 업무에 어떻게 적용할 수 있을 지에 대한 힌트를 얻을 수 있기를 기대 한다. 단, 이 프로그램은 Tcl/Tk 를 공부하고 제작한 첫 프로그램이라 Tcl/Tk 를 자유자재로 이용하는 완숙한 기교를 보여줄 수 없다는 점에 대하여 양해를 구하는 바이다.

6.1. 프로그램의 목적 및 기능

SRAM 의 Cell 들 중 Fault 가 발생한 Cell 들에 대한 정보를 파일로 입력 받아 화면에 도시해 주는 프로그램이다. 사용자는 Fault 의 종류에 따라 색깔을 다르게 지정할 수 있고 Fault 에 대한 간단한 Comment 를 삽입할 수 있다. 그리고 화면에 표시된 내용을 프린트로 출력할 수 있다.

UNIX 환경이 갖추어 지지 않은 FAB 안에서도 사용할 수 있도록 PC 에서도 실행 가능해야 한다. 그리고 UNIX 환경에 친숙한 연구원들을 위하여 UNIX 상에서도 실행 가능해야 한다.

6.2. 프로그램 실행 모습



6.3. Readme 파일

#####

SRAM Viewer Ver 1.0

System IC R&D in Hyundai Electronics Inc. Copryright (c), 1998

by Nara Won

#####

1. 시스템 요구 사항

- OS
 - Windows 95 or NT 또는
 - UNIX with X Windows
- Required Software
 - Tcl/Tk 8.0 or later

2. 프로그램 인스톨

- Windows 95 or NT
 - 적당한 Folder 를 만든다.
 - 예) C:\CADUtil
 - sramview.zip 파일을 그 Folder 내에서 압축을 푼다.
 - pkunzip 혹은 winzip 이용
- UNIX
 - 적당한 directory 를 만든다.
 - 예) ~/CADUtil
 - sramview.tar.gz 파일의 압축을 해제 한다.
 - gunzip sramview.tar.gz
 - sramview.tar.gz 에서 sramview.tar 로 바뀜
 - sramview.tar 파일에서 파일들을 추출한다.
 - tar xvf sramview.tar ~/CADUtil

3. 배포되는 파일 리스트

- ~/CADUtil 에 프로그램을 인스톨 했을 경우
- ~/CADUtil/sramview
 - sramView.tcl
 - SRAM Viewer main program file
 - sramView.resource
 - SRAM Viewer resource file
 - readme.txt
 - This file
 - sample.srv
 - Sample Data
- ~/CADUtil/GUI
 - GUI.tcl
 - SRAM Viewer GUI package
 - pkgIndex.tcl
 - index file of SRAM Viewer GUI package routines

4. 프로그램 사용 방법

- 프로그램 시작

- UNIX
 - % sramView.tcl [data_file_name]
- Windows
 - C:> wish sramView.tcl [data_file_name]
 - Tcl/Tk의 버전에 따라 Tcl/Tk Shell 프로그램 이름이 달라질 수 있다. 예) wish -> wish80
 - sramView.tcl을 Double Click하여 실행할 수 있다.
 - sramView.tcl의 단축아이콘을 만들어 놓고 이것을 Double Click하여도 실행할 수 있다.
- command line에서 데이터 파일 이름을 줄 경우 그 파일이 자동으로 Open된다.

- 메뉴

- File
 - Open
 - 새로운 데이터 파일을 Open한다.
 - Close
 - 현재 보고 있는 데이터 File을 Close한다.
 - Print
 - 현재 윈도우의 내용을 postscript 파일로 만든다.
 - !!주의!! 윈도우에 보이는 부분만 출력됨
 - Quit
 - Program을 종료한다.
- View
 - ZoomIn
 - 확대하여 본다.
 - 원본 이미지의 4배까지 확대 가능하다.
 - 한번 누를 때 마다 25%씩 확대비율을 높인다.
 - ZoomOut
 - 축소하여 본다.
 - 원본 이미지의 1/4까지 축소 가능하다.
 - 한번 누를 때 마다 25%씩 확대비율을 줄인다.
- Help
 - About
 - 프로그램에 대한 정보를 보여준다.

5. 데이터 파일 형식

- 확장자

- *.srv

- 파일 내용

- comment
 - #으로 시작하는 라인은 comment로 간주
 - 화면 하단에 모든 comment line들이 그대로 출력될 것임

- SRAM 정보

- info_name = value
 - info_name의 종류

- xunit, yunit : Cell 의 실제 길이 (단위 ?)
 - xnum, ynum : Cell 의 갯수
 - 한 줄에 정보 하나씩만
 - 위의 4 가지 정보는 반드시 제공되어야 함
- Failed Cell 정보 블록
 - 색깔 정보
 - color = color_value : comment
 - color_value 에 가능한 값
 - red, green, blue, grey, black
 - yellow, magenta, violet, orange,
 - pink, purple, brown
 - comment 는 화면 하단에 color box 다음에 출력될 것임
 - 색깔 정보는 위치 정보보다 앞에 위치해야 함
 - 색깔 정보 다음에 나오는 위치정보의 Cell 들은 모두 그 색으로 채워짐
 - 위치 정보
 - Xi, Yi
 - Xi : X index
 - Yi : Y index
 - 한 줄에 한 Cell 의 정보만
 - 위치정보 규약
 - 가장 왼쪽 가장 아래쪽이 (1,1)로 간주
 - 오른쪽이 x+ 방향
 - 위쪽이 y+ 방향
 - 기타
 - space, tab, blank line 은 가독성을 높이기 위해 적절히 사용해도 무방
 - 각 정보가 나오는 순서는 Failed Cell 정보를 제외하고는 어디에 위치해도 무방함
 - Failed Cell 정보 블록 자체는 어디에 위치해도 무방
 - Failed Cell 정보 블록 내에서 Color 정보와 위치 정보의 순서는 지켜야 함
- 데이터 파일 예제는 함께 배포된 "sample.srv"파일 참조

6.4. Source Files

6.4.1. sramview.tcl

```
#!/usr/local/bin/wish
#####
#
#   SRAM VIEWER Ver 0.0
#
#       for SRAM Fail Analysis
#
#   Copyright (c) System ID R&D in Hyundai Electronics Inc. , 1998
#
#
#   Programed by :
#       1998.8.18      Nara Won
#
#####

#
# read resource file
#

option readfile sramView.resource

#
# load package files
#

lappend auto_path ../GUI

package require GUI

#*****
# Global Variables
#-----
# comments
#     - comments list for print
# sramUnitX
#     - width of SRAM cell
# sramUnitY
#     - height of SRAM cell
# sramNumX
#     - width of SRAM
# sramNumY
#     - height of SRAM
# faultCells
#     - struct array for fault cell coordinates
#     - $faultCells($index,xi) : X index
#     - $faultCells($index,yi) : Y index
#     - $faultCells($index,color) : color of fault cell
# faultCellNum
#     - # of faulted cell
# colorComments
#     - array for comments about color
#     - $colorComments($color)
# canvas
```

```

#      - canvas widget to draw SRAM
# zoomFactor
#      - zoomFactor
# scrollOrgWidth, scrollOrgHeight
#      - original whole canvas width and height
#*****

#*****
# Subroutines
#*****

#=====
# ReadDataFile
#      : Read SRAM Data File
#-----
# Input :
#      fileName - SRAM Data File Name
#-----
# Return Value :
#      1 - no error
#      0 - error occurred
#=====

proc ReadDataFile { fileName } {

    global sramUnitX sramUnitY sramNumX sramNumY
    global faultCells faultCellNum
    global comments colorComments

    #
    # open file
    #

    if [catch { open $fileName } fileID ] {
        GUIErrorBox "$fileName is not found"
        return 0
    }

    #
    # global variable initialize
    #

    set sramUnitX 0
    set sramUnitY 0
    set sramNumX 0
    set sramNumY 0
    set faultCellNum 0

    #
    # process file data
    #

    set color 0
    set lineNum 0

    foreach currLine [split [read $fileID] \n] {

        incr lineNum
    }
}

```

```

#
# extract comment
#

if [regexp {[ \t]*#(.*)$} $currLine match comment] {
    lappend comments $comment
    continue
}

#
# extract color
#
# Note : color clause has comment.
#       so it must be manipulated before white spaces are
#       removed
#

if [regexp {[ \t]*color[ \t]*=[ \t]*([a-z]+)[ \t]*:(.*)$} \
    $currLine match color comment] {

    set colorComments($color) $comment
    continue
}

#
# remove all white spaces
#
# Note : \t must be matched with tab character.
#       But in regsub, it does not work.
#       I think this is bug of PC version wish80.exe
#       So I replaced \t with real tab character.
#

regsub -all -- {[ \t]+} $currLine {} line

#
# null line skip
#

if {$line == ""} \
    continue

#
# extract xunit, yunit, width, height
#

if [regexp {^xunit=([0-9.]+)$} \
    $line match sramUnitX] {
    continue
}
if [regexp {^yunit=([0-9.]+)$} \
    $line match sramUnitY] {
    continue
}
if [regexp {^xnum=([0-9.]+)$} \
    $line match sramNumX] {
    continue
}

```

```

    }
    if [regexp {^ynum=([0-9.]+)$} \
        $line match sramNumY] {
        continue
    }

    #
    # extract fault cell geometry
    #

    if [regexp {^([0-9.]+),([0-9.]+)$} \
        $line match xi yi ] {

        if {$color == 0} {
            GUIErrorBox "$lineNum : Cell color not
defiled\n$currLine"
            return 0
        }
        set faultCells($faultCellNum,xi) $xi
        set faultCells($faultCellNum,yi) $yi
        set faultCells($faultCellNum,color) $color
        incr faultCellNum

        continue
    }

    #
    # if execution flow is here then error
    #

    GUIErrorBox "$lineNum : Invalide line \n$currLine"
    return 0

}

close $fileID

#
# Check all informations were given
#

if { $sramUnitX == 0 } {
    GUIErrorBox "No information about xunit in \"$fileName\""
    return 0
}

if { $sramUnitY == 0 } {
    GUIErrorBox "No information about yunit in \"$fileName\""
    return 0
}

if { $sramNumX == 0 } {
    GUIErrorBox "No information about Width in \"$fileName\""
    return 0
}

if { $sramNumY == 0 } {
    GUIErrorBox "No information about Height in \"$fileName\""
    return 0
}

```

```

}

#puts $sramUnitX
#puts $sramUnitY
#puts $sramNumX
#puts $sramNumY
#puts $faultCellNum

foreach color [array names colorComments] {
    switch -- $color {
        red -
        green -
        blue -
        grey -
        black -
        yellow -
        magenta -
        violet -
        orange -
        pink -
        purple -
        brwon {
        }
        default {
            GUIErrorBox "$color is not valid color"
            return 0
        }
    }
}

}

return 1

}

```

```

#=====
# Draw :
#     Draw SRAM (all cells and faulted cells )
#=====

```

```

proc Draw {} {
    global canvas
    global sramUnitX sramUnitY sramNumX sramNumY
    global scrollOrgWidth scrollOrgHeight
    global comments colorComments

    set canvasMarginX    30
    set canvasMarginY    30

    #
    # Draw SRAM cells
    #

    DrawSRAMCells \
        $sramUnitX $sramUnitY $sramNumX $sramNumY \
        $canvasMarginX $canvasMarginY

```

```

#
# Print comments
#

set xstart $canvasMarginX
set yStartText [expr $sramUnitY * $sramNumY + $canvasMarginY*2]

foreach comment $comments {
    $canvas create text $xstart $yStartText \
        -text $comment -anchor nw -justify left
    set yStartText [expr $yStartText + 15]
}

#
# Print color comments
#

set xstart 500
set yStartColor [expr $sramUnitY * $sramNumY + $canvasMarginY*2]

foreach color [array names colorComments] {
    set comment $colorComments($color)

    set xl [expr $xstart - 10]
    set xr [expr $xstart - 5]
    set yt [expr $yStartColor + 0]
    set yb [expr $yStartColor + 5]

    $canvas create rectangle $xl $yt $xr $yb -fill $color
    $canvas create text $xstart $yStartColor \
        -text $comment -anchor nw -justify left
    set yStartColor [expr $yStartColor + 15]
}

#
# Set Scroll region
#

if { $yStartText > $yStartColor } {
    set ymax $yStartText
} else {
    set ymax $yStartColor
}

set scrollOrgWidth [expr $sramUnitX * $sramNumX + $canvasMarginX*2]
set scrollOrgHeight [expr $ymax + $canvasMarginY]

$canvas configure -scrollregion [list 0 0 $scrollOrgWidth
$scrollOrgHeight]
$canvas config -width $scrollOrgWidth -height $scrollOrgHeight

}

#=====

```

```

# DrawSRAMCells
#       : for drawing SRAM cells with rectangles
#-----
# Input :
#       canvas - canvas widget path
#       xunit  - cell width
#       yunit  - cell height
#       xnum   - # of cells on row
#       ynum   - # of cells on column
#       xmargin, ymargin - margine for SRAM
#-----
# Output :
#       cellBoxes - 2 dimension array for cell rectangles on canvas
#=====

proc DrawSRAMCells { xunit yunit xnum ynum \
                    xmargin ymargin } {

    global canvas colNormalFill colNormalLine
    global faultCellNum faultCells

    set xstart $xmargin
    set ystart $ymargin

    set xend [expr $xunit * $xnum + $xstart]
    set yend [expr $yunit * $ynum + $ystart]

    #
    # Draw Vertical lines
    #

    for { set w 1 } { $w < $xnum } { incr w } {

        set xl [expr $w * $xunit + $xstart]

        if { [expr $w % 10] != 0 } {

            $canvas create line $xl $ystart $xl $yend \
                -fill $colNormalLine

        }
    }

    #
    # Draw Horizontal lines
    #

    for { set h 1 } { $h < $ynum } { incr h } {

        set yb [expr $h * $yunit + $ystart]
        if { [expr $h % 10] == 0 } {
            set lineCol black
        } else {
            set lineCol $colNormalLine
        }
        $canvas create line $xstart $yb $xend $yb \
            -fill $lineCol
    }
}

```



```

}

#
# Draw Horizontal lines per 10 units
#

for { set w 10 } { $w < $xnum } { incr w 10 } {
    set xl [expr $w * $xunit + $xstart]
    $canvas create line $xl $ystart $xl $yend \
        -fill black
}

#
# Draw border of SRAM
#

$canvas create rectangle $xstart $ystart $xend $yend

#
# Draw Faulted Cells
#

for { set i 0 } { $i < $faultCellNum } { incr i } {

    set xi $faultCells($i,xi)
    set yi $faultCells($i,yi)
    set col $faultCells($i,color)

    set xl [expr ($xi - 1) * $xunit + $xstart]
    set xr [expr $xi * $xunit + $xstart]
    set yt [expr ($ynum - $yi) * $yunit + $ystart]
    set yb [expr ($ynum - $yi + 1) * $yunit + $ystart]

    $canvas create rectangle $xl $yt $xr $yb -fill $col
}
}

#=====
#=====

proc Zoom { how } {
    global zoomFactor canvas
    global scrollOrgWidth scrollOrgHeight

    set revFactor [expr 1.0 / $zoomFactor]

    if { $show == "in" } {
        set zoomFactor [expr $zoomFactor + 0.25]
        if { $zoomFactor > 4.0 } {
            set zoomFactor 4.0
            return
        }
    } else {
        set zoomFactor [expr $zoomFactor - 0.25]
        if { $zoomFactor < 0.25 } {

```

```

        set zoomFactor 0.25
        return
    }
}

foreach tag [$canvas find all] {
    $canvas scale $tag 0 0 $revFactor $revFactor
}

foreach tag [$canvas find all] {
    $canvas scale $tag 0 0 $zoomFactor $zoomFactor
}

set width [expr $scrollOrgWidth * $zoomFactor]
set height [expr $scrollOrgHeight * $zoomFactor]
$canvas configure -scrollregion [list 0 0 $width $height]
}

#####
# MenuOpen :
#   command routine for Open menu
#####

proc MenuOpen {} {

    set typelist {
        { "SRAM Viewer Files" {"*.srv"} }
        { "All Files" {*} }
    }

    set filename [tk_getOpenFile -filetypes $typelist -defaultextension srv]
    set ret [ReadDataFile $filename]

    if { $ret == 1 } {
        Draw
    }
}

#####
# MenuClose :
#   command routine for Close menu
#####

proc MenuClose {} {
    global canvas

    foreach tag [$canvas find all] {
        $canvas delete $tag
    }
}

#####
# MenuPrint :
#   command routine for Print menu
#####

proc MenuPrint {} {

```

```

global canvas

set width [$canvas cget -width]
set height [$canvas cget -height]
set region [$canvas cget -scrollregion]

set typelist {
    { "Postscript Files" {"*.ps"} }
    { "All Files" {*} }
}
set filename [tk_getSaveFile -filetypes $typelist -defaultextension ps\
    -initialfile noname ]
$canvas postscript -file $filename -rotate true\
    -pagewidth 27c -pageheight 18c
}

#####
# MenuQuit :
#     command routine for Quit menu
#####

proc MenuQuit {} {
    set answer [ GUIQuestionBox \
        "Do you want to quit SRAM Viewer?" \
        yes ]
    if { $answer == "yes" } {
        exit 0
    }
}

#####
# MenuZoomIn :
#     command routine for ZoomIn menu
#####

proc MenuZoomIn {} {
    puts "zoom in "

    Zoom in
}

#####
# MenuZoomOut :
#     command routine for ZoomOut menu
#####

proc MenuZoomOut {} {
    puts "zoom out "
    Zoom out
}

#####
# MenuAbout :
#     command routine for About menu
#####

proc MenuAbout {} {
    GUIMessageBox About \
        "          SRAM Viewer Ver 1.0\n\n  Copyright (c) System IC R&D \nin

```

```

Hyundai Electronics Inc. , 1998\n\n    Program by Nara Won"
}

#*****
# Main routine
#*****

#=====
#
# Temporary Data Setting
#
#=====

if { $tcl_platform(platform) == "windows" } {
    #console show
}

#=====
#
# Make Menu
#
#=====

#
# Full Down Menu
#

GUIMenuSetup .menubar

GUIMenu File
GUIMenuCommand File Open      MenuOpen
GUIMenuCommand File Close     MenuClose
GUIMenuSeparator File
GUIMenuCommand File Print     MenuPrint
GUIMenuSeparator File
GUIMenuCommand File Quit      MenuQuit

GUIMenu View
GUIMenuCommand View ZoomIn    MenuZoomIn
GUIMenuCommand View ZoomOut   MenuZoomOut

GUIMenu Help
GUIMenuCommand Help About     MenuAbout

#
# User Defined Tool Bar
#

GUIResourceButtonFrame .user User

#=====
#
# Make canvas
#
#=====

set colNormalFill    white

```

```

set colNormalLine    grey
set zoomFactor       1

set canvas [ GUIScrolledCanvas .c -width 300 -height 200 \
    -relief sunken -bd 2 -background white]

if { $argc > 0 } {

    set ret [ReadDataFile [lindex $argv 0]]
    if { $ret == 1 } {
        Draw
    }
}

pack .c -fill both -expand true

```

6.4.2. gui.tcl

```
package provide GUI 1.0

#####
#
# GUI package for Tcl/Tk applications
#
# Copyright (c) System IC R&D in Hyundai Electronics Inc. , 1998
#
# Programed by :
#     1998.8.18.      Nara Won
#
#####

#=====
# GUIScrollCanvas :
#     Subroutine for making scrolled canvas
#-----
# Input :
#     c - parent widget
#     args - options for canvas widget
#-----
# Return vlaue :
#     canvas widget path
#=====

proc GUIScrolledCanvas { c args } {
    frame $c
    eval {canvas $c.canvas \
        -xscrollcommand [list $c.xscroll set] \
        -yscrollcommand [list $c.yscroll set] \
    } $args
    scrollbar $c.xscroll -orient horizontal \
        -command [list $c.canvas xview]
    scrollbar $c.yscroll -orient vertical \
        -command [list $c.canvas yview]
    grid $c.canvas $c.yscroll -sticky news
    grid $c.xscroll -sticky ew
    grid rowconfigure $c 0 -weight 1
    grid columnconfigure $c 0 -weight 1
    return $c.canvas
}

#=====
# GUIMenuSetup :
#     for initialize full down menu
#-----
# Input :
#     menubar - Menu bar widget path
#=====

proc GUIMenuSetup { menubar } {
    global menu
    menu $menubar
    set top [wininfo parent $menubar ]
    $top config -menu $menubar
}
```

```

        set menu(menubar) $menubar
        set menu(uid) 0
    }

#=====
# GUIMenu :
#     Insert menu item in menu bar
#     and make subpane for submenus
#-----
# Input :
#     label - name(text) of menu item
#=====

proc GUIMenu { label } {
    global menu
    if [info exists menu(menu,$label)] {
        error "Menu $label already defined"
    }
    set menuName $menu(menubar).mb$menu(uid)
    incr menu(uid)
    menu $menuName -tearoff 1
    $menu(menubar) add cascade -label $label -menu $menuName
    set menu(menu,$label) $menuName
}

#=====
# GUIMenuGet :
#     Get subpane handler with menu name
#     subpane was made in GUIMenu
#-----
# Input :
#     menuName - menu name of subpane
#-----
# Return :
#     subpane handler
#=====

proc GUIMenuGet { menuName } {
    global menu
    if [ catch {set menu(menu,$menuName) } m] {
        return -code error "No such menu: $menuName"
    }
    return $m
}

#=====
# GUIMenuCommand :
#     Bind command with menu item
#-----
# Input :
#     menuName - name of parent menu
#     label - name of its own
#     command - command of the menu
#=====

proc GUIMenuCommand { menuName label command } {
    set m [GUIMenuGet $menuName]
    $m add command -label $label -command $command
}

```

```

#=====
# GUMenuCheck :
#     Add check button menu
#-----
# Input :
#     menuName - name of parent menu
#     label - name of its own
#     var - variable of the menu
#     command - command of the menu (option)
#=====

proc GUMenuCheck { menuName label var { command {} } } {
    set m [GUMenuGet $menuName]
    $m add check -label $label -command $command\
        -variable $var
}

#=====
# GUMenuRadio :
#     Add radio button menu
#-----
# Input :
#     menuName - name of parent menu
#     label - name of its own
#     var - variable of the menu
#     val - value of radio button (option)
#     command - command of the menu (option)
#=====

proc GUMenuRadio { menuName label var { val {} } {command {}} } {
    set m [GUMenuGet $menuName]
    if {[string length $val] == 0} {
        set val $label
    }
    $m add radio -label $label -command $command\
        -value $val -variable $var
}

#=====
# GUMenuSeparator :
#     Insert separator to menu pane
#-----
# Input :
#     menuName - name of parent menu
#=====

proc GUMenuSeparator { menuName } {
    [GUMenuGet $menuName] add separator
}

#=====
# GUMenuCascade :
#     Insert cascade menu
#     cascade menu has submenu pane
#-----
# Input :
#     menuName - name of parent menu
#     label - name of its own
#=====

proc GUMenuCascade { menuName label } {

```



```

global menu
set m [ GUIMenuGet $menuName]
if [info exists menu(menu,$label)] {
    error "Menu $lable already defined"
}
set sub $m.sub$menu(uid)
incr menu(uid)
menu $sub -tearoff 0
$m add cascade -label $label -menu $sub
set menu(menu,$label) $sub
}

#####
# GUIResourceButtonFrame :
#     make user defined (at resource file) menu buttons
#-----
# Input :
#     f - path of parent of user defined menu button frame
#     class - class name of user defined menu button frame
#=====

proc GUIResourceButtonFrame { f class } {
    frame $f -class $class -borderwidth 2 -relief groove
    pack $f -side top -fill x
    foreach b [option get $f buttonlist {} ] {
        if [catch {button $f.$b}] {
            button $f.$b -font fixed
        }
        pack $f.$b -side left -pady 2
    }
}

#####
# GUIMessageBox :
#     Make simple message box dialog with only OK button
#-----
# Input :
#     title - title of message box dialog
#     message - message of message box dialog
#=====

proc GUIMessageBox { title message } {
    tk_messageBox -type ok -message $message -title $title
}

#####
# GUIQuestionBox :
#     Make simple question box dialog with YES and NO button
#-----
# Input :
#     message - message of message box dialog
#     default - default choice of dialog (yes or no)
#=====

proc GUIQuestionBox { message default} {
    return [tk_messageBox -type yesno -default $default -title Question\
        -message $message -icon question ]
}

```

```

#=====
#-----
#=====

proc GUIErrorBox { message } {
    tk_messageBox -type ok -message $message -title Error -icon error
}

```

6.4.3. sramview.resource

```

!
! Resource file for SRAM Viewer
!

*User.buttonlist : open print zoomIn zoomOut about

*User.open.text : Open
*User.open.command : MenuOpen

*User.print.text : Print
*User.print.command : MenuPrint

*User.zoomIn.text : ZoomIn
*User.zoomIn.command : MenuZoomIn

*User.zoomOut.text : ZoomOut
*User.zoomOut.command : MenuZoomOut

*User.about.text : About
*User.about.command : MenuAbout

```

6.5. Sample Data

6.5.1. sample.srv

```
# Synchronous Single-Port SRAM 512 X 18  
# (for 0.35um MML Qualification Vehicle)  
# Shon B.B. DL (TEL 2146)
```

```
xunit = 5.4  
yunit = 6.56  
xnum = 144  
ynum = 64
```

```
color = red : fault type 1
```

```
2, 3  
3, 5  
4, 6  
5, 7  
6, 7  
33, 44  
10, 50  
100, 200
```

```
color = blue : fault type 2
```

```
12, 3  
13, 5  
24, 6  
5, 17  
6, 17  
33, 44  
10, 50  
100, 200
```

```
color = yellow : fatal fault
```

```
1, 1  
144, 1  
1, 64  
144, 64
```

7. 연습문제 풀이

7.1. 구구단 출력 프로그램

```
#!/usr/local/bin/tclsh
# 구구단 출력 프로그램

for { set i 2 } { $i < 10 } { incr i } {
    for { set j 1 } { $j < 10 } { incr j } {
        puts "$i * $j = [ expr $i * $j ] "
    }
    puts "---- Press Return Key to Continue --- "
    flush stdout
    gets stdin temp
}
```

7.2. 숫자 맞추기 게임

```
#!/usr/local/bin/tclsh
# 숫자 맞추기 게임

set mynum [ exec date | cut -c18-19 ]
incr mynum

set score 100

while { 1 } {

    puts -nonewline "Input Your Guess \[ 1 ~ 60 \] : "
    flush stdout
    gets stdin yournum

    if { $yournum > $mynum } {
        puts " Oh, too big"
    } elseif { $yournum < $mynum } {
        puts " Oh, too small"
    } else {
        break
    }

    incr score -10

}

puts "Congratulation!!"
puts "Your Score is $score"
```

7.3. 데이터 분석 프로그램

```
#!/usr/local/bin/tclsh

while { [ eof stdin ] != 1 } {
    gets stdin line

    if [ string match "#*" $line ] continue

    if { "$line" == "" } continue

    if [ string match "*:~" $line ] {

        set ll [ split $line : ]

        set type [ lindex $ll 0 ]
        set value [ lindex $ll 1 ]

        set type [ string trim $type " " ]
        set value [ string trim $value " " ]

        switch -- $type {
            TITLE      { set title    $value }
            COMMENT    { set comment  $value }
            DATA_NUM  { set num      $value }
            LOT_NUM    { set lot      $value }
        }
    } else {

        set ll [ split $line " " ]

        set first 1
        set ll2 ""
        foreach i $ll {
            if { "$i" == "" } continue
            set i [ string trim $i " " ]
            if { $first == 1 } {
                lappend field_name $i
                set first 0
            } else {
                lappend ll2 $i
            }
        }
        lappend field_data $ll2
    }
}

puts "title      : $title"
puts "comment    : $comment"
puts "data number : $num"
puts "lot number  : $lot"

foreach name $field_name data $field_data {
    puts "DATA : $name - $data"
}
```

7.4. Stack

```
#!/usr/local/bin/tclsh

set spointer 0

proc push { data } {

    global mystack
    global spointer

    set mystack($spointer) $data
    incr spointer

}

proc pop { } {

    global mystack
    global spointer

    incr spointer -1
}

proc top { } {

    global mystack
    global spointer

    return $mystack([ expr $spointer - 1 ])
}

puts "PUSH 3"
push 3

puts "PUSH 4"
push 4

puts "PUSH 5"
push 5

puts "TOP [ top ] "

puts "POP"
pop

puts "TOP [ top ] "

puts "POP"
pop

puts "TOP [ top ] "
```

7.5. Queue

```

#!/usr/local/bin/tclsh

set qfirst 0
set qlast 0
set qsize 1000

proc enQ { data } {

    global qfirst
    global qlast
    global myQ
    global qsize

    set myQ($qlast) $data
    set qlast [ expr ( $qlast + 1 ) % $qsize ]
}

proc deQ { } {
    global qfirst
    global qlast
    global myQ
    global qsize

    set qfirst [ expr ( $qfirst + 1 ) % $qsize ]
}

proc first { } {
    global qfirst
    global qlast
    global myQ
    global qsize

    return $myQ($qfirst)
}

proc last { } {
    global qfirst
    global qlast
    global myQ
    global qsize

    return $myQ($qlast)
}

puts "ENQUEUE 4 "
enQ 4
puts "ENQUEUE 5 "
enQ 5
puts "ENQUEUE 6 "
enQ 6

puts "FIRST : [ first ] "
puts "DEQUEUE"
deQ

puts "FIRST : [ first ] "
puts "DEQUEUE"
deQ

```

```
puts "FIRST : [ first ] "
```

7.6. 다차원 배열

```
#!/usr/local/bin/tclsh

for { set ii 0 } { $ii < 10 } { incr ii } {
    for { set jj 0 } { $jj < 10 } { incr jj } {
        set array2d($ii.$jj) [ expr $ii + $jj ]
    }
}

for { set ii 0 } { $ii < 10 } { incr ii } {
    for { set jj 0 } { $jj < 10 } { incr jj } {
        puts $array2d($ii.$jj)
    }
}
```

7.7. 데이터 변환

```
#!/usr/local/bin/tclsh

while { [ eof stdin ] == 0 } {
    gets stdin line
    regsub {[A-Z]+}_([A-Z]+)([0-9]+)} $line {\2-\3\1} line
    puts $line
}
```


7.8. Verilog 파일 분석

```
#!/usr/local/bin/tclsh

set all_line ""

while { [ eof stdin ] == 0 } {

    gets stdin line

    append all_line $line

}

set each_line [ split $all_line ";" ]

foreach line $each_line {

    if [ regexp {module[          ]+([a-zA-Z0-9_]+)} $line all mname ] {
        lappend module_names $mname
    }
    if      [ regexp {input[          ]+([a-zA-Z0-9_]+)} $line all inputs ] {
        regsub -all {[          ]+} $inputs {} inputs
        set inlist [ split $inputs "," ]
        foreach e $inlist {
            lappend data($mname.inputs) $e
        }
    } elseif [ regexp {output[          ]+([a-zA-Z0-9_]+)} $line all outputs ] {
        regsub -all {[          ]+} $outputs {} outputs
        set outlist [ split $outputs "," ]
        foreach e $outlist {
            lappend data($mname.outputs) $e
        }
    } elseif [ regexp {inout[          ]+([a-zA-Z0-9_]+)} $line all inouts ] {
        regsub -all {[          ]+} $inouts {} inouts
        set iolist [ split $inouts "," ]
        foreach e $iolist {
            lappend data($mname.inouts) $e
        }
    }

}

foreach mn $module_names {
    puts "MODULE NAME : $mn"
    puts "    inputs : $data($mn.inputs)"
    puts "    outputs : $data($mn.outputs)"
    puts "    inouts : $data($mn.inouts)"
}
```

7.9. 총 라인 수 구하기

```
#!/usr/local/bin/tclsh

set files [ exec ls ]

set total 0

foreach file $files {
    set fID [ open $file r ]
    set lines [ split [ read $fID ] "\n" ]
    close $fID

    set lnum [ expr [ llength $lines ] - 1 ]

    puts "    $lnum $file"

    set total [ expr $total + $lnum ]
}

puts " TOTAL $total lines"
```

7.10. Package 만들기

```
#!/usr/local/bin/tclsh

package provide QUEUE 1.0

proc initQ { } {

    global qfirst qlast qsize

    set qfirst 0
    set qlast 0
    set qsize 1000
}

proc enQ { data } {

    global qfirst
    global qlast
    global myQ
    global qsize

    set myQ($qlast) $data
    set qlast [ expr ( $qlast + 1 ) % $qsize ]
}

proc deQ { } {

    global qfirst
    global qlast
    global myQ
    global qsize

    set qfirst [ expr ( $qfirst + 1 ) % $qsize ]
}

proc first { } {

    global qfirst
    global qlast
    global myQ
    global qsize

    return $myQ($qfirst)
}

proc last { } {

    global qfirst
    global qlast
    global myQ
    global qsize

    return $myQ($qlast)
}
```

```

#!/usr/local/bin/tclsh

package provide STACK 1.0

proc initStack { } {

    global spointer

    set spointer 0

}

proc push { data } {

    global mystack
    global spointer

    set mystack($spointer) $data
    incr spointer

}

proc pop { } {

    global mystack
    global spointer

    incr spointer -1
}

proc top { } {

    global mystack
    global spointer

    return $mystack([ expr $spointer - 1 ])
}

```

```

#!/usr/local/bin/tclsh

lappend auto_path ./lib

package require STACK 1.0
package require QUEUE 1.0

#=====
# Stack test
#=====

initStack

puts "PUSH 3"
push 3

puts "PUSH 4"
push 4

puts "PUSH 5"
push 5

puts "TOP [ top ] "

puts "POP"
pop

puts "TOP [ top ] "

puts "POP"
pop

puts "TOP [ top ] "

#=====
# Queue test
#=====

initQ

puts "ENQUEUE 4 "
enQ 4
puts "ENQUEUE 5 "
enQ 5
puts "ENQUEUE 6 "
enQ 6

puts "FIRST : [ first ] "
puts "DEQUEUE"
deQ

puts "FIRST : [ first ] "
puts "DEQUEUE"
deQ

puts "FIRST : [ first ] "

```

7.11. 숫자 맞추기 게임 GUI

```
#!/usr/local/bin/wish

proc NewGame { } {
    global message
    global myguess
    global user

    set myguess [ exec date | cut -c18-19 ]
    incr myguess
    set message "I have a number 1~60"
    set user "Enter your guess"
    update
}

proc Quit { } {
    exit
}

proc Yes { } {
    global message
    global myguess
    global user
    global score

    update

    if { $user > $myguess } {

        set message "Too Big!!"
        incr score -10

    } elseif { $user < $myguess } {

        set message "Too Small!!"
        incr score -10

    } else {
        set message "Your Score is $score"
    }
    update
}

frame .top
button .top.new -text "New Game" -fg blue -command { NewGame }
button .top.quit -text Quit -fg red -command { Quit }
label .mess -textvariable message
frame .bot
entry .bot.ent -textvariable user -bg white
button .bot.yes -text "Enter" -command { Yes }

pack .top.new .top.quit -side left
pack .top
pack .mess
pack .bot.ent .bot.yes -side left
```

```
pack .bot

set score 100

NewGame
```

7.12. X Scrollbar

```
#!/usr/local/bin/wish

scrollbar .sy -orient vertical -command { .lbox yview }
scrollbar .sx -orient horizontal -command { .lbox xview }

listbox .lbox -xscrollcommand { .sx set } -yscrollcommand { .sy set }

for { set num 1 } { $num < 20 } { incr num } {
    .lbox insert end "This is list item $num. long line is need for x
scroll"
}

grid .lbox .sy -sticky news
grid .sx -sticky news
```

7.13. 구구단 GUI

```
#!/usr/local/bin/wish

text .out -bg white -yscrollcommand { .sy set }
scrollbar .sy -command { .out yview }

grid .out .sy -sticky news

for { set i 2 } { $i < 10 } { incr i } {
    for { set j 2 } { $j < 10 } { incr j } {
        .out insert end "$i x $j = [ expr $i * $j ]\n"
        update
    }
    .out insert end "-----\n"
}

.out yview end
.out config -state disabled
```

7.14. 게임 프로그램

```
#!/usr/local/bin/wish

set windowW 300
set windowH 500
set fighterX [ expr int( $windowW / 2 ) ]
set fighterY [ expr $windowH - 50 ]
set fighterD 5

set bulletExist 0
set bulletX 0
set bulletY 0

proc BulletMove { } {

    global bulletExist
    global bulletX
    global bulletY
    global bullet

    if { $bulletY < 10 } {
        .ca delete $bullet
        set bulletExit 0
    } else {
        incr bulletY -10
        .ca move $bullet 0 -10
        update
        after 50 BulletMove
    }
}

proc FighterFire { } {

    global bulletExist
    global bulletX
    global bulletY
    global bullet

    global fighterX
    global fighterY

    if { $bulletExist == 1 } {
        return
    }

    set bulletX $fighterX
    set bulletY $fighterY

    set xl [ expr $fighterX - 1 ]
    set xr [ expr $fighterX + 1 ]
    set yt [ expr $fighterY - 5 ]
    set yb [ expr $fighterY ]

    set bullet [ .ca create rectangle $xl $yt $xr $yb -fill white ]
    set bulletExit 1
}
```



```

        after 50 BulletMove
    }

proc FighterMove { where } {

    global fighterX
    global fighterD
    global fighter

    switch $where {

        left {

            set delta [ expr -$fighterD ]

        }

        right {

            set delta $fighterD

        }

    }

    incr fighterX $delta
    .ca move $fighter $delta 0

    update

}

canvas .ca -width $windowW -height $windowH -bg black

set fighter [ .ca create poly 150 450 160 480 140 480 -fill yellow ]

bind . <Key-Left> { FighterMove left }
bind . <Key-Right> { FighterMove right }
bind . <Key-space> { FighterFire }

pack .ca

```

7.15. Full Down 메뉴

```
#!/usr/local/bin/wish
```

```
lappend auto_path ./menulib
```

```
package require MENU 1.0
```

```
Menu_Setup .menubar
```

```
Menu DemoMenu
```

```
Menu_Command DemoMenu Hello! { puts "Hello, World!" }
```

```
Menu_Check DemoMenu Boolean foo { puts "foo = $foo" }
```

```
Menu_Separator DemoMenu
```

```
Menu_Cascade DemoMenu Fruit
```

```
Menu_Radio Fruit apple fruit
```

```
Menu_Radio Fruit orange fruit
```

```
Menu_Radio Fruit kiwi fruit
```

```
#!/usr/local/bin/wish
```

```
package provide MENU 1.0
```

```
proc Menu_Setup { menubar } {
```

```
    global menu
```

```
    menu $menubar
```

```
    # Associated menu with its main window
```

```
    set top [wininfo parent $menubar]
```

```
    $top config -menu $menubar
```

```
    set menu(menubar) $menubar
```

```
    set menu(uid) 0
```

```
}
```

```
proc Menu { label } {
```

```
    global menu
```

```
    if [info exists menu(menu,$label)] {
```

```
        error "Menu $label already defined"
```

```
    }
```

```
    # Create the cascade menu
```

```
    set menuName $menu(menubar).mb$menu(uid)
```

```
    incr menu(uid)
```

```
    menu $menuName -tearoff 1
```

```
    $menu(menubar) add cascade -label $label -menu $menuName
```

```
    # Remember the name to menu mapping
```

```
    set menu(menu,$label) $menuName
```

```
}
```

```
proc Menu_Command { menuName label command } {
```

```
    set m [MenuGet $menuName]
```

```
    $m add command -label $label -command $command
```

```
}
```

```
proc Menu_Check { menuName label var { command {} } } {
```

```
    set m [MenuGet $menuName]
```

```
    $m add check -label $label -command $command -variable $var
```

```
}
```

```
proc Menu_Radio { menuName label var {val {}} {command {}} } {
```

```
    set m [MenuGet $menuName]
```

```

        if {[string length $val] == 0} {
            set val $label
        }
        $m add radio -label $label -command $command -value $val -variable
$var
    }

proc Menu_Separator { menuName } {
    [MenuGet $menuName] add separator
}

proc Menu_Cascade { menuName label } {
    global menu
    set m [MenuGet $menuName]
    if [info exists menu(menu,$label)] {
        error "Menu $label already defined"
    }
    set sub $m.sub$menu(uid)
    incr menu(uid)
    menu $sub -tearoff 0
    $m add cascade -label $label -menu $sub
    set menu(menu,$label) $sub
}

proc MenuGet { menuName } {
    global menu
    if [catch {set menu(menu,$menuName)} m] {
        return -code error "No such menu: $menuName"
    }
    return $m
}

proc Menu_Bind { what sequence menuName label } {
    global menu
    set m [ MenuGet $menuName ]
    if [catch {$m index $label} index] {
        error "$label not in menu $menuName"
    }
    set command [$m entrycget $index -command]
    bind $what $sequence $command
    $m entryconfigure $index -accelerator $sequence
}

```